

“Deep Learning-Based Grasp Pose Detection on Point Clouds and Manipulating the Object using Generated Grasp Pose”

*A thesis submitted in partial fulfillment of
the requirements for the award of the degree of*

**Master of Technology
in
Automation and Robotics**

By
Anushka Singhania
Registration Number: 24-06-14

Supervisor's Name
Prasad Naik
Atul Chavan



**Department of Aerospace Engineering and Autonomous Systems
Defence Institute of Advanced Technology,
Girinagar, Pune 411025**

May 2026

Approval Sheet

Thesis Entitled: “Deep Learning-Based Grasp Pose Detection on Point Clouds and Manipulating the Object using Generated Grasp Pose”

Submitted By: Anushka Singhania (24-06-14)

Is approved for the degree of Master of Technology in “*Aerospace Engineering and Autonomous Systems (Automation and Robotics)*”.

Examiner(s)

Supervisor(s)

Chairman

Date:

Place:

Certificate of Course Work

This is to certify that **ANUSHKA SINGHANIA** has completed the necessary course work as required for the award of the degree of MTech in **Automation and Robotics**. The details of the Course work are as follows:

Sr. No.	Code	Course name	Credits
Semester I			
1	SR 601	Industrial Automation	4
2	SR 602	Introduction to Robotics	4
3	SR 603	Sensors, Actuators and Drives	4
4	SR 605	Introduction to Mechanisms	4
5	SR 610	Introduction to Automatic Control System	4
6	AM 607	Mathematics for Engineers	4
7	PGC 601	Research Methodology and IPR	2
Semester II			
1	SR 607	Robot Dynamics and Control	4
2	SR 608	Machine Vision and Image Processing for Robots	4
3	SR 613	AI and Machine Learning in Robotics	4
4	SR 614	Swarm Robotics	4
5	AP 609	Sensor Technology Laboratory-II	4
6	AM 622	AI for Tech Management	4
7	PGC 602	Communication Skills and Personality Development	2
Semester III			
1	SR 651	M. Tech Dissertation Phase I	14
Semester IV			
1	SR 652	M. Tech Dissertation Phase II	14

Date:

Place: DIAT(DU), Pune

Controller of Examinations

Declaration

I hereby certify that the work which is being presented in the thesis entitled “**Deep Learning-Based Grasp Pose Detection on Point Clouds and Manipulating the Object using Generated Grasp Pose**” in the partial fulfillment of the requirements for the award of the degree of **Master of Technology in Aerospace Engineering and Autonomous Systems (Automation and Robotics)** and submitted in the **Department of Aerospace Engineering and Autonomous Systems, Defence Institute of Advanced Technology, Pune**, is an authentic record of my own work carried out during the time period from July 2024 to April 2026 under the supervision of Prasad Naik, Department of Aerospace Engineering and Autonomous Systems, DIAT Pune and Atul Chavan SC-E at Research and Development Establishment (Engineers) DRDO Pune.

This thesis comprises of original work (except where indicated) carried out by me and due acknowledgments have been made in the text to all other material used. The thesis does not contain any classified information which will be detrimental to national security and has not been submitted in any form for another degree or diploma at any other institute/university.

Anushka Singhanian
Registration No.: 24-06-14

This is to certify that the above statement made by the candidate is correct to the best of our knowledge.

Internal Supervisor
Prasad Naik

External Supervisor
Atul Chavan

Date:

Place:

Certificate

This is to certify that the thesis entitled “**Deep Learning-Based Grasp Pose Detection on Point Clouds and Manipulating the Object using Generated Grasp Pose**”, submitted by **Anushka Singhania (Reg. No.: 24-06-14)** to **Defence Institute of Advanced Technology Pune**, for the award of the degree of **Master of Technology in Aerospace Engineering and Autonomous Systems (Automation and Robotics)** is a bonafide record of research work carried out by her under our supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Supervisor(s):

Prasad Naik

Department of Aerospace Engineering and Autonomous Systems

Defence Institute of Advanced Technology, Pune

Atul Chavan

SC-E

Research and Development Establishment (Engineers) DRDO Pune

Date:

Place:

Acknowledgement

I express my sincere gratitude to Prasad Naik, my project guide at the Defence Institute of Advanced Technology (DIAT), and my previous supervisor Dr. RK Satapathy for their valuable guidance, constant encouragement, and insightful feedback throughout the course of this work. Their expertise, patience, and constructive suggestions played a crucial role in shaping the technical depth and overall direction of this project.

I would also like to extend my heartfelt thanks to my co-guide, Mr. Atul Chavan of R&DE (DRDO), for his continuous support, practical insights, and technical mentorship. His experience and thoughtful inputs greatly enriched the design and development process and helped me overcome many challenges encountered during the implementation of this project.

Finally, I am grateful to the Family, Friends, faculty, staff, and all those who directly or indirectly supported me during the completion of this work.

ABSTRACT

This work adapts and deploys GraspGen, a diffusion-based 6-DoF grasp generation framework, to a novel hardware configuration comprising a Robotiq 3F adaptive gripper, a UR10 robotic arm, and a NVIDIA Jetson AGX Orin edge computing platform. The original framework, designed for symmetric parallel-jaw grippers on high-performance GPU clusters, is systematically adapted to address challenges in dataset generation, edge deployment, perception, and hardware integration.

A gripper-specific dataset was generated for the Robotiq 3F using 227 objects from the Objaverse corpus, yielding 9,685 positive grasp annotations. A novel FCL-free proximity-based labelling method was developed using trimesh.proximity, enabling the full data generation pipeline to run on the edge without an x86 simulation cluster. The generator and discriminator were trained over 500 epochs each, achieving approximately 75% generator loss reduction and a discriminator Average Precision of 0.995 (train) and 0.988 (validation). The on-generator training recipe was implemented in full via a two-phase procedure exposing the discriminator to the diffusion model's specific failure modes.

The perception pipeline uses an OAK-D Pro Wide wrist camera executing a three-pose scanning sequence, fusing partial clouds via robot forward kinematics to eliminate manual extrinsic calibration. SAM2 provides interactive click-based segmentation. The full ROS2 Humble pipeline with MoveIt2 and OMPL was validated in simulation with two complete sequential pick-and-place executions in a cluttered environment. Warm inference on the Jetson averages approximately 1.85 seconds per object, producing 87-108 scored grasp candidates per call. Real-robot evaluation across 80 trials achieved a 66.3% overall grasp success rate (72.5% on isolated objects, 60.0% in clutter), validating sim-to-real transfer under the constrained training and labelling pipeline.

Keywords: Robotic Grasping, 6-DoF Grasp Pose Detection, Diffusion Models, Point Cloud Processing, GraspGen, Robotiq 3F Adaptive Gripper, Edge Deployment, Jetson AGX Orin, Sim-to-Real Transfer, On-Generator Training, SAM2, MoveIt2, ROS2.

Contents

Approval Sheet.....	i
Certificate of Course Work	ii
Declaration	iii
Certificate	iv
Acknowledgement.....	v
ABSTRACT	vi
Contents.....	vii
List of Figures	ix
List of Tables.....	xi
List of symbols	xii
CHAPTER 1.....	1
INTRODUCTION.....	1
CHAPTER 2.....	4
LITERATURE SURVEY	4
2.1 Classical Approaches to Grasp Planning.....	4
2.2 Transition to Learning-Based Grasp Detection.....	5
2.3 6-DoF Grasp Detection Using Point Clouds	6
2.4 Grasping Under Occlusion and Partial Observations.....	8
2.5 End-to-End Versus Sampling-Based Pipelines	11
2.6 Generative Deep Learning Models	12
2.8 Grasp Representation Through Graph Neural Networks	20
2.9 Language-Guided Grasping	20
2.10 Multimodal Grasping: Visual, Tactile, and Force-Torque Integration.....	21
2.11 Datasets for Grasp Pose Learning	21
.....	24
2.12 Sim-to-Real Transfer Challenges	24
2.13 Literature Gap	25
CHAPTER 3.....	30
PROBLEM FORMULATION & METHODOLOGY	30
3.1 Problem Definition.....	30
3.2 Scope and Objectives	31
3.3 Methodology	32
CHAPTER 4.....	66
RESULT AND DISCUSSION.....	66
4.1 Generator Training Results	66

4.2 Discriminator Training Results	72
4.3 On-Policy Discriminator Training Results.....	81
4.4 Inference Performance on Point Clouds.....	90
4.5 Real-Robot Grasp Success Results	92
4.6 Simulation Validation	94
4.7 Sim-to-Real Transfer Analysis.....	94
CHAPTER 5.....	96
CONCLUSION AND FUTURE SCOPE	96
5.1 Conclusion.....	96
5.2 Limitations	98
5.3 Future Scope.....	99
PUBLICATION	101
REFERENCES.....	102
APPENDICES.....	105
APPENDIX A	105
APPENDIX B	106
APPENDIX C	108
APPENDIX D	109
APPENDIX E.....	110
Declaration for Plagiarism Check	111

List of Figures

FIG.1. OCCLUSION HANDLING STRATEGIES IN GRASP POSE DETECTION	9
FIG. 2. PERFORMANCE VS. OCCLUSION LEVEL	11
FIG.3. TAXONOMY OF GENERATIVE DEEP LEARNING FRAMEWORKS FOR GRASP POSE DETECTION, ILLUSTRATING VAE, GAN, DIFFUSION, AND FLOW-BASED MODEL FAMILIES	13
FIG.4. VAE-BASED GRASP GENERATION FLOW	13
FIG.5. GAN-BASED FRAMEWORK	14
FIG.6. DIFFUSION-BASED GRASP GENERATION PIPELINE SHOWING PROGRESSIVE DENOISING FROM NOISE TO VALID GRASP POSES	15
FIG.7. FLOW-BASED / HYBRID GENERATIVE GRASPING FRAMEWORK	19
FIG.8. DATASET AND EVALUATION FLOWCHART FOR GRASP POSE DETECTION	23
FIG.9. SYSTEM ARCHITECTURE	33
FIG.10. HARDWARE SETUP WITH PICK AND PLACE TRAY	35
FIG.11. UR10 ARM	36
FIG.12. TEACH PENDANT	36
FIG.13. UR10 WITH ROBOTIQ GRIPPER MOUNDED ON IT IN RVIZ	36
FIG.14. ROBOTIQ 3F ADAPTIVE GRIPPER [44]	37
FIG.15. ROBOTIQ 3F GRIPPER AND OAK-D PRO WIDE WRIST CAMERA MOUNTED ON THE CUSTOM 3D-PRINTED ADAPTER SLAB ON THE UR10 TOOL FLANGE	38
FIG.16. NVIDIA JETSON AGX ORIN 64GB [45]	39
FIG.17. FULL END-TO-END MANIPULATION PIPELINE WITH ROS2 TOPIC NAMES	41
FIG.18. GRASP POSE GENERATION AND LABELLING IN ISAAC SIM WITH ROBOTIQ 3F GRIPPER	45
FIG.19. GRASP SAMPLING AND FCL-FREE LABELLING FLOWCHART	46
FIG.20. THREE-STAGE ON-GENERATOR TRAINING RECIPE	52
FIG.21. GENERATION OF POINT CLOUDS AND GRASP POSES	54
FIG.22. SAM2 SEGMENTATION NODE SHOWING OPERATOR LEFT-CLICK ON TARGET OBJECT (GREEN MASK) AND RVIZ2 WORLD-FRAME POINT CLOUD	55
FIG.23. RVIZ2 PLANNING SCENE SHOWING UR10 ARM, PICK TRAY (BROWN), PLACE TRAY (YELLOW) AND TARGET OBJECTS	59
FIGURE 24. EIGHT-STEP PICK-AND-PLACE EXECUTION SEQUENCE	62
FIG.25. INTEGRATED SYSTEM EXECUTION SHOWING SEGMENTATION, PLANNING, AND ROS2 NODE COMMUNICATION	63
FIG.26. ROS2 COMPUTATION GRAPH SHOWING INTERACTION BETWEEN PERCEPTION, GRASP GENERATION, AND EXECUTION NODES	63
FIG.27. GENERATOR TRAINING AND VALID TOTAL LOSS	68
FIG.28. GENERATOR TRAINING AND VALIDATION NOISE PREDICTION LOSS	69
FIG.29. GENERATOR TRAINING AND VALIDATION TRANSLATION L2 ERROR	70
FIG.30. GENERATOR TRAINING AND VALIDATION GEODESIC ROTATION ERROR	71
FIG.31. GENERATOR TRAINING AND VALIDATION PHI3 ROTATION ERROR	72
FIG.32. GENERATOR VALIDATION RECONSTRUCTION RECALL	72
FIG.33. DISCRIMINATOR TOTAL LOSS: TRAINING AND VALIDATION	75
FIG.34. BCE TOPK LOSS: TRAINING AND VALIDATION	75
FIG.35. AVERAGE PRECISION (AP): TRAINING AND VALIDATION	76
FIG.36. TOPK RATIO TRUE POSITIVE GRASPS: TRAINING AND VALIDATION	77
FIG.37. TOPK RATIO TRUE NEGATIVE GRASPS: TRAINING AND VALIDATION	77
FIG.38. LOSS AND TOPK RATIO NEGATIVE FREE-SPACE GRASPS : TRAINING AND VALIDATION	79
FIG.39. TOPK RATIO COLLIDING GRASPS: TRAINING AND VALIDATION	79
FIG.40. LOSS ON POSITIVE GRASPS: TRAINING AND VALIDATION	80
FIG.41. LOSS ON NEGATIVE GRASPS: TRAINING AND VALIDATION	81

FIG.42. ON-POLICY TOTAL LOSS: TRAINING AND VALIDATION	82
FIG.43. ON-POLICY COLLIDING GRASPS: TRAINING AND VALIDATION	83
FIG.44. ON-POLICY BCE TOPK LOSS: TRAINING AND VALIDATION	83
FIG.45. ON-POLICY LOSS ON POSITIVE TRUE GRASPS	84
FIGURE 46. ON-POLICY LOSS ON NEGATIVE TRUE GRASPS.....	84
FIG.47. ON-POLICY LOSS NEGATIVE TRUE: TRAINING AND VALIDATION.....	85
FIG.48. ON-POLICY TOPK RATIO NEGATIVE TRUE: TRAINING AND VALIDATION.....	85
FIG.49. ON-POLICY TOPK RATIO POSITIVE TRUE: TRAINING AND VALIDATION	86
FIG.50. ON-POLICY LOSS NEGATIVE FREESPACE: TRAINING AND VALIDATION.....	87
FIG.51. ON-POLICY TOPK RATIO NEGATIVE TRUE: TRAINING AND VALIDATION	87
FIG.52. ON-POLICY LOSS POSITIVE TURE: TRAINING AND VALIDATION	88
FIG.53. ON-POLICY AP: TRAINING AND VALIDATION	89
FIG. 54. ON-POLICY TOPK RATIO NEGATIVE FREESPACE: TRAINING AND VALIDATION.....	89
FIG.55. ON-POLICY TOPK RATIO POSITIVE TRUE: TRAINING AND VALIDATION	90
FIG.56. UR10 ARM EXECUTING A REAL GRASP ATTEMPT ON THE PICK TRAY IN ISOLATED ENVIRONMENT	92
FIG.57. UR10 ARM EXECUTING A REAL GRASP ATTEMPT ON THE PICK AND PLACE TRAY DURING HARDWARE EVALUATION	93

List of Tables

TABLE 1: COMPARISON OF OCCLUSION HANDLING ARCHITECTURES	10
TABLE 2: REPRESENTATIVE GENERATIVE GRASPING APPROACHES.....	16
TABLE 3: QUANTITATIVE COMPARISON OF PURELY GENERATIVE GRASPING METHODS	17
TABLE 4: COMPUTATIONAL EFFICIENCY COMPARISON.....	18
TABLE 5: QUANTITATIVE COMPARISON OF HYBRID GENERATIVE GRASPING METHODS	18
TABLE 6: SYNTHETIC DATASETS COMPARISON.....	23
TABLE 7: REAL-WORLD DATASETS COMPARISON.....	24
TABLE 8: DATASET STATISTICS AND TRAINING CONFIGURATION FOR ALL THREE TRAINED MODELS	47
TABLE 9: SCAN POSE JOINT ANGLES MEASURED ON TEACH PENDANT. ONLY WRIST_2 CHANGES BETWEEN POSES 2 AND 3	56
TABLE 10: COMPARISON OF SIMULATION AND REAL HARDWARE PERCEPTION PIPELINES	57
TABLE 11: GRASPGEN INFERENCE PERFORMANCE ON JETSON AGX ORIN	91
TABLE 12: PRECISION COVERAGE COMPARISON BETWEEN GT AND ON-POLICY DISCRIMINATORS ON THE 46-OBJECT VALIDATION SET	91
TABLE 13: REAL-WORLD GRASP EXECUTION RESULTS	93
TABLE 14: FAILURE MODE DISTRIBUTION ACROSS REAL-WORLD TRIALS.....	94
TABLE 15: SIMULATION VALIDATION RESULTS	94
TABLE 16: QUANTITATIVE COMPARISON WITH EXISTING METHODS	95
TABLE 17: HARDWARE COMPONENT SPECIFICATIONS.....	105
TABLE 18: SOFTWARE STACK	106
TABLE 19: ROS2 TOPIC SUMMARY	108
TABLE 20: INFERENCE PARAMETERS	110

List of symbols

$\text{SE}(3)$	Special Euclidean group denoting 6-DoF poses and transformations
$\text{SO}(3)$	Special Orthogonal group denoting 3D rotation representation
$\mathbb{R}^3$	3D Euclidean space denoting translation representation
$\mathcal{X}$	Object-centric point cloud input
g	6-DoF grasp pose
$\mathcal{G}^+$	Ground-truth positive grasp poses
t	Diffusion timestep
T	Total number of diffusion timesteps
$\tilde{g}$	Noised grasp pose after adding Gaussian noise
ϵ	Ground-truth noise
φ	Noise prediction network
$\mathcal{L}$	Denoising loss training objective
κ	Translation normalisation multiplier
t_i	Translation component across positive grasp poses
N	Total number of training objects
K	Camera intrinsic matrix
f_x, f_y	Focal lengths along x and y axes
c_x, c_y	Principal point coordinates
(u, v)	Image pixel coordinates
d	Depth value or distance to object surface
X, Y, Z	3D point coordinates in the camera or world frame
σ	Standard deviation used for statistical outlier removal thresholds

CHAPTER 1

INTRODUCTION

Robotic grasping forms the foundation of autonomous manipulation and is central to enabling robots to interact meaningfully with the physical world. Whether in industrial automation, service robotics, or assistive technologies, the ability to reliably grasp objects under varying conditions remains a critical requirement. However, despite substantial advances in perception systems, motion planning, and machine learning, achieving robust grasping in real-world environments continues to be a challenging problem. This is primarily due to the inherent complexity of real-world scenes, where objects may be partially visible, irregular in shape, or surrounded by clutter.

Traditional grasp planning approaches are largely based on geometric reasoning, analytical models, and physical principles such as force-closure and contact stability. These methods assume accurate and complete knowledge of object geometry, often relying on precise 3D models or dense point cloud representations. While effective in controlled environments, such assumptions rarely hold in practical scenarios. In real-world settings, sensor data is frequently incomplete, noisy, or distorted due to occlusions, reflective surfaces, or limited viewpoints. As a result, classical methods tend to degrade significantly when applied to sparse or partial point cloud data, limiting their applicability in unstructured environments [1], [2].

The limitations of analytical methods have led to the increasing adoption of deep learning-based approaches for grasp detection. Early learning-based systems introduced discriminative models that map sensory inputs such as RGB-D images or point clouds

directly to grasp poses or grasp quality scores. These methods improved generalization compared to purely geometric techniques and demonstrated the ability to handle moderate variations in object shape and pose. However, they still face significant challenges when dealing with severe occlusions, incomplete observations, and previously unseen object categories. More importantly, grasping is inherently a multi-modal problem, where multiple valid grasp configurations may exist for a single object. Deterministic models that predict a single output often fail to capture this diversity, resulting in suboptimal or unstable grasps [3], [4].

To overcome these limitations, recent research has shifted toward generative deep learning frameworks that model the distribution of feasible grasps rather than predicting a single solution. Approaches based on Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), diffusion models, and transformer architectures have shown promising results in capturing the uncertainty and diversity of grasp configurations. These models are particularly effective in reasoning under incomplete information, as they can infer missing geometric structures and generate multiple plausible grasp candidates from partial observations. Diffusion-based methods, in particular, have gained attention due to their ability to iteratively refine noisy grasp hypotheses into stable and physically valid configurations, leading to improved robustness in complex and cluttered environments[5], [6], [7] .

Despite these advancements, several practical challenges remain when deploying such models on real robotic systems. Point cloud data acquired from depth sensors is often affected by noise, varying resolution, and missing regions, especially in the case of transparent, reflective, or texture-less objects. While multi-sensor fusion techniques can improve perception quality, they introduce additional complexities such as calibration errors and synchronization issues. Furthermore, a persistent gap exists between simulation and real-world performance, commonly referred to as the sim-to-real gap. Models trained in simulation often fail to generalize effectively due to differences in sensor characteristics, physical dynamics, and environmental variability[8], [9].

In this context, the present work focuses on the development of a deep learning–based grasp pose detection pipeline tailored specifically for point cloud data. The proposed system integrates modern generative grasp prediction techniques with a complete robotic manipulation framework. The hardware setup consists of a UR10 robotic manipulator

equipped with a Robotiq adaptive gripper, supported by a system that includes OAK-D Pro Wide depth sensing point cloud acquisition. The computational backbone is powered by the NVIDIA Jetson AGX Orin platform, enabling efficient real-time inference and control.

The overall pipeline is designed to bridge the gap between perception and action. It begins with object perception and point cloud acquisition, followed by grasp candidate generation using a generative deep learning model. These grasp hypotheses are then evaluated and filtered based on feasibility and collision constraints. This integrated approach ensures that grasp predictions are not only theoretically valid but also executable within the constraints of a real robotic system.

A key motivation of this work is to move beyond isolated grasp prediction models and develop an end-to-end system capable of operating reliably in real-world conditions. While existing research has demonstrated impressive performance on benchmark datasets, practical deployment requires addressing additional challenges such as sensor noise, partial visibility, dynamic environments, and hardware limitations. By combining generative grasp modeling with a complete robotic manipulation pipeline and validating it on an industrial-grade platform, this project aims to contribute toward the development of more robust and deployable robotic grasping systems.

Furthermore, this work builds upon recent advances in diffusion-based grasp generation, particularly the GraspGen framework, which models grasp synthesis as a probabilistic generation process over 6-DoF poses. By leveraging such approaches, the system aims to improve grasp diversity, robustness, and adaptability, especially in scenarios involving incomplete or noisy point cloud data [5].

CHAPTER 2

LITERATURE SURVEY

Robotic grasping has evolved through contributions from classical mechanics, machine perception, learning-based vision, and generative deep learning. The field has progressively shifted from analytically derived solutions toward data-driven and probabilistic approaches capable of operating under uncertainty, driven largely by the challenges of unstructured environments partial observations, sensor noise, and unseen object geometries. The increasing availability of 3D sensing technologies has further emphasized grasp detection directly from point cloud data as a more realistic representation of real-world perception.

This chapter reviews existing literature on grasp pose detection, following the historical and methodological evolution from classical analytical approaches to modern learning-based and generative methods, highlighting their strengths, limitations, and relevance to grasping under incomplete and noisy observations.

2.1 Classical Approaches to Grasp Planning

Early research in robotic grasping was grounded in analytical formulations derived from rigid-body mechanics and contact modeling, using criteria such as force-closure, wrench space analysis, and the Ferrari–Canny stability measure [10]. These methods required precise knowledge of contact points, surface normals, friction coefficients, and full 3D object models assumptions that rarely hold in practice. Real-world depth sensors frequently produce incomplete or noisy point clouds, causing significant degradation in grasp quality estimation [2].

Classical approaches also operate deterministically, producing a single grasp configuration and failing to model the multimodal nature of grasp affordances. They do not generalise across diverse object categories, requiring manual tuning for each new geometry [11]. These limitations sensitivity to incomplete observations, lack of uncertainty modeling, and poor scalability motivated the shift toward data-driven frameworks capable of learning grasp representations directly from sensory data.

2.2 Transition to Learning-Based Grasp Detection

The increasing availability of affordable RGB-D sensors significantly accelerated the shift toward vision-based grasp detection methods, enabling robots to perceive both color and depth information in real time. This advancement allowed grasp planning to move away from purely geometric modeling toward data-driven approaches that operate directly on sensory inputs. Early learning-based methods primarily relied on convolutional neural networks (CNNs) trained on RGB-D images to predict grasp configurations in 2D or 2.5D representations. These approaches framed grasp detection as a regression or classification problem, where the network outputs grasp rectangles or quality scores corresponding to feasible grasp regions. Notable works such as those discussed in [12] demonstrate that such models can outperform classical analytical methods in cluttered environments by learning visual patterns associated with successful grasps.

However, these early approaches were inherently limited to planar grasp representations, typically constrained to 4-DoF configurations (position, orientation, and gripper width in the image plane). This restriction significantly limited their applicability in real-world manipulation tasks, where full 6-DoF grasping is required to account for arbitrary object orientations and spatial arrangements.

The introduction of deep learning architectures designed specifically for point cloud processing marked a major advancement in grasp detection. Models such as PointNet and PointNet++ enabled direct learning from raw 3D point sets by providing permutation-invariant feature extraction and hierarchical geometric reasoning. These architectures, as discussed in [13], allow neural networks to capture both local and global geometric structures without requiring explicit surface reconstruction. As a result, grasp detection systems began to operate directly in 3D space, enabling more accurate estimation of grasp poses in six degrees of freedom.

Building upon these representations, several methods extended grasp detection pipelines to predict grasp quality scores or poses directly from point clouds. For instance, approaches discussed in [4] demonstrate how learning-based models can infer contact points and generate grasp candidates in cluttered scenes using raw geometric input. Similarly, large-scale datasets such as those described in [14] enabled training of deep models capable of generalizing across diverse objects and environments.

Despite these advancements, most learning-based grasp detection methods remained fundamentally discriminative in nature. They typically predict a single grasp pose or assign scores to a limited set of candidates, which restricts their ability to capture the inherently multimodal nature of grasp affordances. In practice, multiple valid grasp configurations often exist for a single object, depending on viewpoint, accessibility, and task constraints. This limitation is emphasized in [5], where it is noted that deterministic prediction frameworks fail to represent the full distribution of feasible grasps.

In addition, these models exhibit significant performance degradation when operating on incomplete or partial observations, a common scenario in real-world environments due to occlusions and limited sensor viewpoints. Single-view depth sensing often results in sparse and missing regions in point clouds, making it difficult for discriminative models to infer stable grasp configurations. Studies such as [15] highlight the challenges of generalization under such conditions, particularly when models encounter unseen object geometries or noisy sensory inputs.

2.3 6-DoF Grasp Detection Using Point Clouds

A substantial body of research has focused on estimating full 6-DoF grasp poses directly from 3D point clouds, enabling robots to operate in unconstrained environments where objects can assume arbitrary orientations. Unlike planar grasp detection, these approaches aim to predict both position and orientation in 3D space, making them significantly more suitable for real-world manipulation tasks.

One of the foundational methods in this domain is Grasp Pose Detection (GPD), which operates by sampling a large number of candidate grasp configurations from local regions of the point cloud and evaluating them using learned geometric descriptors. As described in [2], GPD introduced an effective pipeline for identifying feasible grasps in cluttered scenes. However, its reliance on exhaustive candidate sampling leads to high computational cost, limiting its applicability in real-time robotic systems.

To address these limitations, learning-based extensions such as PointNetGPD were developed, where grasp quality is predicted directly from point-level features without relying solely on handcrafted geometric descriptors. These methods build upon architectures designed for unordered point cloud processing, enabling more efficient feature extraction and improved generalization. Further advancements leveraging hierarchical feature learning, as discussed in [13], allow networks to capture both local surface structure and global object geometry, improving grasp prediction accuracy in complex scenes.

End-to-end approaches further streamlined the grasp detection pipeline by removing explicit candidate sampling. For instance, the Volumetric Grasping Network (VGN), detailed in [16], employs voxelized occupancy grids and 3D convolutional networks to predict dense grasp affordances across the workspace. This formulation enables faster inference compared to sampling-based methods while maintaining robustness in cluttered environments.

More recent approaches have explored implicit representations and contact-based reasoning to enhance grasp prediction. Contact-GraspNet, presented in [4], directly infers contact points and generates 6-DoF grasp poses from scene-level point clouds. By modeling contact interactions explicitly, it improves grasp quality in cluttered and partially occluded scenes, particularly for parallel-jaw grippers.

Despite these advancements, most of these methods remain fundamentally discriminative, predicting either a single grasp pose or assigning scores to a limited set of candidates. This restricts their ability to capture the full distribution of feasible grasps, especially in scenarios where multiple valid grasp configurations exist. Additionally, their performance degrades significantly when large portions of the object are not visible to the sensor. Sparse or incomplete point clouds, which are common in single-view depth sensing, introduce ambiguity that these models struggle to resolve. This limitation is further highlighted in [15], where robustness to partial observations remains an open challenge.

These challenges have motivated the exploration of approaches that go beyond deterministic prediction, focusing instead on modeling uncertainty, inferring missing geometry, and generating diverse grasp candidates from incomplete observations.

2.4 Grasping Under Occlusion and Partial Observations

Occlusion remains one of the most significant challenges in grasp pose detection, particularly in real-world environments where objects are rarely fully visible. Depth sensing systems frequently produce incomplete point clouds due to self-occlusion, limited viewpoints, reflective or transparent surfaces, and sensor noise. As a result, large portions of object geometry may be missing, making it difficult to infer stable grasp configurations. Conventional learning-based models, including CNN-based and PointNet-based approaches, typically rely on visible surface geometry and therefore exhibit substantial performance degradation under such conditions. This limitation has been widely discussed in [3] and [17], where incomplete observations are identified as a primary source of grasp prediction failure.

To address this issue, several strategies have been proposed, broadly focusing on reconstructing missing geometry, selectively attending to informative regions, or aggregating multiple observations. These approaches are illustrated conceptually in Fig. 1, where different pipelines are designed to compensate for incomplete point cloud data through reconstruction, attention mechanisms, or multi-view fusion.

Point cloud completion has emerged as a widely adopted solution for handling occlusion. The Point Completion Network (PCN), introduced in [18], employs a coarse-to-fine reconstruction strategy to infer plausible full object geometry from partial inputs. By learning shape priors from large datasets, such models are able to generate complete point clouds that can subsequently be used for grasp prediction. Building upon this idea, PCF Grasp, described in [19], integrates point cloud completion with grasp detection, demonstrating improved grasp stability and success rates in heavily occluded environments.

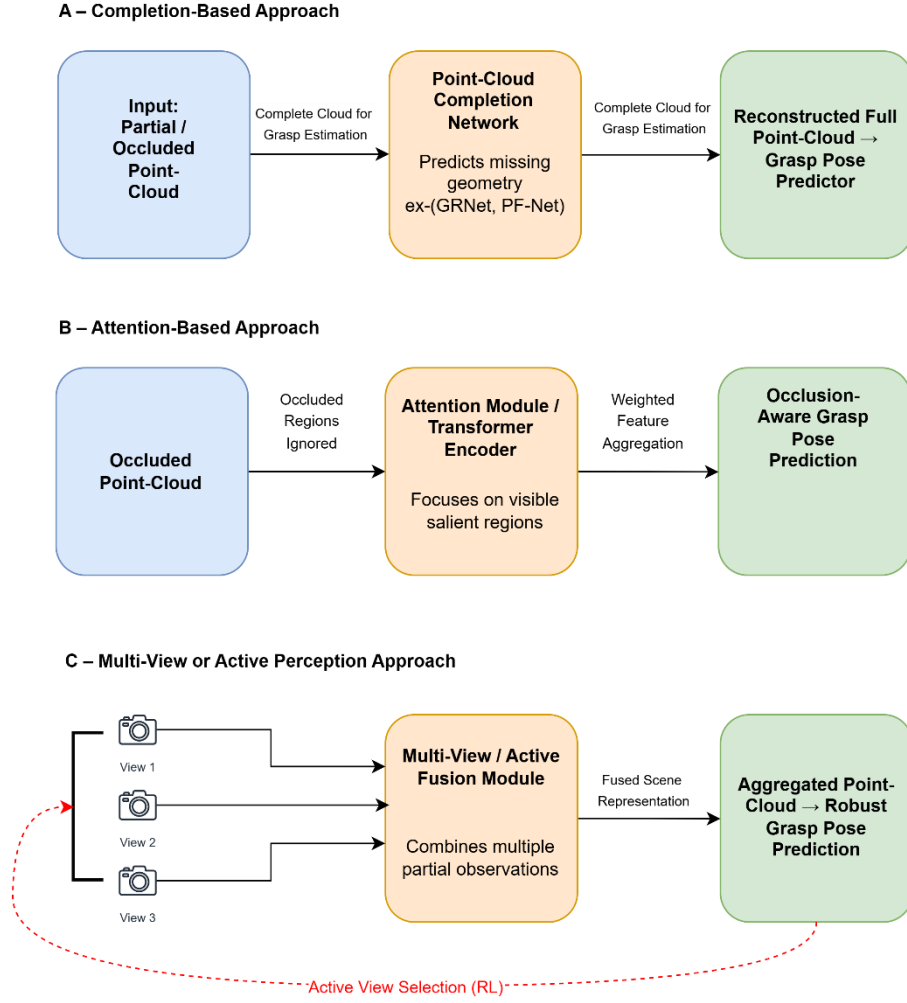


Fig.1. Occlusion Handling Strategies in Grasp Pose Detection

In parallel, attention-based and context-aware models have been developed to enhance robustness without explicitly reconstructing missing geometry. Methods such as those proposed in [20] utilize attention mechanisms to focus on salient and reliable regions of the observed point cloud, reducing the impact of missing data. Similarly, transformer-based architectures discussed in [21] enable global context reasoning, allowing the model to infer relationships between visible and partially occluded regions.

Another important direction involves multi-view and active perception strategies, where multiple observations of the same scene are fused to construct a more complete representation. As shown in Fig. 1, multi-view fusion modules aggregate partial observations captured from different viewpoints, resulting in a denser and more informative point cloud. Approaches such as those discussed in [22] incorporate sequential observations and reinforcement learning-based view selection to actively reduce occlusion and improve grasp success rates.

The comparative effectiveness of these approaches is summarized in Table 1, which highlights improvements in grasp success rates under varying levels of occlusion. Furthermore, Fig. 2 illustrates the relative robustness of different grasp generation models, showing that diffusion-based methods tend to exhibit higher resilience to occlusion compared to purely discriminative approaches, owing to their ability to model uncertainty and infer missing geometry.

TABLE 1: COMPARISON OF OCCLUSION HANDLING ARCHITECTURES

Method	Input Modality	Planning Algorithm	Reported Grasp Success Improvement
Grasp-the-Graph	Scene point cloud	Graph-based next-best-view selection	+12% GSR under 60% occlusion
TransSC	Sequential partial point clouds	Spatiotemporal fusion + active NBV	+15% GSR, -25% reconstruction error
Contact-GraspNet	RGB-D + tactile contacts	Contact-aware grasp ranking	+18% GSR in cluttered scenes
Single-View Scene $\rightarrow$ Point Cloud	Single RGB-D frame	Hypothesis refinement via one additional view	+10% GSR with one extra view
DORA	RGB-D + affordance maps	RL-based active perception policy	+14% GSR under heavy occlusion

Despite these advancements, significant challenges remain. Completion-based methods may introduce geometric artifacts or hallucinated structures that do not correspond to the true physical object, potentially leading to unstable grasps. Attention-based methods, while more efficient, are limited by the quality of the visible regions and may fail when critical grasp features are entirely occluded. Multi-view approaches improve completeness but require additional sensing, calibration, and planning overhead, making them less suitable for time-constrained applications.

Overall, grasping under occlusion continues to be an open problem, requiring models that can effectively reason under uncertainty, integrate partial observations, and generate robust grasp hypotheses without relying on complete geometric information.

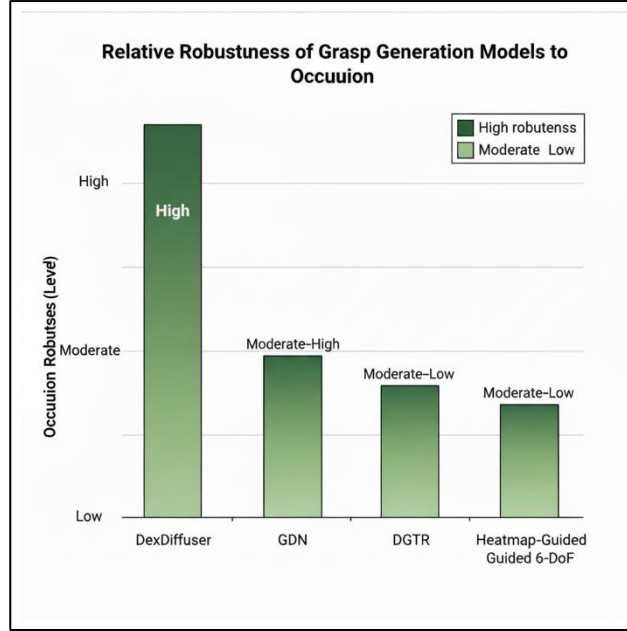


Fig. 2. Performance vs. Occlusion Level

2.5 End-to-End Versus Sampling-Based Pipelines

Grasp detection pipelines can be broadly categorized into sampling-based and end-to-end approaches, each offering distinct advantages and limitations in terms of accuracy, computational efficiency, and suitability for real-time applications.

Sampling-based methods, such as the Grasp Pose Detection (GPD) framework described in [2], operate by generating a large number of candidate grasp configurations from local regions of the point cloud. These candidates are subsequently evaluated using learned scoring functions or geometric heuristics to identify feasible grasps. While this approach is general and capable of handling diverse object geometries, it is inherently computationally expensive due to the need for exhaustive candidate generation and evaluation. As a result, such pipelines often struggle to meet real-time constraints, particularly in industrial or dynamic environments where rapid decision-making is required.

In contrast, end-to-end learning-based architectures eliminate the explicit sampling stage by directly predicting grasp parameters from learned feature representations. These models leverage deep neural networks to infer grasp poses at either the point level or voxel level, significantly reducing inference time. Approaches such as those discussed in [23] highlight the shift toward direct prediction frameworks that learn mappings from sensory inputs to grasp configurations without intermediate candidate enumeration. Similarly,

methods like REGNet and other per-point prediction models estimate grasp parameters directly for each point in the cloud, enabling more efficient and scalable grasp detection.

Recent advancements have further improved the efficiency of end-to-end pipelines through the incorporation of spatial reasoning mechanisms. Heatmap-based approaches, such as the method presented in [24], utilize semantic heatmaps to identify promising grasp regions, followed by local feature refinement for precise pose estimation. By focusing computational resources on high-probability regions, these methods achieve a balance between speed and accuracy, making them particularly suitable for real-time applications in cluttered environments.

Despite these improvements, both paradigms exhibit inherent trade-offs. Sampling-based methods offer greater flexibility and can explore a wider range of grasp candidates but suffer from high computational cost. End-to-end methods, while efficient, may be limited by their reliance on learned representations and can struggle to generalize in highly complex or unseen scenarios. Consequently, the choice of pipeline often depends on the specific requirements of the application, including computational constraints, environmental complexity, and the level of robustness required.

2.6 Generative Deep Learning Models

Generative deep learning models represent a significant shift in grasp pose detection, particularly in scenarios involving occlusion, uncertainty, and incomplete observations. Unlike discriminative approaches that predict a single grasp configuration, generative models aim to learn the underlying distribution of feasible grasps, enabling the synthesis of multiple valid 6-DoF grasp poses from partial point cloud inputs. This capability is critical in real-world environments, where multiple grasping strategies may exist depending on object geometry, accessibility, and task constraints.

A conceptual overview of generative approaches is illustrated in Fig. 3, which categorizes existing methods into Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), diffusion-based models, and flow-based methods. These frameworks

differ in how they represent uncertainty, generate grasp hypotheses, and handle incomplete observations.

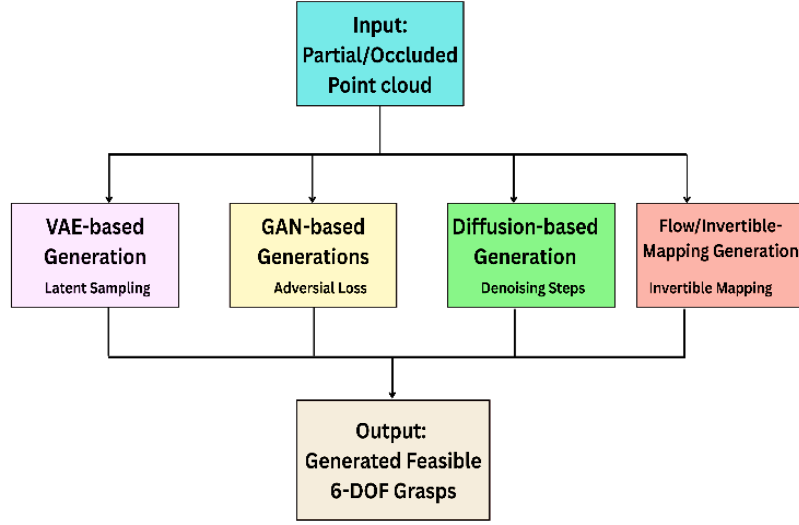


Fig.3. Taxonomy of generative deep learning frameworks for grasp pose detection, illustrating VAE, GAN, diffusion, and flow-based model families

2.6.1 Variational Autoencoder (VAE) Approaches

Variational Autoencoders model grasp generation through a latent variable framework, where grasp poses are encoded into a continuous latent space and sampled to produce diverse grasp candidates. As shown in Fig. 4, an encoder network extracts latent representations from point cloud inputs, and a decoder reconstructs grasp pose parameters from sampled latent variables.

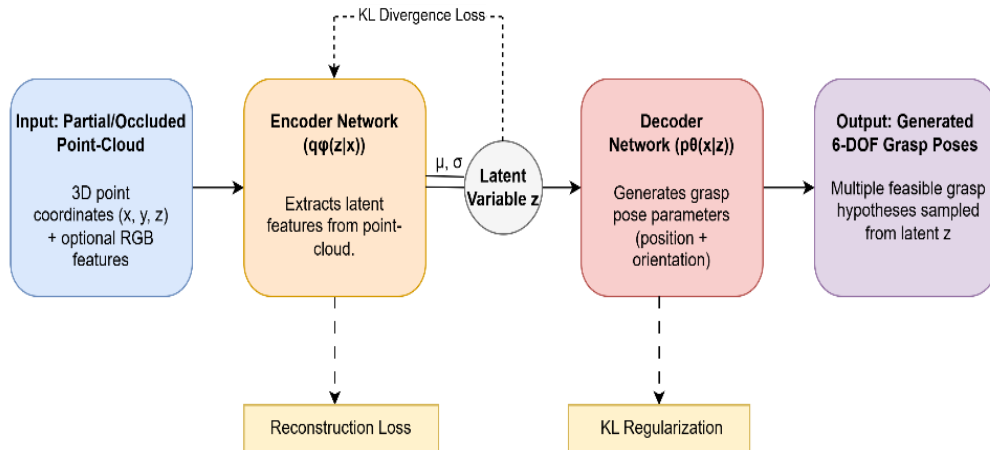


Fig.4. VAE-Based Grasp Generation Flow

Methods such as those presented in [12] demonstrate that VAE-based models can generate multiple grasp hypotheses from a single observation, capturing variability in grasp configurations. However, the regularization imposed by the Kullback–Leibler divergence often leads to over-smoothed outputs, reducing the precision of generated grasps. This limitation is also noted in [6], where VAE-based approaches struggle to model sharp and physically accurate grasp distributions.

2.6.2 GAN-Based Methods

Generative Adversarial Networks employ an adversarial training mechanism in which a generator produces grasp candidates while a discriminator evaluates their realism and feasibility. As illustrated in Fig. 5, this interaction encourages the generation of sharper and more realistic grasp configurations compared to VAE-based methods.

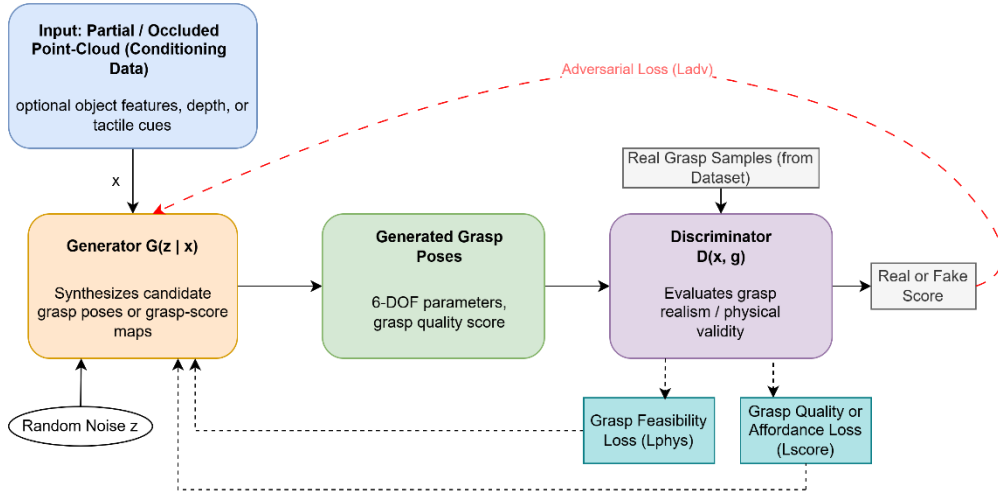


Fig.5. GAN-based framework

Approaches such as [25] explore adversarial learning for improving grasp prediction, particularly in challenging scenarios involving transparent or reflective objects. While GANs can enhance output quality, they suffer from well-known issues such as training instability, mode collapse, and sensitivity to hyperparameter selection. These limitations restrict their applicability in grasp synthesis tasks where consistent and diverse output generation is required, leading to a gradual shift toward more stable generative frameworks.

2.6.3 Diffusion Models

Diffusion-based models represent a major advancement in generative grasp synthesis and are currently regarded as state-of-the-art for modeling complex grasp distributions. These models iteratively refine random noise into structured grasp poses through a learned denoising process, as illustrated in Fig. 6. This formulation allows the model to

progressively recover geometric structure and generate physically plausible grasp configurations conditioned on partial point cloud inputs.

Several recent works demonstrate the effectiveness of diffusion models for robotic grasping. The Grasp Diffusion Network, presented in [7], formulates grasp generation in SE(3) space, enabling robust 6-DoF pose prediction from partial observations. DexDiffuser, described in [6], extends this framework to dexterous multi-finger grasping, achieving high performance in simulation while maintaining adaptability across diverse object geometries.

Forward (Training) Process

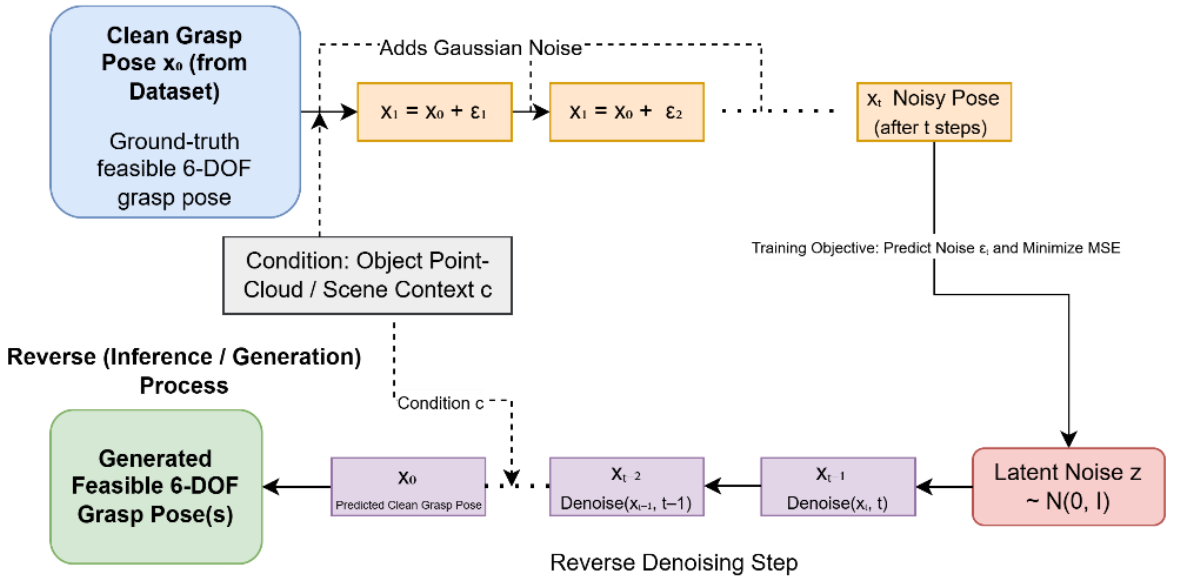


Fig. 6. Diffusion-based grasp generation pipeline showing progressive denoising from noise to valid grasp poses

GraspGen, introduced in [5], further enhances this paradigm by integrating transformer-based architectures with diffusion modeling, enabling efficient generation and filtering of grasp candidates through an on-generator evaluation mechanism. Additionally, diffusion-based approaches have been extended to human grasp modeling in single-view settings, as explored in [12], demonstrating their versatility across different grasping scenarios.

The comparative performance of these approaches, summarized in Table 2, indicates that diffusion-based models consistently achieve higher grasp success rates and improved robustness under partial observations compared to earlier generative and discriminative methods. A more detailed comparison presented in Table 3 further highlights the strengths of diffusion-based frameworks such as [6] and [5] particularly in terms of grasp diversity,

generalization across object categories, and adaptability to complex grasp configurations.

TABLE 2: REPRESENTATIVE GENERATIVE GRASPING APPROACHES

Method	Architecture	Dataset	Input Representation	Output Representation	Performance (Success Rate/Metric)
Grasp Diffusion Network	Diffusion Model (DDPM in $SO(3) \times \mathbb{R}^3$)	ACRONYM (CAT10 subset, 567 objects, 1.1M grasps)	Partial Point Cloud	6-DOF Grasp (parallel-jaw)	90% (sim), 90% (real, best config)
DexDiffuser	Diffusion Model (DexSampler + DexEvaluator)	DexGraspNet (1.7M grasps, 5378 objects)	Partial Point Cloud	16-DOF Grasp (Allegro Hand)	98.77% (sim), 68.89% (real)
Dexterous Grasp Transformer (DGTR)	Transformer-based (DETR-like)	DexGraspNet	Complete Point Cloud	22-DOF Grasp (Shadow Hand)	98.77% (sim), 66.6% (real)
GraspGen	Diffusion-Transformer (DiT)	GraspGen Dataset (53M grasps, 36K objects)	Partial/Complete Point Cloud	6-DOF Grasp (multi-gripper)	81.3% (real robot overall) 65.3% (FetchBench task success),
Contact-GraspNet	Contact-based CNN	GraspNet-1Billion	Scene Point Cloud	6-DOF Grasp	26.1% (FetchBench task success)
Efficient Heatmap-Guided Grasp	Heatmap-based CNN	GraspNet-1Billion, TS-ACRONYM	RGB-D Image	6-DOF Grasp	28 ms inference, 64.45% AP (GraspNet-1B seen)
Constrained 6-DoF Grasp (CGDF)	Diffusion Model + Convolutional Plane Features	DA2 Dataset	Point Cloud (full/partial)	6-DOF Grasp (dual-arm)	60.3% GSR (sim), 43.51% FC (unconstrained)
GraspNet-1Billion Baseline	PointNet++ + Grasp Sampling	GraspNet-1Billion	Scene Point Cloud	6-DOF Grasp	37.7% AP (seen), 19.7% AP (similar), 9.6% AP (novel)

In addition, Table 5 provides a comparative analysis of hybrid and non-generative approaches, showing that while methods such as [4] and heatmap-based models offer faster inference and real-time feasibility, they often exhibit reduced performance in novel or highly occluded scenarios. This contrast reinforces the advantage of generative models in capturing the multimodal nature of grasp affordances, albeit at a higher computational cost. As shown in Table 4, diffusion-based models require significantly higher memory and inference time compared to lightweight architectures such as heatmap-guided or transformer-based methods, which may limit their deployment in real-time or resource-constrained systems.

TABLE 3: QUANTITATIVE COMPARISON OF PURELY GENERATIVE GRASPING METHODS

Method	Generative Model Type	Input Representation	Output Representation	Dataset Used	Grasp Success Rate / Accuracy (%)	Key Strengths	Key Limitations
DexDiffuser	Diffusion Model	Partial point cloud (Kinect v3)	16-DOF Allegro Hand grasp poses	DexGraspNet (1.7M grasps, 5,378 objects)	Sim: 98.77; Real: 68.89	High sampling diversity, end-to-end diffusion sampler + evaluator	Computationally heavy; sim-to-real gap
ZeroGrasp	Octree-based CVAE	Single RGB-D image	Joint 3D reconstruction + 6-DOF grasps	ZeroGrasp-11B (11.3B grasps, 12K objects) + GraspNet-1B	SOTA on GraspNet-1B	Zero-shot capability; handles transparent/specular surfaces	Compute-intensive; occlusion-sensitive
GraspGen	Diffusion-Transformer	Point cloud (object-centric)	6-DOF grasps for multiple grippers	GraspGen Dataset (53M grasps, 36K objects)	81.3 (real robot)	Multi-gripper compatibility; on-generator scoring filter	Long inference; gripper-specific
PSAGrasp	Transformer + Point Serialization	Serialized point cloud (Z-order)	6-DOF grasps with antipodal constraints	GraspNet-1Billion	Seen: 76.17; Similar: 70.59; Novel: 41.16	Attention-driven serialized points; antipodal grasp sampling	Serialization overhead; high transformer cost
Generalizing-Grasp	Domain Prior Knowledge + Diffusion	Multi-view TSDF	6-DOF grasps with constraints	GraspNet-1Billion	36.67 (novel)	Physically constrained diffusion; contact optimization	Multi-view requirement; slow
PhyGrasp	Multimodal Large Model (3D + Language)	Point cloud + natural language	Physics-informed grasp probability maps	PhyPartNet (195K part instances)	+10 over GraspNet baseline	Commonsense physical reasoning; language-conditioned	Dataset-limited; language dependency
Reasoning Grasping	Multimodal LLM + Vision	RGB + language	Instruction-conditioned grasp poses	Custom from GraspNet-1B	> baselines (CLIP/LLaVA) 120–180	Instruction-based grasp synthesis; multimodal reasoning	Early-stage; language-heavy

2.7 Transformer-Based Approaches in 3D Grasping

Transformer-based architectures have recently emerged as a powerful paradigm in 3D grasp detection due to their ability to model long-range dependencies and capture global context through attention mechanisms. Unlike convolutional or point-based networks that primarily focus on local geometric features, transformers enable the integration of both local surface structure and global object relationships, which is particularly important for grasping in cluttered and partially observed environments.

TABLE 4: COMPUTATIONAL EFFICIENCY COMPARISON

Model	Inference Time (ms)	Peak Memory Usage (MB)	Model Parameters (Millions)	Edge Deployment Feasibility
DexDiffuser	120–180	~4500	~85	Low (GPU recommended)
Grasp Diffusion Network	150–250	~3800	~72	Low (GPU recommended)
GraspGen	2500–3000	~6200	~120	Very Low (workstation-class GPU)
Dexterous Grasp Transformer, DGTR	50–80	~2100	~45	Moderate (edge GPU feasible)
Transformer-Grasp, TF-Grasp	30–60	~1800	~38	High
Contact-GraspNet	190–280	~1500	~28	High
HGGD, Efficient Heatmap-Guided 6-DoF	28–36 (~28 FPS)	~900	~15	Very High

TABLE 5: QUANTITATIVE COMPARISON OF HYBRID GENERATIVE GRASPING METHODS

Method	Generative Model Type	Input Representation	Output Representation	Dataset Used	Grasp Success Rate / Accuracy (%)	Inference Time (ms)	Key Strengths	Key Limitations
Contact-GraspNet	Direct Regression (4-DOF reduced)	Depth / Point cloud	6-DOF grasps via contact points	ACRONYM (17M grasps, 8,872 objects)	> 90 (cluttered scenes)	190–280	Collision-aware; contact-driven; end-to-end	Limited to parallel-jaw; occlusion challenges
GraspNet-1Billion	Analytical + CNN prediction	Point cloud (RGB-D)	6-DOF grasps with approach vectors	GraspNet-1Billion (>1B grasps, 88 objects)	Benchmark baseline	Not specified	Large-scale, unified evaluation benchmark	Not generative; limited object diversity
FlexLoG	Region-based Local Grasp Model	RGB-D + local/global cues	Regional 6-DOF grasps	GraspNet-1Billion (regional subset)	95 (real robot)	~38 (26 FPS)	Real-time; grasp-centric; flexible guidance	Requires explicit guidance input
GSNet	Graspness-based Generation	Point cloud	Graspness score + 6-DOF poses	GraspNet-1Billion	Seen: 67.12; Similar: 54.81; Novel: 24.31	Real-time	Simple and efficient graspness concept	Weak performance on novel objects
HGGD	Heatmap-Guided CNN	RGB-D + heatmap	6-DOF grasps	GraspNet-1Billion + TS-ACRONYM	Seen: 59.36; Similar: 51.20; Novel: 22.17	~36 (28 FPS)	Fast; local-global semantic fusion	Heatmap dependency; lower novel accuracy

In the context of robotic grasping, transformer-based models have been employed to learn spatial correlations across different regions of an object, allowing more informed grasp predictions. For instance, approaches such as the Dexterous Grasp Transformer described in [26] leverage attention mechanisms to model interactions between multiple contact points, enabling high-dimensional grasp representation for multi-fingered robotic hands. Similarly, transformer-based grasp detection frameworks discussed in [27] utilize encoder-decoder structures to directly predict grasp poses from point cloud inputs, improving performance in complex scenes.

More advanced methods such as [22] extend this idea by incorporating spatiotemporal reasoning, where sequential partial observations are fused using attention mechanisms to improve grasp prediction under occlusion. In addition, broader architectural insights from [21] highlight how transformers can effectively bridge local geometric features with global semantic understanding, making them well-suited for grasp detection tasks that require contextual reasoning.

As illustrated in Fig. 7, transformer-based frameworks are often integrated with generative or flow-based models to enhance grasp synthesis. In such hybrid formulations, transformer modules are used to fuse scene-level context with latent grasp representations, improving the quality and feasibility of generated grasp poses. This integration enables the model to reason not only about local contact geometry but also about global object structure and spatial constraints.

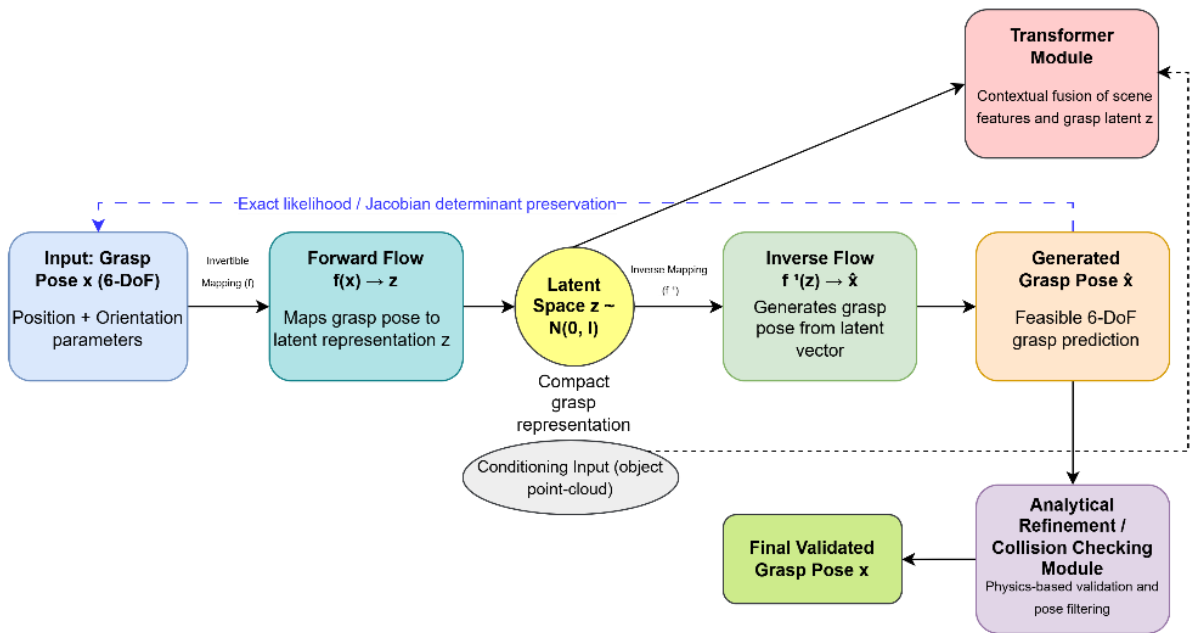


Fig.7. Flow-Based / Hybrid Generative Grasping Framework

Despite their advantages, transformer-based approaches introduce additional computational complexity and require large-scale training data to achieve optimal performance. Furthermore, their effectiveness depends on the quality of input representations, particularly when operating on sparse or incomplete point clouds. Nevertheless, their ability to model contextual relationships makes them a key component in modern grasp detection pipelines, especially when combined with generative frameworks.

2.8 Grasp Representation Through Graph Neural Networks

Graph-based representations provide an alternative formulation for modeling point cloud data by explicitly encoding relationships between points using nodes and edges. Unlike point-wise or voxel-based methods, graph neural networks (GNNs) enable structured reasoning over local geometric neighborhoods, making them particularly suitable for capturing contact relationships and surface interactions required for stable grasping.

Approaches such as [28] leverage graph convolutional operations to model grasp affordances by learning interactions between neighboring points. This allows the network to incorporate both geometric structure and relational context, leading to more informed grasp predictions. Such representations are especially advantageous when dealing with objects that exhibit complex topology, articulated structures, or non-uniform surface geometry, where local connectivity plays a critical role in determining feasible grasp regions.

However, graph-based methods often introduce additional computational overhead due to graph construction and message passing operations. Furthermore, their performance is sensitive to the quality of neighborhood definition and edge construction, particularly in sparse or noisy point clouds. Despite these challenges, GNN-based approaches provide a valuable framework for capturing fine-grained geometric relationships that are difficult to model using purely point-based or convolutional architectures.

2.9 Language-Guided Grasping

The integration of natural language understanding with robotic grasping has gained increasing attention with the emergence of multimodal learning frameworks. By combining visual perception with linguistic input, these approaches enable robots to interpret user intent and perform task-specific grasping, moving beyond purely geometry-driven predictions.

Methods such as [29] demonstrate how textual instructions can be incorporated alongside RGB-D or point cloud inputs to guide grasp selection. Similarly, the approach

presented in [30] combines vision and language features to identify target objects and refine grasp candidates based on semantic context. This allows the system to distinguish between multiple objects in a scene and select grasps aligned with user intent.

These developments reflect a broader shift toward contextual and semantic reasoning in robotic manipulation. Rather than treating grasping as an isolated geometric problem, language-guided approaches enable interaction-aware grasping, which is particularly relevant in human-robot collaboration scenarios. However, such methods remain dependent on large-scale multimodal datasets and robust language grounding, and their performance can degrade when visual and linguistic cues are misaligned.

2.10 Multimodal Grasping: Visual, Tactile, and Force-Torque Integration

Reliable robotic manipulation extends beyond visual perception, particularly in scenarios involving uncertainty, occlusion, or complex contact interactions. While vision provides global scene understanding, tactile sensing and force-torque feedback offer direct information about contact dynamics, enabling robots to detect slippage, estimate grasp stability, and refine grasp execution in real time.

Studies such as [31] and [32] emphasize the importance of integrating visual and tactile modalities to achieve robust manipulation. These works demonstrate that multimodal fusion significantly improves grasp reliability, especially in cases where visual data is incomplete or ambiguous. Furthermore, transformer-based tactile learning approaches, as presented in [33], show that attention mechanisms can effectively model temporal and spatial dependencies in tactile signals, leading to improved stability during dynamic grasping tasks.

The combination of visual, tactile, and force-torque sensing enables a more comprehensive representation of the interaction between the gripper and the object. This multimodal integration is particularly beneficial for handling deformable objects, transparent surfaces, and occluded geometries, where reliance on vision alone is insufficient. However, challenges remain in sensor calibration, synchronization, and real-time data fusion, which must be addressed for practical deployment in robotic systems.

2.11 Datasets for Grasp Pose Learning

The development of large-scale datasets has played a pivotal role in advancing learning-based grasp pose detection. The availability of diverse and richly annotated

datasets enables deep models to generalize across object categories, environmental conditions, and sensor configurations.

Datasets such as Dex-Net, ACRONYM, and GraspNet-1Billion have become foundational benchmarks in grasping research. Among these, [14] provides one of the most comprehensive resources, containing over one billion annotated 6-DoF grasp poses across multiple scenes and viewpoints. This scale allows for robust evaluation and comparison of grasp detection models under realistic conditions, including clutter and occlusion.

Synthetic datasets, including ACRONYM and DexGraspNet, expand dataset size through simulation-based generation, enabling controlled variation in object geometry, lighting, and sensor noise. More recent datasets such as [12] introduce human grasp annotations from single-view observations, bridging the gap between robotic and human grasping strategies.

A comparative overview of synthetic datasets is presented in Table 6, which highlights differences in scale, modality, and simulation environments. These datasets offer significant advantages in terms of data availability and diversity, but may suffer from domain gaps when transferred to real-world scenarios. In contrast, Table 7 summarizes real-world datasets, which provide physically grounded data but are often limited in scale and diversity due to the challenges of manual data collection and annotation.

As illustrated in Fig. 8, the dataset generation and evaluation pipeline typically involves synthetic data generation, model training, validation through ablation studies, and eventual deployment on real robotic systems. This workflow reflects the standard paradigm adopted in modern grasp detection research, where models are first trained on large-scale synthetic datasets and subsequently evaluated or fine-tuned using real-world data to ensure robustness and practical applicability.

TABLE 6: SYNTHETIC DATASETS COMPARISON

Dataset	Modality	#Objects	#Scenes/#Grasps/#Images	Supports Occlusion?	Sensor Types / Virtual Setup
GraspNet-1Billion	RGB-D, Point Cloud	88	97,280 images/1,000,000,000 grasps	Yes	Simulated Azure Kinect in Isaac Gym
ACRONYM	Point Cloud	8872	17.7 million grasps / 5,000 scenes	Yes	NVIDIA FleX simulation
DexGraspNet	Point Cloud	5355	1.32 million grasps / 8,270 scenes	Yes	Simulated Kinect v3 in NVIDIA Omniverse Robotics
GraspGen	Point Cloud	36000	53 million grasps	Yes	PyBullet with randomized lighting and textures
Dex-Net 2.0	Depth, Point Cloud	1000+	6.8 million grasps	Yes	Simulated Depth Cameras with analytic friction and mass parameters
TS-ACRONYM	RGB-D	88	97,280 images	Yes	Azure Kinect with synthetic noise injection
YCB-GraspNet	RGB-D	77	21,000 grasps	Yes	RealSense SR300 and Kinect v2, manual annotation
ShapeNetGrasp	Point Cloud	55000	2 million grasps	No	Mesh renderer with random camera views

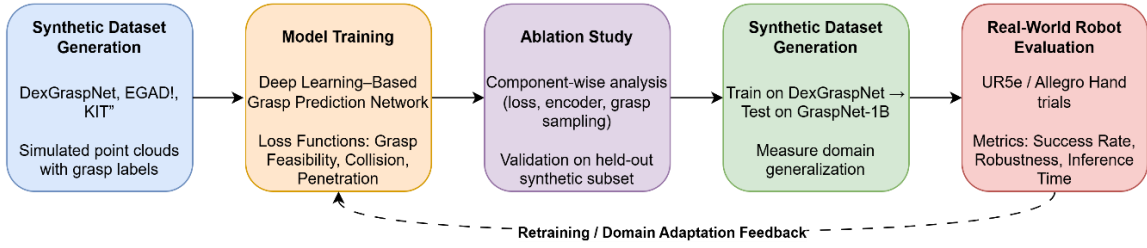


Fig.8. Dataset and Evaluation Flowchart for Grasp Pose Detection

Table 7: Real-world Datasets Comparison

Dataset	Year	Modality (RGB / Depth / Point Cloud)	#Objects	#Scenes / #Images / #Grasps	Supports Occlusion?	Sensor Types / Capture Setup
Cornell Grasping Dataset	2011	RGB-D	240	1,035 images; 8,019 grasps	No (single-object scenes)	One Kinect camera; tabletop single-object captures
Pinto & Gupta 50K	2016	RGB-D	150	50,000 images; 50,000 grasps	No (single-object trials)	Single camera; robot trials for grasp attempts across multiple objects
YCB-Video (PoseCNN)	2017	RGB-D	21	92 scenes; ~134,000 frames	Yes (multi-object occlusion and clutter)	Single RGB-D camera; video sequences capturing multiple objects per scene
Levine et al. 800K (Hand-Eye Coordination)	2018	RGB-D	-	800,000 images; 800,000 grasps	No (mostly single-object in bin picking; minimal occlusion)	Single camera per robot; large-scale multi-robot data collection
VMRD (Visual Manipulation Relationship Dataset)	2018	RGB-D	— (multi-object scenes)	4,700 images; ~3 objects per scene	Yes (stacking/occlusion relations)	RGB camera; tabletop stacking scenes capturing manipulation relationships
Multimodal Grasp Dataset (RGB-D-T)	2019	RGB-D-T	single-object setting	2,550 samples	No (single-object scenes)	Controlled setup with RGB-D + thermal/tactile sensing
GraspNet-1Billion	2020	RGB-D, Point Cloud	88	190 scenes; 97,280 images; >1B grasps	Yes (multi-object clutter)	Multi-view capture (Intel RealSense D435 + Azure Kinect) on robot arm; 256 viewpoints/scene
SuctionNet-1Billion (suction grasping)	2021	RGB-D, Point Cloud labels	— (shared object set with GraspNet scenes)	97,000 images; 10 scenes per object; 3-8M grasps per scene	Yes (multi-object clutter)	Robot-mounted multi-view RGB-D capture; large-scale suction grasp annotations

2.12 Sim-to-Real Transfer Challenges

Simulation-to-real (sim-to-real) transfer remains a persistent challenge in robotic grasping systems. Models trained in simulation often experience performance degradation when deployed in real-world environments due to multiple sources of domain discrepancy. These include depth sensor noise and missing regions, variations in lighting and surface reflectivity, inaccuracies in contact and friction modeling within simulators, biases in synthetic datasets, and differences in point cloud density and distribution.

Surveys such as [8] and [9] highlight several strategies to mitigate these challenges, including domain randomization, feature-level domain adaptation, adversarial learning, and sensor noise modeling. These techniques aim to improve the generalization of learned models by exposing them to diverse simulated conditions that approximate real-world variability.

Despite these advancements, achieving consistent real-world performance remains difficult, particularly for grasping tasks involving occlusion, deformable objects, and complex contact interactions. As a result, sim-to-real transfer continues to be an active area of research and a critical consideration in the deployment of learning-based grasping systems.

The literature reviewed reflects a clear progression from classical analytical grasping methods to modern data-driven approaches capable of operating in unstructured and dynamic environments. Early geometric techniques, while foundational, were limited by their reliance on complete and noise-free object representations. The emergence of deep learning enabled direct grasp prediction from visual and point cloud data, leading to the development of robust 6-DoF grasp detection frameworks.

Recent advances in generative modeling, particularly diffusion-based approaches such as [5], have further enhanced the ability to model uncertainty, handle partial observations, and generate diverse grasp hypotheses. In parallel, transformer-based architectures, graph representations, and multimodal perception have expanded the capability of grasping systems to incorporate contextual, structural, and semantic information. Additionally, the availability of large-scale datasets and the development of sim-to-real transfer techniques have played a crucial role in bridging the gap between simulation and real-world deployment.

Together, these developments establish the foundation for the approach adopted in this work, which focuses on generative grasp pose detection from point cloud data and its integration into a complete robotic manipulation pipeline.

2.13 Literature Gap

Despite significant advances, several persistent gaps motivate this work:

2.13.1 Limitations in Handling Partial and Occluded Point Clouds

A major challenge in grasp pose detection arises from the reliance on incomplete point cloud observations. In practical scenarios, depth sensors often produce partial data due to occlusion, self-shadowing, reflective surfaces, and limited viewpoints. These issues

introduce ambiguity in object geometry, making it difficult to infer stable grasp configurations.

Many discriminative grasp detection frameworks, particularly those based on architectures such as [34] and related extensions, exhibit degraded performance when significant portions of the object are not visible. As discussed in [3] and [13], such models struggle to generalize under incomplete observations, often generating unstable or infeasible grasp poses.

Point cloud completion methods, including those presented in [18], [19], attempt to address this limitation by reconstructing missing geometry. However, these approaches may introduce hallucinated structures that do not accurately reflect the true object shape. Consequently, grasp poses generated on reconstructed point clouds may fail during execution due to discrepancies between predicted and actual surfaces.

2.13.2 Limited Multi-Modal Integration for Reliable Grasp Execution

Most existing grasp detection systems rely primarily on visual inputs such as RGB-D images or point clouds. While these modalities provide geometric and spatial information, they are insufficient for capturing contact dynamics during grasp execution.

Studies such as [31], [32] demonstrate that integrating tactile sensing and force–torque feedback significantly improves grasp stability and robustness. These modalities enable detection of slippage, estimation of contact forces, and adaptation during manipulation.

However, the integration of such multimodal feedback into learning-based and generative grasping frameworks remains limited. Most current models are trained on static datasets without incorporating real-time feedback during grasp execution. This restricts their ability to adapt to variations in object properties, such as deformability, friction, and surface texture, thereby limiting practical deployment.

2.13.3 Inadequate Modeling of Grasp Multimodality

Grasping is inherently a multimodal problem, as multiple valid grasp configurations may exist for a single object. However, many early and even recent grasp detection models produce a single deterministic output, failing to capture this diversity.

Discriminative models, which directly map input observations to a single grasp pose, overlook alternative feasible grasps that may be more stable or collision-free. Generative models such as VAEs and GANs attempt to address this limitation. However, VAE-based

approaches often produce over-smoothed predictions [6], while GAN-based models suffer from instability and mode collapse [25].

Diffusion-based methods, including [7] [5], have shown promising results in modeling grasp distributions. Nevertheless, challenges remain in fully capturing complex multimodal grasp spaces, particularly in cluttered scenes and for objects with intricate geometries.

2.13.4 Constraints in Generalizing to Unseen Objects and Environments

Generalization to unseen objects and environments remains a significant limitation in current grasp detection systems. Although large-scale datasets such as [14] provide diverse training data, models often exhibit reduced performance when encountering novel object categories or out-of-distribution scenarios.

Additionally, models trained on synthetic datasets frequently struggle when applied to real-world data due to discrepancies in sensor characteristics, noise patterns, and environmental conditions. As discussed in [35], [36], these domain gaps significantly affect model robustness.

While techniques such as domain randomization and domain adaptation have been proposed, as explored in [9], achieving consistent generalization across diverse real-world conditions remains an open challenge.

2.13.5 Insufficient Exploitation of Global Context in 6-DoF Grasping

Many grasps detection approaches primarily focus on local geometric features while neglecting global spatial context. However, grasp feasibility often depends on the broader structure of the object and its surrounding environment.

Recent methods, including transformer-based approaches in [26] and attention-based frameworks in [27], attempt to address this limitation by modeling long-range dependencies. Similarly, graph-based methods such as [28] capture relationships between different regions of the object.

Despite these advancements, effective integration of global context into real-time grasp detection pipelines remains limited, particularly in cluttered scenes where interactions between multiple objects influence grasp feasibility.

2.13.6 Computational Inefficiencies in Sampling-Based Grasp Pipelines

Sampling-based grasp detection methods, including classical approaches such as Grasp Pose Detection in Point Clouds [2], rely on generating and evaluating a large number of candidate grasp poses. While these methods are general and flexible, they are computationally expensive and unsuitable for real-time robotic applications.

Even more recent approaches that improve efficiency, such as attention-based methods [20], still face challenges in balancing accuracy and inference speed. This highlights the need for efficient, end-to-end architectures capable of directly predicting grasp poses without exhaustive sampling, while maintaining robustness under noisy and incomplete observations.

2.13.7 Underdeveloped Reasoning for Object Semantics and Task Context

Most grasp detection systems focus primarily on geometric feasibility, without considering semantic information such as object category, intended task, or user instructions. However, human grasping behaviour is inherently guided by such contextual factors. Recent works, including Vision Language Guidance and Negative Prompt Guidance [29], [30], demonstrate the potential of integrating language and vision for task-oriented grasping. Additionally, affordance-based approaches, as explored in weakly supervised affordance grounding [37], aim to incorporate semantic understanding into grasp selection. Despite these advances, semantic reasoning remains underutilized in mainstream grasp detection pipelines, limiting their applicability in human-robot interaction and task-driven manipulation scenarios.

2.13.8 Incomplete Addressing of Sim-to-Real Gap for Real-World Deployment

The discrepancy between simulated training environments and real-world deployment remains a major challenge in robotic grasping. Differences in sensor noise, lighting conditions, material properties, and physical interactions lead to significant performance degradation when models are transferred from simulation to real robots. Key contributing factors include variations in depth sensing, inaccuracies in physics simulation, and distribution shifts in point cloud data. Studies highlight the importance of domain randomization, noise modeling, and adaptive learning techniques for bridging this gap [8], [38]. Despite ongoing research, achieving reliable sim-to-real transfer remains an open problem, particularly for grasping tasks involving complex objects and dynamic environments.

2.13.9 Limited Consideration of Execution-Phase Dynamics

Most grasp detection methods focus on predicting static grasp poses without considering the dynamic aspects of grasp execution. However, successful manipulation depends not only on grasp selection but also on motion planning, collision avoidance, and feedback during execution.

Important factors such as approach trajectories, in-hand adjustments, slippage detection, and re-grasping strategies are often not incorporated into existing grasping frameworks. While some reinforcement learning and multimodal approaches address dynamic manipulation, they are not widely integrated into standard 6-DoF grasp detection pipelines [39].

The research gaps identified highlight the need for a grasp detection framework that is robust to partial and noisy point cloud observations, capable of modeling multimodal grasp distributions, and adaptable to real-world execution constraints. In particular, the limitations of deterministic grasp prediction, insufficient robustness under occlusion, and the persistent sim-to-real gap motivate the adoption of generative approaches that can capture uncertainty and produce diverse grasp candidates. These considerations directly inform the methodology of the present work, which focuses on diffusion-based grasp generation from point cloud data and its integration into a complete robotic manipulation pipeline.

CHAPTER 3

PROBLEM FORMULATION & METHODOLOGY

3.1 Problem Definition

Robotic grasping remains a fundamental challenge in autonomous manipulation, particularly in unstructured environments where objects are partially visible, occluded, affected by sensor noise, or positioned in orientations that restrict accessible grasp regions. Traditional approaches relying on complete geometric information fail under such conditions, while modern learning-based methods still struggle with partial or noisy point clouds leading to inaccurate pose estimation and execution failures in cluttered scenes.

The challenge is compounded by the sim-to-real gap: grasp poses feasible in simulation may fail on physical systems due to sensor inaccuracies, calibration errors, and discrepancies in physical interaction modeling. Additionally, most existing frameworks predict a single grasp configuration, whereas real-world manipulation often permits multiple valid poses depending on object orientation and task constraints limiting flexibility and increasing failure rates.

The problem can be formulated along two dimensions:

1. **Generation of robust grasp poses:** Generating accurate and diverse 6-DoF grasp configurations from partial, noisy point cloud data across varying object geometries and sensing conditions.

2. **Reliable real-world execution:** Translating predicted grasp poses into executable robot actions accounting for kinematic constraints, collision avoidance, and real-world uncertainties.

This research addresses both through an integrated framework using a UR10 arm with a Robotiq 3F gripper, designed to generate feasible grasps from point cloud data and execute them reliably in simulation and real-world environments. The core problem is: *how to design a deep learning-based grasp pose detection framework that generates stable, diverse 6-DoF configurations from incomplete point cloud data and executes them reliably on a physical robot.*

3.2 Scope and Objectives

This project develops a complete grasping and manipulation pipeline combining generative deep learning with point cloud processing and robotic execution — integrating perception, grasp generation, motion planning, and hardware execution into a unified framework, validated on a UR10 with a Robotiq 3F gripper.

3.2.1 Scope of the Study

- 1) **Point Cloud Acquisition and Processing:** 3D point clouds from the OAK-D Pro Wide are processed to remove noise, segment objects, and extract geometric features relevant to grasp synthesis.
- 2) **Deep Learning–Based Grasp Prediction:** A generative model predicts multiple feasible 6-DoF grasp poses from partial and occluded inputs, modeling grasp uncertainty and incomplete observations.
- 3) **Robotic Manipulation Pipeline:** Grasp prediction is integrated with ROS2 and MoveIt2 for collision-aware motion planning and execution on the UR10 with Robotiq 3F gripper.
- 4) **Simulation-Based Development:** RViz2 with MoveIt2 is used for iterative testing of perception, grasping, and motion planning prior to real-hardware deployment.
- 5) **Real-World Deployment and Evaluation:** The system is evaluated under real-world conditions including sensor noise, lighting variation, and environmental uncertainty, analysing grasp success, stability, and repeatability.
- 6) **Object Variability and Practical Constraints:** Evaluation spans objects of varying shape, size, material, and surface properties under occlusion, orientation, and workspace constraints.

3.2.2 Objectives of the Present Research

- To develop a generative diffusion-based model for predicting 6-DoF grasp poses, utilizing a multi-pose point cloud fusion strategy to overcome partial observability
- To investigate the effectiveness of diffusion-based grasp generation under occlusion, sparse geometry, and real-world uncertainty.
- To design and implement a perception pipeline for point cloud noise filtering, segmentation, and object-level representation suitable for grasp synthesis.
- To integrate generative grasp prediction with a manipulation pipeline ensuring kinematic feasibility, collision avoidance, and execution reliability.
- To evaluate system performance in simulation and real-world environments, analysing grasp success, robustness, and generalisation across diverse objects.
- To analyse sim-to-real discrepancies and assess robustness under real sensor noise and environmental variation.
- To validate an end-to-end framework bridging perception, grasp generation, and physical execution on a UR10 robotic platform, achieving measurable real-world grasp success under constrained training and hardware conditions.

3.3 Methodology

3.3.1 Overview of the Proposed Framework

The methodology adopted in this work centres on adapting and extending the GraspGen diffusion-based grasp generation framework [5] to a custom hardware platform and deploying it within a complete robotic manipulation pipeline. Rather than evaluating the framework in isolation as a grasp pose predictor, the present work integrates perception, instance segmentation, grasp generation, motion planning, and physical execution into a unified end-to-end system. This integration is validated both in simulation using MoveIt2 and RViz, and on a physical UR10 robotic arm operating under real-world sensing conditions.

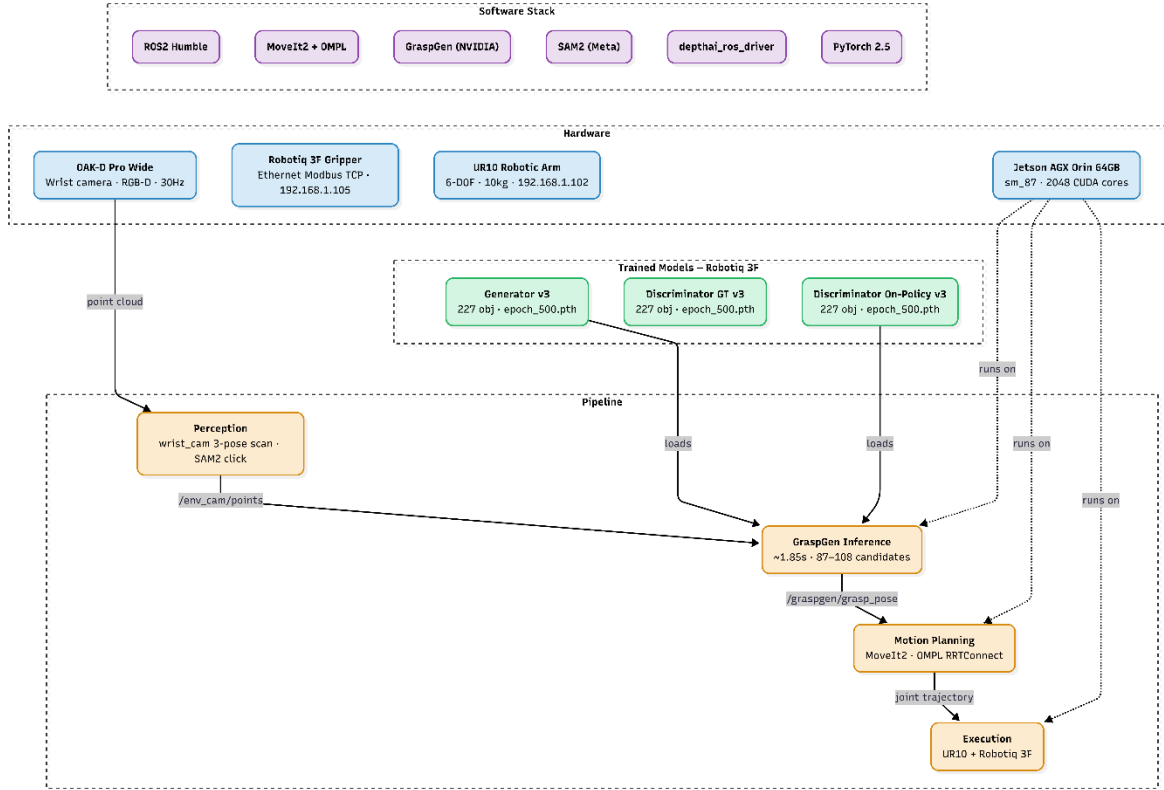


Fig.9. System Architecture

The overall pipeline is structured around a single architectural principle: a common inference and execution stack that operates identically regardless of whether point cloud data originates from a simulated synthetic publisher or from the OAK-D Pro Wide wrist camera in real-hardware mode. This design choice ensures that the transition from simulation to real hardware requires no modification to the grasp generation or execution nodes, effectively isolating the sim-to-real challenge to the perception front-end alone. As illustrated in Fig.9, the architecture is organised into two source branches simulation and real that converge at the point cloud topic `/env_cam/points`, after which the pipeline is entirely unified.

The framework comprises six primary functional stages. In the first stage, object point clouds are acquired either from a synthetic publisher in simulation or, in the real-world setting, through the OAK-D Pro Wide wrist camera via a three-pose scanning sequence combined with SAM2-based instance segmentation. At scan pose 1, the operator selects the target object by clicking once on it in the live RGB stream displayed by the SAM2 segmentation node; the resulting mask is used to isolate the corresponding depth pixels, which are back-projected to 3D using the camera intrinsics and transformed into the world frame via ROS2 TF. This click-based approach draws upon the interactive segmentation

capability of SAM2 [40], enabling the system to isolate arbitrary objects without requiring object-specific training or category labels. The segmented object point cloud is then published on the same topic consumed by the GraspGen inference node, making the real-world and simulation paths fully interchangeable from the perspective of all downstream components.

In the second stage, the GraspGen generator a Denoising Diffusion Probabilistic Model (DDPM) operating in SE(3) [41], [42] receives the object-centric point cloud and generates a batch of candidate 6-DoF grasp poses through iterative denoising. In the third stage, a separately trained discriminator scores and ranks each candidate, returning only the highest-quality grasps above a configurable confidence threshold. The fourth stage involves collision-aware motion planning using MoveIt2 and the OMPL planning library, where the top-ranked grasp poses serve as Cartesian goal targets for the UR10 arm. In the fifth stage, the planned trajectory is executed on the UR10 arm and Robotiq 3F gripper through the ROS2 control stack. The final stage involves conveying the grasped object to a designated placement location, completing the pick-and-place task.

A key contribution of this implementation is the adaptation of GraspGen to the Robotiq 3F adaptive gripper a three-fingered underactuated gripper that presents a fundamentally different morphology from the parallel-jaw configurations for which GraspGen was originally designed and evaluated. As noted in GraspGen [5], contact point representations such as those used in M2T2 [43] are designed for symmetric parallel-jaw grippers and do not transfer directly to adaptive multi-fingered designs. Adapting GraspGen to this gripper required the generation of a new dataset specific to the Robotiq 3F, modifications to the on-policy discriminator training to operate without the FCL collision library which is unavailable on the ARM64 Jetson platform and appropriate tuning of the inference pipeline for edge deployment. These adaptations are described in detail in the sections that follow.

3.3.2 Hardware Setup

The experimental platform consists of an industrial robotic arm, a three-fingered adaptive gripper, dual RGB-D depth cameras, and an edge computing unit serving as the unified computational backbone for perception, inference, planning, and control. Each component was selected to satisfy the requirements of real-time grasp inference and physical manipulation while remaining deployable on a resource-constrained edge platform. The

complete real hardware setup showing the UR10 arm with Robotiq 3F gripper and OAK-D Pro Wide wrist camera mounted on the adapter slab, cardboard pick tray with objects, yellow place crate, and teach pendant is shown in the Fig. 10



Fig.10. Hardware setup with pick and place tray

3.3.2.1 Robotic Arm

The manipulation platform is a Universal Robots UR10 as shown in Fig. 11 which is a 6 DOF collaborative robot arm with a maximum payload of 10 kg and a reach of 1300 mm. The UR10 was chosen for its suitability to tabletop pick-and-place tasks and its compatibility with the ROS2 `ur_robot_driver` package, which exposes joint-level trajectory control through the standard ROS2 control interface. Communication between the Jetson AGX Orin and the UR10 control box is established over Ethernet, with the Jetson assigned to the fixed address 192.168.1.10 and the robot controller at 192.168.1.102. The arm operates in ExternalControl mode via the URCap installed on the teach pendant shown in Fig. 12, which forwards trajectory commands received from MoveIt2 over the ROS2

action interface as shown in Fig. 13. All grasping experiments are conducted within the arm's reachable workspace over a fixed tray surface.



Fig.12. UR10 Arm

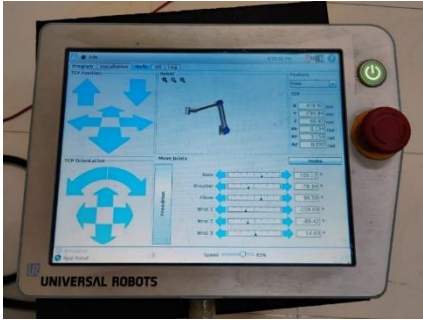


Fig.11. Teach Pendant

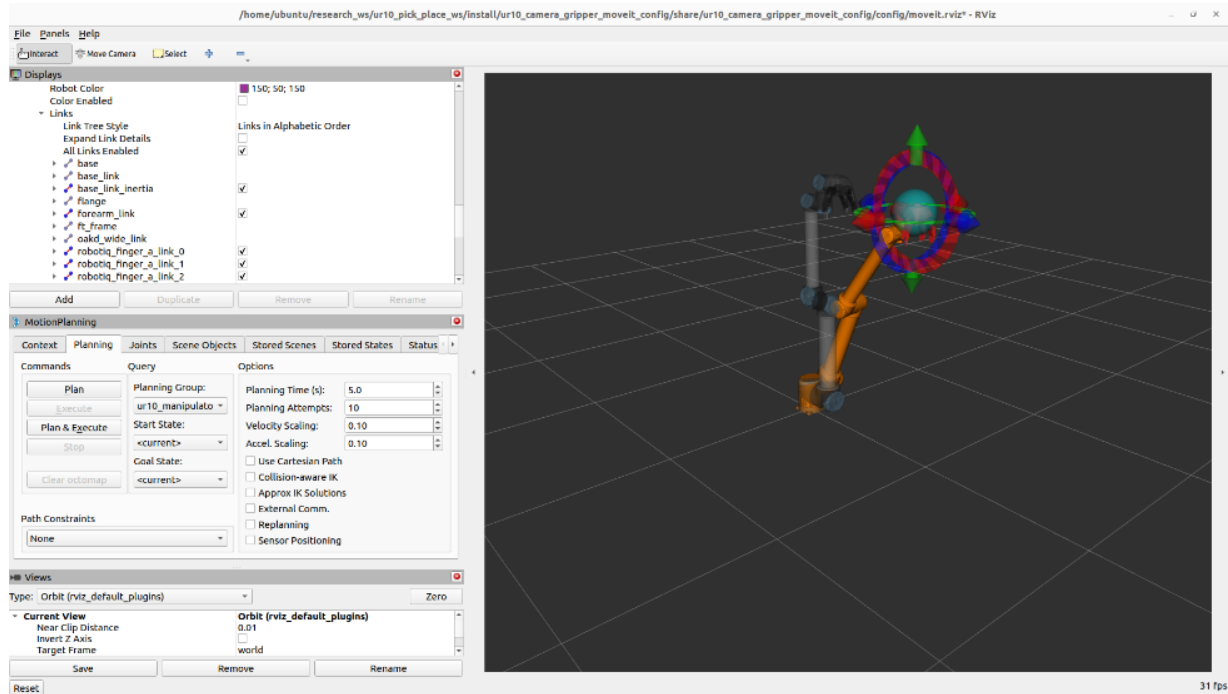


Fig.13. UR10 with Robotiq Gripper mounded on it in Rviz

3.3.2.2 Gripper

The Robotiq 3F is a three-fingered as shown in Fig. 14 underactuated adaptive gripper attached to the UR10 tool flange via a custom 3D-printed adapter slab. The gripper supports multiple configurable grasping modes - basic, pinch, wide, and scissor, and communicates with the Jetson AGX Orin via Ethernet Modbus TCP at IP address 192.168.1.105, port 502, over the same laboratory Ethernet switch that connects the UR10 control box. Gripper control is implemented as a pure-Python Modbus TCP client (`robotiq_3f_gripper.py`) that activates the gripper on node startup and sends open and close commands during pick execution, without requiring a separate ROS2 gripper driver package. In simulation, the `gripper_trajectory_controller` defined in the MoveIt2 configuration manages the three finger joints through the `ros2_control` mock hardware interface; this controller is not used in real-hardware mode.



Fig.14. Robotiq 3F Adaptive Gripper [44]

The Robotiq 3F introduces an important distinction from the parallel-jaw grippers used in the original GraspGen evaluations. Its three-finger geometry and underactuated compliance allow it to conform to irregular object shapes, offering grasping versatility that single-width parallel-jaw grippers cannot achieve. However, this also means that grasp pose representations based on a single gripper width or contact point pairs such as those used in Contact-GraspNet [4] and M2T2 [43] do not apply directly. Generating a gripper-specific dataset and training GraspGen from scratch for this embodiment was therefore a necessary component of the work, following the multi-gripper training paradigm established in the original framework. The 3D-printed adapter slab simultaneously mounts the OAK-D Pro Wide wrist camera alongside the gripper, serving as the sole perception sensor in the real hardware pipeline. The gripper and camera assembly is shown in Fig. 15.



Fig.15. Robotiq 3F gripper and OAK-D Pro Wide wrist camera mounted on the custom 3D-printed adapter slab on the UR10 tool flange

3.3.2.3 Perception System

Depth perception in the real hardware pipeline is provided by the **OAK-D Wide**, an RGB-D stereo camera mounted on the wrist slab alongside the Robotiq 3F gripper via the custom 3D-printed adapter slab. The camera is physically mounted upside-down on the bracket; this is compensated entirely in software through a 180° image rotation applied before display and a corresponding yaw offset of π in the static transform from `oakd_wide_link` to `wrist_cam_rgb_camera_optical_frame`, ensuring that depth pixels are back-projected to geometrically correct world-frame positions. The camera connects to the Jetson via a powered USB 3.0 hub and is driven by the `dephai_ros_driver` package, publishing synchronised RGB images, stereo depth images, and camera intrinsics at approximately 30 Hz over ROS2 topics.

The extrinsic transform between the wrist camera frame and the robot world frame is derived entirely from the robot's own joint encoder readings through the URDF kinematic chain via forward kinematics. This approach requires no manual calibration the robot knows the exact pose of the camera at every configuration through its internal joint state and is therefore robust to physical perturbation of the hardware setup between sessions.

For object-level point cloud isolation, the pipeline employs SAM2 [40] in an interactive click-based mode. The operator views the live RGB image stream from the wrist camera and selects the target object by clicking on it; SAM2 processes the click as a positive spatial prompt and generates a precise segmentation mask. The depth pixels corresponding

to the masked region are back-projected to 3D using the camera intrinsic matrix K , and the resulting point set is transformed into the world frame via the forward kinematics transform. This object-centric paradigm is consistent with the design philosophy advocated in GraspGen [5], where the maturation of foundation segmentation models such as SAM2 is identified as enabling the shift from scene-centric to object-centric grasp generation. A right-click adds a negative spatial prompt to refine the mask for partially occluded or cluttered objects, and pressing R resets the selection entirely.

3.3.2.4 Compute Platform

All perception, grasp inference, motion planning, and robot control computations are executed on the NVIDIA Jetson AGX Orin 64GB shown in Fig. 16, an edge AI platform featuring a 12-core ARM Cortex-A78AE CPU, an NVIDIA Ampere GPU (sm_87 architecture, 2048 CUDA cores), and 64 GB of unified LPDDR5 memory. The platform runs Ubuntu 22.04 with JetPack 6.1, Python 3.10, and PyTorch 2.5 compiled for CUDA 12.6 on the aarch64 architecture.

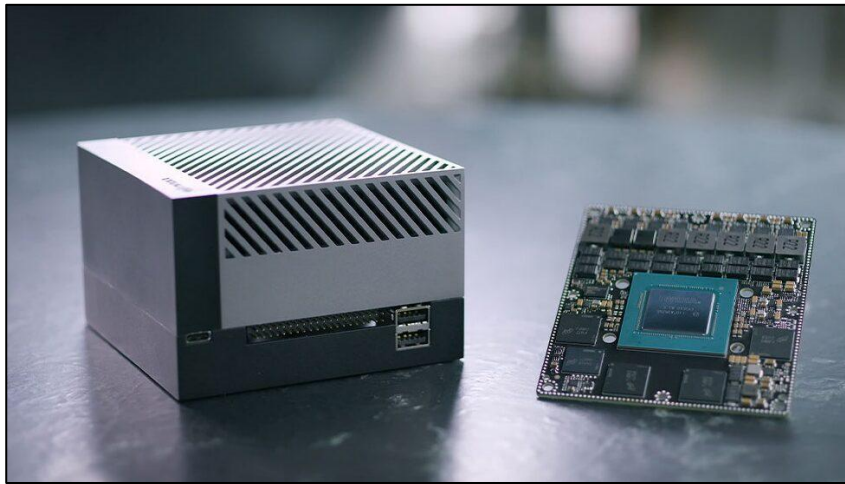


Fig.16. NVIDIA Jetson AGX Orin 64GB [45]

Deploying GraspGen on the Jetson required several platform-specific adaptations that are worth noting explicitly, as they distinguish this deployment from the original NVIDIA V100 cluster setup described in the GraspGen [5]. The pointnet2_ops C++ extension required by PointNet++ [46] for ball-query and neighbourhood grouping operations was manually recompiled for the aarch64 target architecture. A torchvision compatibility fix was applied by wrapping the `register_fake("torchvision::nms")` call in a try/except block within `_meta_registrations.py`, as the NMS operator is not present in the Jetson-specific PyTorch binary build. As detailed in Section 3.3.4.3, the system utilizes

an alternative FCL-free trimesh proximity method for grasp labelling.. The LD_LIBRARY_PATH was also configured to include the cuSPARSELt library path required by PyTorch on JetPack. Despite these hardware constraints, the Jetson platform sustains a mean forward pass time of approximately 130-150 ms per diffusion denoising step consistent with the inference latency reported for comparable batch sizes in the GraspGen evaluation confirming the feasibility of edge deployment for this class of diffusion-based grasp generation model.

3.3.3 Software Architecture and ROS2 Integration

The software stack is built on ROS2 Humble Hawksbill running on Ubuntu 22.04, and is organised into a collection of purpose-specific nodes that communicate through standardised ROS2 topics, services, and actions. The architecture is designed such that the grasp generation and execution nodes are agnostic to the source of the point cloud, allowing the same codebase to operate identically in simulation and on physical hardware. The complete node graph and communication topology are illustrated in Fig.10.

3.3.3.1 Node Architecture

The pipeline consists of four primary runtime nodes, each with a well-defined responsibility boundary.

The `sim_object_pc_publisher` node is active only during simulation. It generates a synthetic cylindrical point cloud at a fixed known position in the world frame and publishes it directly to `/env_cam/points`. This node exists purely to provide a deterministic point cloud input for development and validation of the GraspGen inference and execution nodes without requiring physical hardware or a live camera driver.

The `sam2_segmentation_node` replaces the synthetic publisher in the real-world pipeline. It subscribes simultaneously to `/wrist_cam/rgb/image_raw`, `/wrist_cam/stereo/image_raw`, and `/wrist_cam/rgb/camera_info`. When the operator clicks on the target object in the displayed RGB window, the click coordinates are forwarded as a positive spatial prompt to the SAM2 [40], running on the Jetson GPU, which generates a segmentation mask. The whole Full End-to-End Manipulation Pipeline with ROS2 Topic Names is shown in Fig. 17.

and the highest-scoring pose is published as a `geometry_msgs/PoseStamped` message on `/graspgen/grasp_pose`.

The `grasp_executor_node` receives the grasp pose from the bridge node and orchestrates the full pick-and-place execution sequence. It first computes an inverse kinematics solution using the `/compute_ik` service, seeded from the current joint configuration read from `/joint_states` to prevent degenerate elbow-flip solutions. Collision-free trajectory planning is performed through the MoveIt2 move group action, with OMPL as the underlying planning library. The executor implements an eight-step pick-and-place sequence: (1a) home to a ready pose via joint-space goal, (1b) move to a forward-look intermediate pose to establish a stable IK seed, (2) move to a pre-grasp position above the object at a fixed world-frame height, (3) descend to the grasp pose with collision checking relaxed for the gripper zone, (4) close the gripper, (5) lift the object, (6) transport to the place location with full collision avoidance restored, (7) open the gripper, and (8) return to home. In the real-hardware mode, an additional geometry estimation step is performed prior to planning: the executor derives the object's bounding cylinder from the incoming point cloud by computing the median XY centroid, the 5th-to-95th percentile height range, and the 90th-percentile radial distance from the centroid, and registers this cylinder as a collision object in the MoveIt2 planning scene.

3.3.3.2 Motion Planning Configuration

Motion planning is handled by MoveIt2 with the `ur10_manipulator` planning group defined in the SRDF. The `joint_trajectory_controller` manages arm trajectory execution and the `gripper_trajectory_controller` manages the three Robotiq finger joints, both operating under the `ros2_control` framework. In simulation, mock hardware components are used; in real-hardware mode, the live `ur_robot_driver` provides joint feedback. The planning scene includes a tabletop surface primitive and a floor plane to prevent OMPL from generating underground approach paths. Two tolerance levels are applied depending on the execution phase: a loose tolerance of 0.02 radians for hover and lift motions, and a tight tolerance of 0.003 radians for the grasp and place steps. The Allowed Collision Matrix is dynamically configured per execution step surrounding distractor objects are exempted during the descent phase to allow the arm to approach through constrained clearances, and full collision avoidance is reinstated for transport and placement.

3.3.3.3 Simulation Environment

The simulation environment uses RViz2 as the primary robot visualiser and motion planning interface, with a MoveIt2 planning scene populated with tabletop geometry and object collision primitives. All robot motion, joint state publishing, and trajectory planning are therefore handled entirely through RViz2 and MoveIt2 with mock hardware. The planning scene setup script publishes the scene repeatedly at launch with short intervals to ensure MoveIt2 reliably ingests the scene geometry before the first grasp trigger arrives.

3.3.4 Dataset Generation for the Robotiq 3F Gripper

Training GraspGen for a new gripper embodiment requires a gripper-specific annotated grasp dataset, since no existing public dataset provides grasps labelled for the Robotiq 3F configuration. The original GraspGen dataset [5] covers the Franka Panda parallel-jaw gripper, the Robotiq 2F-140, and a vacuum suction gripper none of which are directly applicable to the three-finger adaptive morphology of the Robotiq 3F. Accordingly, a custom dataset was generated using the **GraspDataGen** pipeline adapted to the constraints of the Jetson AGX Orin platform.

3.3.4.1 Object Corpus

The object corpus was sourced from a subset of the Objaverse [47], a large-scale repository of over 800,000 annotated 3D models spanning diverse object categories with descriptive captions and per-object metadata. The mesh selection criterion follows that of the GraspGen [5] objects whose categories overlap with the 1,156 semantic classes defined in LVIS [48] and which are licensed under Creative Commons Attribution (CC BY), ensuring compatibility with research use and redistribution. This selection strategy ensures broad object diversity across common manipulation-relevant categories while maintaining licence compliance.

The training corpus used in this work referred to as the dataset comprises 181 training objects and 46 validation objects, totalling 227 objects. Some objects were excluded from training after it was confirmed that the Robotiq 3F gripper geometry could not produce any feasible positive grasps for them within the simulation constraints used for labelling; these were added to a denylist and silently skipped during training. Object meshes were stored at `~/GraspDataGen/objects/` and processed through the GraspDataGen pipeline to produce the dataset in the webdataset shard format expected by the GraspGen training code.

3.3.4.2 Grasp Sampling

For each object, grasp transformations were uniformly sampled around the mesh surface in $SE(3)$, producing candidate grasp poses spanning the full range of approach directions and wrist orientations accessible by the Robotiq 3F. The number of grasps per object was fixed at 100 per training batch using replacement sampling to ensure consistent tensor shapes across the batched training loop. The gripper geometry used for all sampling and labelling operations was derived from the Robotiq 3F collision mesh at `~/robotiq_clean/meshes/collision/`.

3.3.4.3 Grasp Labelling: FCL-Free Proximity Method

Grasp label generation is the most critical step in dataset construction, as it determines which candidate grasp poses are classified as successful and which are not. In the standard GraspGen pipeline [5], grasp labelling is performed through physics simulation using the Isaac simulator introduced in Isaac Gym [49] as shown in Fig. 18. For each candidate grasp pose sampled around an object mesh, the simulated gripper is closed on the object and a shaking motion is applied to the held assembly. A grasp is classified as **positive** if a stable contact configuration persists after the shaking sequence concludes indicating that the grasp can resist external disturbances and as **negative** otherwise. This simulation-based labelling approach is physically grounded and captures contact stability in a way that purely geometric heuristics cannot, which is why it is used to generate the 53 million annotated grasps in the original GraspGen dataset. The labelling pipeline additionally relies on the FCL collision library for proximity and collision queries during the simulation setup phase, used to determine initial contact configurations and to filter out grasps that cause interpenetration before the shaking test begins.

Deploying this pipeline on the Jetson AGX Orin platform introduced two fundamental constraints that precluded its direct use. First, Isaac Gym requires an x86-64 CUDA environment and is not available for the ARM64 aarch64 architecture of the Jetson platform. Second, the FCL collision library presented functionality issues within the pipeline and could not be compiled in the available build environment, prompting its replacement with trimesh. Both constraints are platform-level incompatibilities rather than solvable configuration issues, and they necessitated the development of an alternative labelling method that operates within the capabilities of the Jetson platform.

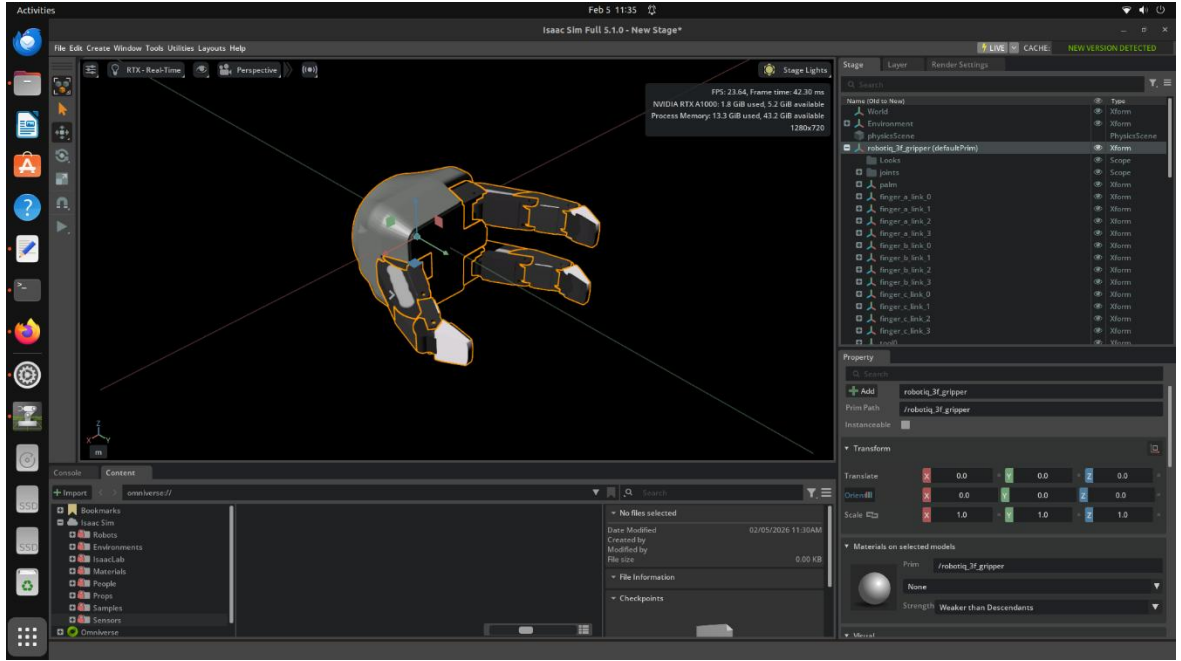


Fig.18. Grasp pose generation and labelling in Isaac Sim with Robotiq 3F gripper

To address this, a custom FCL-free labelling method as shown in Fig. 19 was implemented in `scripts/generate_onpolicy_data.py`, using `trimesh.proximity.closest_point` a pure-Python surface proximity function that operates without any native binary dependencies and is fully compatible with the ARM64 architecture. A candidate grasp is labelled **positive** if the Tool Centre Point (TCP) translation lies within a distance of `clip` ($\text{object_radius} \times 0.5$, 1 cm, 5 cm) from the nearest point on the object mesh surface, and if the pre-grasp TCP computed by backing off 5 cm along the grasp approach axis does not intersect the object bounding volume. All other grasps are labelled **negative**. This heuristic approximates the physical requirement of the Isaac shaking test by ensuring that the gripper is positioned in genuine contact proximity with the object from a clear approach direction, without simulating the dynamic stability test itself. The trade-off inherent in this approximation specifically the absence of contact force and friction modelling is acknowledged as a source of label noise and is examined in the context of real-world grasp success rates in Chapter 4.

The dataset produced through this pipeline contains **9,685 positive grasp annotations** across 227 objects, totalling approximately 18 GB, stored in the H5 cache partitioned into positive and negative grasp sets.

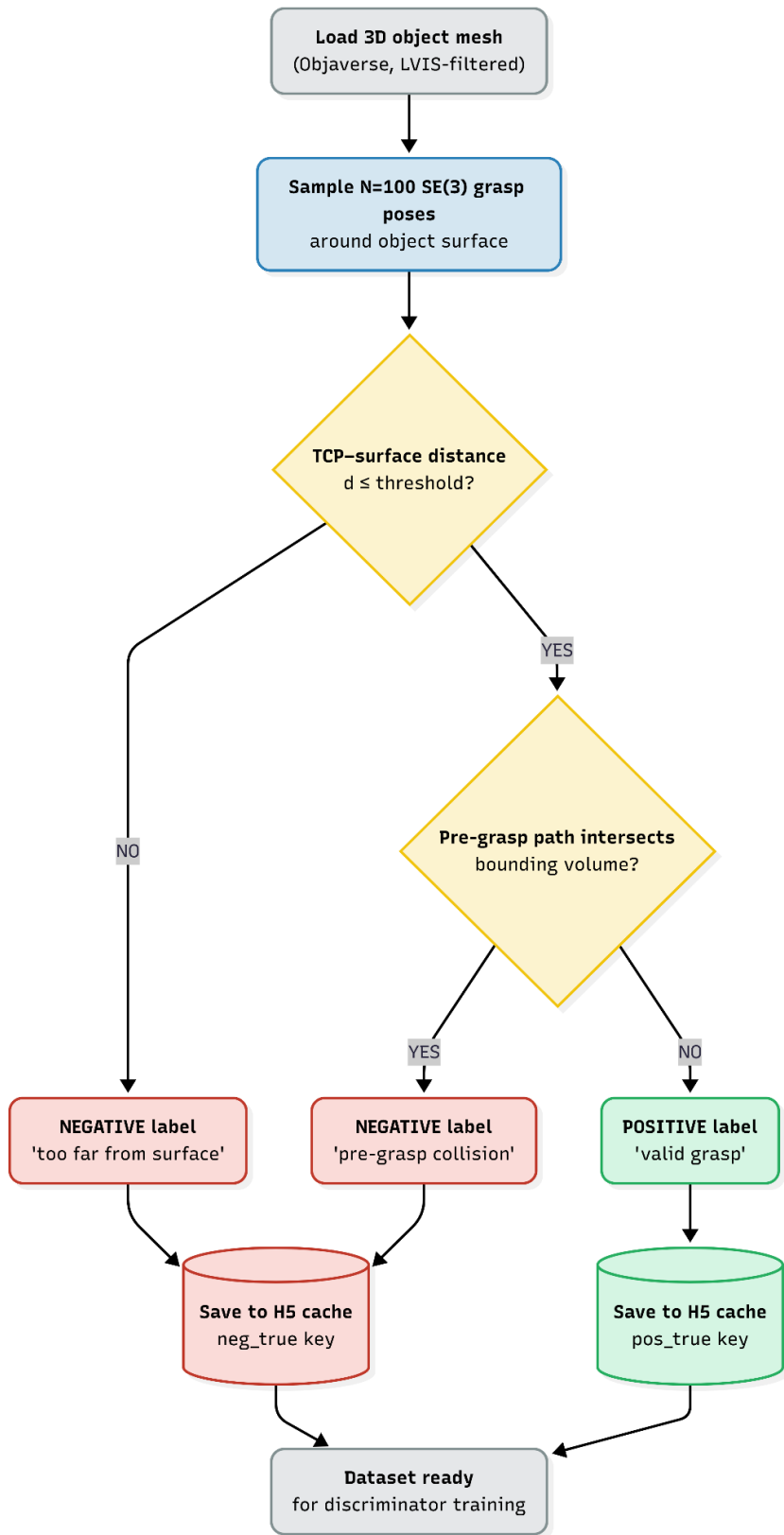


Fig.19. Grasp sampling and FCL-free labelling flowchart

3.3.4.4 Training Configuration

Training is managed through the Hydra configuration framework used by GraspGen, with dataset parameters specified via the `data.*` and `diffusion.*` namespaces. The dataset class is set to `ObjectPickDataset`. Grasp counts per object are fixed at 100 via replacement sampling to prevent shape mismatches in batched tensor operations. The `neg_hncolliding` ratio is set to 0 throughout, as hard-negative colliding grasp generation requires FCL and cannot be computed on the Jetson. The on-policy discriminator training uses a discriminator ratio configuration where the final two indices correspond to the on-policy positive and negative grasp sets respectively, following the ratio convention of the GraspGen discriminator training interface [5]. Table 8 summarises the dataset statistics and key training configuration parameters.

TABLE 8: DATASET STATISTICS AND TRAINING CONFIGURATION FOR ALL THREE TRAINED MODELS

Parameter	Generator	Discriminator GT	Discriminat or On-Policy
Training objects	180	180	180
Validation objects	45	45	45
Total objects	227	227	227
Total positive grasp annotations	9,685	9,685	9,685 + 1,302 on-gen
On-generator positives	—	—	1,302
On-generator negatives	—	—	1,698
Dataset size (H5 cache)	~18 GB	~18 GB	~18 GB
Batch size (objects)	8	8	8
Grasps per object per batch	100	100	100
Training epochs	500	500	500
Total TensorBoard steps	6,000	8,500	8,500
Translation normalisation κ	~5.0 (dataset- adaptive)	—	—
FCL-free positive threshold	$d_{\text{surface}} \leq \text{clip}(r_{\text{obj}} \times 0.5, 1 \text{ cm}, 5 \text{ cm})$	—	—

Parameter	Generator	Discriminator GT	Discriminat or On-Policy
Diffusion timesteps T	10	—	—
Checkpoint location	robotiq_3f_g en_v3/epoch _500.pth	robotiq_3f_disc_v 3/epoch_500.pth	robotiq_3f_d isc_onpolicy _v3/epoch_5 00.pth

3.3.5 Grasp Generation Using Diffusion Models

The grasp generation component of the proposed framework is built upon the diffusion-based architecture introduced in GraspGen [5]. The core formulation models 6-DoF grasp pose generation as a Denoising Diffusion Probabilistic Model (DDPM) operating in the SE(3) Lie group [41], following the probabilistic framework and the SE(3) diffusion formulation for grasping [42]. The key advantage of this formulation over discriminative approaches is that it models the full multimodal distribution of feasible grasp poses rather than predicting a single deterministic output, enabling diverse and physically plausible grasp candidates to be sampled from the observed object point cloud.

3.3.5.1 Problem Formulation

Given an object-centric point cloud $\mathcal{X}$ as input, the objective is to generate a diverse set of 6-DoF grasp poses $g \in \text{SE}(3)$, where each pose encodes the full position and orientation of the gripper relative to the object. Following the formulation of GraspGen [5], the SE(3) space is factorised into $\text{SO}(3) \times \mathbb{R}^3$, treating the rotation and translation components separately, as both subspaces admit Euclidean Lie algebra representations suitable for the Gaussian noise process used in DDPMs.

During training, Gaussian noise is added progressively to ground-truth positive grasp poses $g \in \mathcal{G}^+$ at a randomly sampled diffusion timestep $t \in [0, T]$, producing a noised grasp $\tilde{g} = g + \epsilon$. The noise prediction network φ conditioned on the object point cloud $\mathcal{X}$ is trained to recover the added noise, with the training objective being the denoising loss:

$$\mathcal{L} = \|\epsilon - \varphi(t, \tilde{g}, \mathcal{X})\|_2^2$$

as stated, [5] where ϵ is the ground-truth noise, $\tilde{g}$ is the noised grasp, t is the diffusion timestep, and $\mathcal{X}$ is the conditioning point cloud. During inference, the trained network reverses this process iteratively, starting from random noise and progressively denoising it into a structured and physically plausible grasp pose. In accordance with the empirical findings of GraspGen [5], two separate denoising processes with dedicated schedulers are run for the translation and rotation components respectively, as this yielded better performance than a single unified DDPM over the full SE(3) space.

3.3.5.2 Translation Normalisation

Since the SO(3) rotation space is naturally bounded within $[-\pi, \pi]$ while translation in $\mathbb{R}^3$ is unbounded and scale-dependent, the translation outputs must be normalised before being processed by the network. Following the normalisation strategy of GraspGen [5], a dataset-derived normalisation multiplier κ is computed as:

$$\kappa = 1 / ((1/N) \sum_{i=0}^N (\max(t_i) - \min(t_i)))$$

where t_i is the translation component across all positive grasp poses $\mathcal{G}_i^+$ for object i , and N is the total number of training objects. This avoids the need for a manual hyperparameter grid search while providing a principled, dataset-adaptive scaling factor. For the Robotiq 3F dataset used in this work, κ was computed from the training split statistics prior to generator training and applied consistently across both the generator and discriminator stages.

3.3.5.3 Object Encoding with PointNet++

The noise prediction network φ is conditioned on a learned encoding of the input object point cloud. As the object encoder, this work uses **PointNet++** [46], which provides hierarchical geometric feature extraction through set abstraction layers operating on local neighbourhoods of the input point set. PointNet++ encodes both local surface structure and global shape context into a compact latent representation, which is then used to condition the diffusion denoising process via cross-attention layers in the noise prediction network [5]

Both the object point cloud and the noised grasp poses are centred to the point cloud mean prior to being passed through the network, ensuring translation-invariant conditioning. During inference, a noise vector of the desired batch size is sampled and the diffusion reverse process is executed for $T = 10$ denoising steps substantially fewer than the hundreds of steps typical in image diffusion models. This reduced step count is sufficient for the grasp domain

because the grasp dimensionality (3 for translation + 3 for rotation) is significantly lower than that of image or video generation tasks [5].

3.3.5.4 Rotation Representation

For the rotation component, this work uses the **Lie algebra** representation in $SO(3)$, bounded within $[-\pi, \pi]$ consistent with the implementation, which reports that the Lie algebra, 6D rotation representation, and Euler angles all perform comparably when inputs are properly normalised to $[-1, 1]$ before being passed into the diffusion model. The Lie algebra representation was retained for its geometric interpretability and natural compatibility with the $SE(3)$ factorisation [5].

3.3.5.5 Inference

At inference time, the trained generator receives the object-centric point cloud from the `/env_cam/points` topic and samples a batch of grasp poses through the iterative denoising process. The generated grasps are subsequently passed to the discriminator for scoring and filtering. The mean warm inference time measured on the Jetson AGX Orin is approximately 1.85 seconds per inference call, producing between 87 and 108 scored grasp candidates per object, with top discriminator confidence scores reaching up to 0.997 on verified test inputs.

3.3.6 On-Generator Training of the Discriminator

A generative model trained solely on successful grasp data is prone to producing false positive candidates due to model fitting errors, as discussed in GraspGen [5]. Grasps generated by the diffusion model may be slightly in collision with the object surface, positioned as spatial outliers far from the object, or contain small rotation errors rendering them physically unstable failure modes that are not well represented in the offline training dataset of ground-truth positive and negative grasps. A discriminator is therefore trained separately to score and filter the generated grasps before they are forwarded to the motion planner.

3.3.6.1 Distribution Shift Between Offline and On-Generator Data

The central motivation for on-generator discriminator training is that the distribution of grasps produced by the trained diffusion model differs measurably from the distribution of grasps in the offline dataset. In the offline dataset, negative grasps are typically sampled to be either far from the object or in gross collision with it, and do not capture the subtle

near-surface or marginal-rotation failure modes introduced by diffusion model fitting errors. As demonstrated in GraspGen [5] using Earth Mover's Distance (EMD) as a distributional divergence measure, this shift is particularly pronounced for negative grasps, whose spatial manifold is substantially broader than that of positive grasps and therefore more sensitive to distributional misalignment. A discriminator trained only on offline data consequently fails to identify these generator-specific failure cases, leading to false positives being forwarded to the robot.

3.3.6.2 On-Generator Dataset Construction

To address this, a two-stage training procedure is adopted following the training recipe [5]. In the first stage, the generator is trained to convergence on the dataset. In the second stage, the trained generator is run in inference mode over the training object corpus to produce a large collection of on-policy candidate grasp poses approximately 300 grasps per object across the 227-object set. These on-generator grasps are then annotated using the FCL-free TCP proximity labelling method, assigning positive and negative labels based on surface proximity and pre-grasp clearance. This **on-generator dataset** captures the specific distribution of grasps that the diffusion model produces in practice, including its characteristic failure modes. The complete three-stage training recipe from generator training through on-policy dataset construction to discriminator training is illustrated in Fig.20.

The on-generator positive and negative grasps are stored in the H5 cache alongside the original offline grasps, indexed at discriminator ratio positions 5 and 6 respectively in the `data.discriminator_ratio` configuration [0.25, 0.20, 0.00, 0.05, 0.0, 0.25, 0.25]. The combined on-generator dataset produced in this work contains **1,302 positive and 1,698 negative on-policy annotated grasps** distributed across the training object corpus.

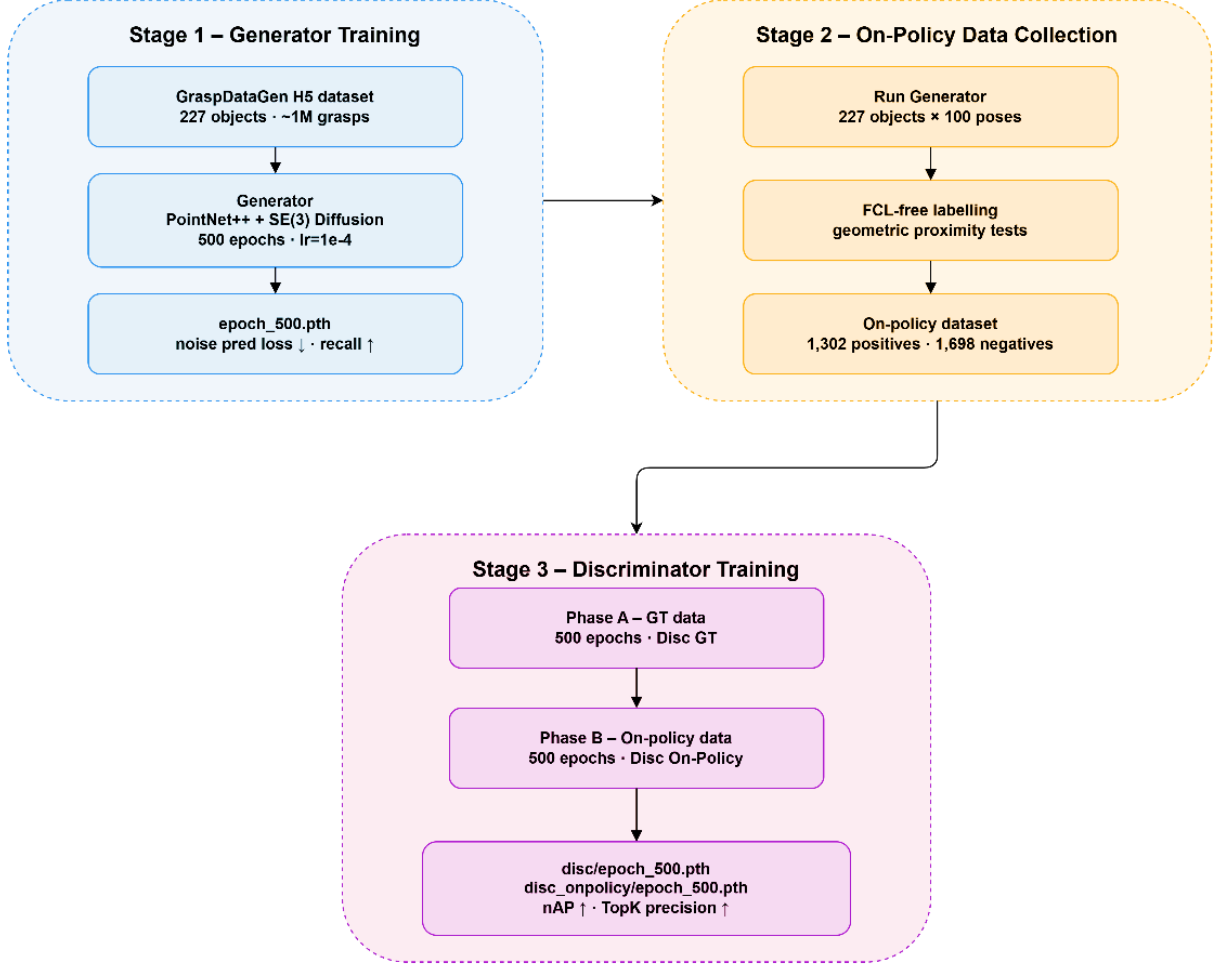


Fig.20. Three-stage on-generator training recipe

3.3.6.3 Discriminator Architecture

The discriminator architecture reuses the PointNet++ object encoder from the trained generator with frozen weights, and appends a lightweight MLP that takes as input the concatenated object embedding and an SE(3) grasp pose representation, producing a sigmoid-valued grasp quality score [5]. This design avoids training a separate object encoder from scratch for the discriminator a significant saving in parameters and GPU memory compared to earlier architectures such as GraspNet [50], where the grasp pose was converted to a canonical gripper point cloud and passed through a separate PointNet++ encoder. Only the final MLP layers are trained from scratch using a binary cross-entropy loss; the object encoder weights inherited from the generator remain frozen throughout. This results in a discriminator that is both more memory-efficient and better calibrated to the specific failure distribution of the diffusion generator.

3.3.6.4 Training Procedure

The discriminator is trained in two phases. In the first phase, it is trained on the offline ground-truth dataset of positive and negative grasps from the corpus to establish baseline grasp quality classification, producing the checkpoint at `~/GraspDataGen/training_logs/robotiq_3f_disc_v3/epoch_500.pth`. In the second phase the on-generator training phase it is retrained using the on-policy annotated grasps, with the discriminator ratio configuration directing sampling towards the on-generator positive and negative sets. The `neg_hncolliding` ratio is set to 0 throughout, as hard-negative colliding grasp generation requires FCL which is unavailable on ARM64. The final on-policy discriminator checkpoint at `~/GraspDataGen/training_logs/robotiq_3f_disc_onpolicy_v3/epoch_500.pth` is used for all inference and real-robot experiments in this work. Fig. [X] shows the training loss and key metric curves for both the generator and on-policy discriminator across 500 epochs from the TensorBoard logs.

3.3.7 Perception Pipeline

The perception pipeline is responsible for converting raw depth sensor output into a clean, object-centric point cloud suitable for GraspGen inference. The pipeline operates differently depending on whether the system is in simulation or real-hardware mode, but produces an identical output on the `/env_cam/points` topic in both cases, preserving the unified downstream architecture.

3.3.7.1 Point Cloud Acquisition

In the real hardware pipeline, point cloud acquisition is performed by the OAK-D Wide wrist camera through a three-pose scanning sequence. The operator initiates the sequence by publishing a `std_msgs/msg/Empty` message to `/pick/start`, which triggers the `grasp_executor_node` to begin. The arm is first driven to scan pose 1 via a direct joint trajectory command that bypasses MoveIt2 and OMPL entirely, completing in approximately 12 seconds. The joint configuration at this pose referred to as `SCAN_JOINTS` places the wrist camera directly above the pick tray with the depth axis pointing downward toward the workspace. The camera publishes synchronised RGB images, stereo depth images, and camera intrinsics at approximately 30 Hz via the `depthai_ros_driver` package. The camera intrinsic matrix K , comprising the focal lengths f_x , f_y and principal point c_x , c_y , is extracted from the `CameraInfo` message at each pose and used for depth back-projection.

Fig. 21 shows the point cloud data acquired via camera and grasp poses generated on it via GraspGen.

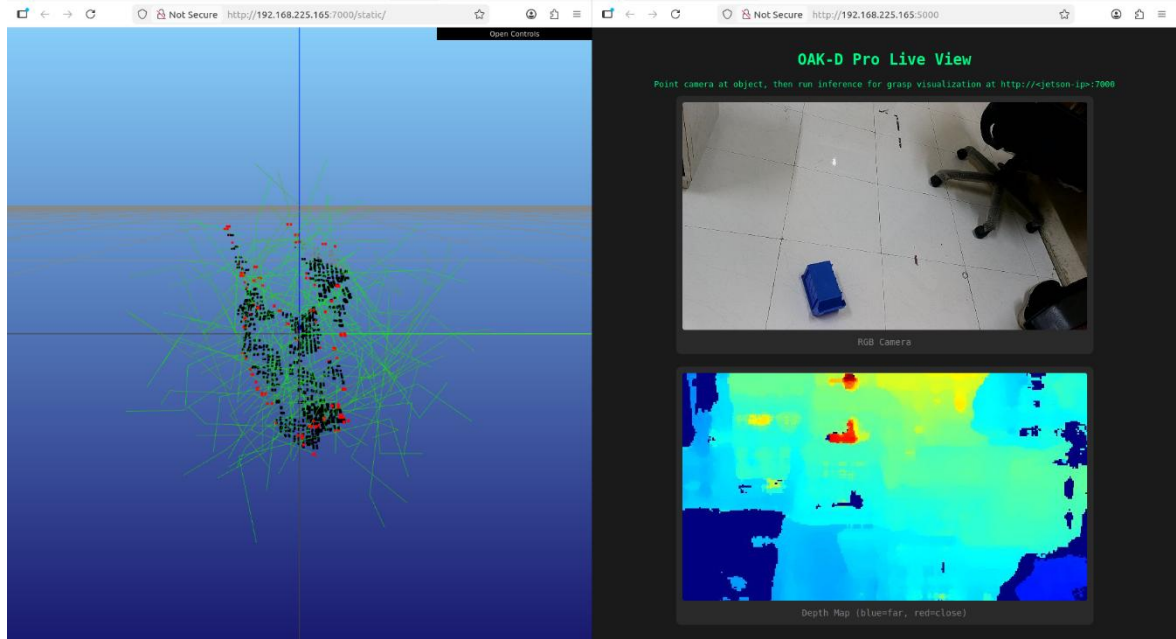


Fig.21. Generation of Point Clouds and Grasp Poses

3.3.7.2 SAM2-Based Interactive Object Segmentation

Once the arm reaches scan pose 1, the `grasp_executor_node` publishes to `/sam2/arm_ready`, which unlocks the click gate in the SAM2 segmentation node the top bar of the display window turns green to indicate readiness. The operator views the live RGB stream, which has been rotated 180° by the segmentation node to compensate for the camera's upside-down physical mounting, and left-clicks once on the target object. The click coordinates are converted back to raw-image space before being forwarded as a positive spatial prompt to the SAM2 [40] running on the Jetson GPU. SAM2 processes the prompt and generates a segmentation mask. If the initial mask overshoots into neighbouring objects or the background, the operator can add a negative prompt via right-click to refine the boundary; pressing R resets the selection entirely for a fresh attempt. Fig. 22 shows the SAM2 segmentation window with an operator click and the resulting mask.

3.3.7.3 Multi-Pose Scanning and Point Cloud Fusion

The masked depth pixels from scan pose 1 are back-projected to 3D using the standard pinhole camera model. For each masked pixel (u, v) with depth value d , the corresponding 3D point in the camera frame is computed as:

$$X = ((u - c_x) \cdot d) / f_x, Y = ((v - c_y) \cdot d) / f_y, Z = d$$

The resulting point set is transformed into the world frame via the robot's forward kinematics the joint encoder readings are propagated through the URDF kinematic chain to yield the exact camera pose at every configuration, requiring no manual calibration. This constitutes partial cloud 1, capturing the top face of the object.



Fig.22. SAM2 segmentation node showing operator left-click on target object (green mask) and RViz2 world-frame point cloud

The arm then automatically moves to scan pose 2 (SCAN_JOINTS_L), in which only wrist_2 is tilted +15° relative to scan pose 1 while all other joints remain unchanged. This wrist-only motion completes in approximately 5 seconds and provides a left-side view of the object without requiring the arm base to swing or OMPL to replan. The sam2_segmentation_node receives a /sam2/scan trigger and auto-captures the depth frame without further operator interaction, retaining only points within 0.25 m of the pose-1 centroid via a spatial filter this produces partial cloud 2, capturing the left face. The arm then moves to scan pose 3 (SCAN_JOINTS_R), with wrist_2 tilted -15° from scan pose 1, and the same auto-capture yields partial cloud 3 capturing the right face.

At every scan pose, two noise filtering steps are applied before the partial cloud is accepted: a world-frame height clamp between 0.13 m and 0.55 m removes the tray surface

and ceiling reflections, and statistical outlier removal using 20 neighbours and a 1.5σ threshold eliminates depth spikes. The three filtered partial clouds are then concatenated into a single near-complete object-centric point cloud, which is published on `/env_cam/points` and consumed by the `graspgen_bridge_node`. The arm returns to home while GraspGen inference runs on the fused cloud.

Table 9 summarises the joint configuration at each scan pose as measured on the teach pendant.

TABLE 9: SCAN POSE JOINT ANGLES MEASURED ON TEACH PENDANT. ONLY WRIST_2 CHANGES BETWEEN POSES 2 AND 3

Joint	Home	Pose 1 - top (SCAN_JOINTS)	Pose 2 - left (SCAN_JOIN TS_L)	Pose 3 - right (SCAN_JOINTS_R)
shoulder_pan	80.00 deg	90.52 deg	90.52 deg (unchanged)	90.52 deg (unchanged)
shoulder_lift	-90.00 deg	-94.38 deg	-94.38 deg (unchanged)	-94.38 deg (unchanged)
elbow	69.00 deg	67.46 deg	67.46 deg (unchanged)	67.46 deg (unchanged)
wrist_1	-69.00 deg	-63.07 deg	-63.07 deg (unchanged)	-63.07 deg (unchanged)
wrist_2	-90.00 deg	-89.97 deg	-74.97 deg (+15 deg)	-104.97 deg (-15 deg)
wrist_3	-15.00 deg	0.96 deg	0.96 deg (unchanged)	0.96 deg (unchanged)

3.3.7.4 Point Cloud Distribution and Training Considerations

The model used in this work was trained exclusively on complete point clouds reconstructed from object meshes, as the `scene_synthesizer` library required to render partial point clouds from camera viewpoints is not installable on the ARM64 Jetson platform. The three-pose scanning strategy described above partially mitigates this by fusing top, left-side, and right-side views into a more complete object representation than a single-viewpoint capture would provide. Nonetheless, occluded surfaces particularly the underside of objects resting on the tray remain unobserved, and the fused cloud still represents a partial

input relative to the complete mesh-rendered training data. This distribution gap is a known limitation of the current deployment and constitutes a direction for future retraining. Table 10 summarises the key differences between the simulation and real hardware perception pipelines.

TABLE 10: COMPARISON OF SIMULATION AND REAL HARDWARE PERCEPTION PIPELINES

Aspect	Simulation	Real Hardware
Point cloud source	sim_object_pc_publisher (synthetic)	wrist_cam (OAK-D Pro Wide, 3-pose scan)
Object segmentation	Hardcoded known position	SAM2 + operator left-click at scan pose 1
TF calibration required	No	No (wrist_cam TF from robot FK)
Number of scan poses	1 (static)	3 (top, wrist_2 +15 deg, wrist_2 -15 deg)
Operator interaction per pick	None (auto-trigger)	One click in SAM2 window
Point cloud completeness	Partial (top view only)	Near-complete (3-face fusion)
Noise filtering	None	Height clamp + statistical outlier removal

3.3.8 Simulation Environment

A simulation environment was developed in parallel with the real-hardware pipeline to support iterative development, debugging, and validation of the grasp generation and execution nodes without requiring physical hardware access. The simulation stack reuses the same ROS2 nodes, MoveIt2 configuration, and GraspGen inference pipeline as the real-world setup, with only the point cloud source differing between the two modes.

3.3.8.1 Robot and Scene Representation

The UR10 robot model including the Robotiq 3F gripper attached via the 3D-printed adapter slab and the OAK-D Wide wrist camera is described in a unified URDF and loaded by the robot_state_publisher node at launch. The robot is visualised and operated entirely within **RViz2**, with ros2_control mock hardware components providing simulated joint state feedback at 100 Hz through the joint_state_broadcaster. The joint_trajectory_controller and

`gripper_trajectory_controller` manage arm and gripper joint commands respectively, both operating over mock hardware in simulation.

The MoveIt2 planning scene is populated by a dedicated scene setup script that publishes the tabletop surface slab, a floor plane, and the target object collision primitives as `moveit_msgs/PlanningScene` messages. The script publishes the scene ten times with 0.3-second intervals at startup to ensure reliable ingestion by the MoveIt2 move group node before any grasp trigger is received.

3.3.8.2 Synthetic Point Cloud Generation

In simulation, the `sim_object_pc_publisher` node generates a synthetic cylindrical point cloud at the known position of the target object in the world frame and publishes it to `/env_cam/points`. This provides a deterministic, noise-free point cloud input that allows the GraspGen inference pipeline to be tested and validated independently of any real depth sensing. The GraspGen bridge node receives this synthetic point cloud identically to how it would receive a real camera point cloud, confirming that the inference and execution nodes are correctly integrated before transitioning to real hardware.

3.3.8.3 Simulation Validation

The full pick-and-place pipeline was validated in simulation prior to any real-hardware deployment. A cluttered tabletop scene was constructed comprising a primary target object surrounded by five distractor objects a bottle, a boba tea cup, a mug, a spray bottle, and a small can placed at varying positions and orientations within the arm's reachable workspace, with minimum clearances of approximately 6.6 cm between objects. This configuration was designed to reflect the kind of constrained manipulation scenario encountered in practice, where the motion planner must route the arm through narrow gaps while the grasp generation component operates solely on the target object's point cloud, remaining unaffected by the surrounding distractors.

Two complete sequential pick-and-place executions were confirmed successfully in this environment, with the arm correctly approaching the target, closing the gripper, lifting the object clear of the distractors, transporting it to the placement location, and returning to the home pose. These results validated the integration of the GraspGen inference pipeline, the MoveIt2 planning stack, and the execution node prior to transitioning to physical

hardware. Fig. 23 illustrates the simulated scene in RViz2 showing the robot, the planning scene objects, and a representative grasp approach trajectory.

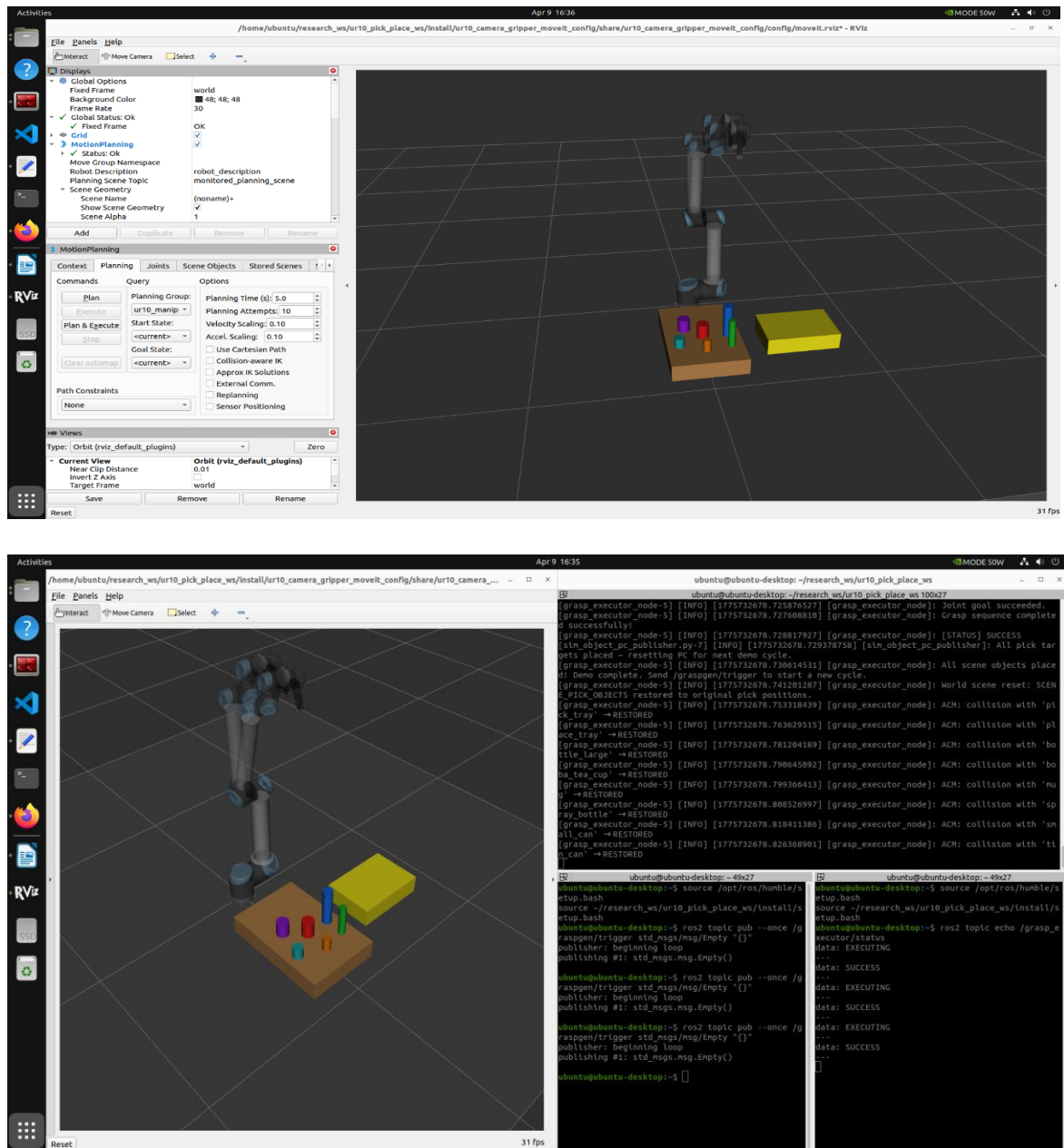


Fig.23. RViz2 planning scene showing UR10 arm, pick tray (brown), place tray (yellow) and Target objects

3.3.9 Motion Planning and Grasp Execution

The grasp execution pipeline translates the scored grasp pose received from the GraspGen discriminator into a complete pick-and-place motion on the UR10 arm and Robotiq 3F gripper. All motion planning is performed by **Movelt2** using the **OMPL**

planning library, with the UR10 kinematic model and Robotiq 3F gripper group defined in the SRDF.

3.3.9.1 Inverse Kinematics and Seed Strategy

A persistent challenge in deploying Cartesian goal-based motion planning on the UR10 is that the IK solver KDL by default in MoveIt2 can produce degenerate solutions such as elbow-flip configurations when seeded from poorly conditioned joint states. To address this, the executor node reads the current joint configuration from `/joint_states` at each planning request and uses it as the IK seed, ensuring continuity with the current arm configuration. For the pre-grasp pose, an intermediate forward-look joint-space goal is first commanded to place the arm in a well-conditioned forward-reaching posture, from which KDL reliably finds a valid elbow-up IK solution for the subsequent Cartesian pre-grasp target. The pre-grasp height is fixed at a world-frame Z of 1.25 m 23.5 cm above the tabletop which was empirically found to avoid collisions with the table slab while remaining within the arm's reachable workspace.

A further consideration in IK seeding concerns the coordinate frame relationship between the world frame and the robot's `base_link` frame. In the real hardware configuration, the URDF was corrected to define this transform as the identity confirmed via `tf2_echo` as translation (0, 0, 0) and rotation (0, 0, 0, 1) ensuring that MoveIt2's internal kinematic model matches the physical TF tree. Under this configuration, the expected `shoulder_pan` seed for a grasp target at world-frame position (x, y) is computed as `expected_pan = atan2(grasp_y, grasp_x)`, clamped to the range $[60^\circ, 160^\circ]$ to prevent the solver from seeding toward configurations that would direct the arm behind itself toward the placement zone. An additional `wrist_2` seed of -90° is applied to match the arm's operational posture and guide KDL toward the correct elbow-up solution. The clamp bounds were determined empirically from the reachable workspace geometry and proved effective in producing consistent, physically sensible IK seeds across the full range of object positions on the pick tray.

3.3.9.2 Eight-Step Pick-and-Place Sequence

The complete pick-and-place motion is decomposed into eight steps, each with specific tolerance and collision settings that reflect the physical requirements of that phase. The sequence begins with the arm moving to a fixed ready pose through a joint-space goal, which places the manipulator in a known, repeatable starting configuration from which all

subsequent planning is seeded. This is followed by a forward-look intermediate motion that positions the end-effector in a forward-reaching posture, chosen specifically to provide the IK solver with a well-conditioned seed joint state for the Cartesian goals that follow. Without this intermediate step, the KDL solver was found to produce degenerate elbow-flip solutions for targets in the lower portion of the workspace, a problem that this seeding strategy reliably eliminates.

Once a valid pre-grasp IK solution is obtained, the arm descends to a pre-grasp position held at a fixed world-frame height above the target object, with the gripper approach axis aligned to the orientation predicted by the GraspGen discriminator. The arm then continues its descent to the grasp pose itself, at which point collision checking for the immediate gripper zone is relaxed this is necessary because the fingers must enter the close vicinity of the object surface to achieve a valid grasp, and strict collision avoidance at this stage would prevent the arm from reaching the required contact position. The Robotiq 3F gripper is then commanded to close, conforming adaptively to the object geometry through its underactuated three-finger mechanism.

Following a successful closure, the arm lifts the grasped object vertically to a safe clearance height before the transport phase begins. During transport, full collision avoidance is reinstated, and OMPL plans a path to the designated placement location that routes around all remaining scene objects and workspace boundaries. At the placement location, the gripper opens to release the object, and the arm returns to the ready pose to complete the cycle. Throughout this sequence, tighter joint tolerances are enforced at the grasp and place steps, where positional precision directly affects task success, while looser tolerances are applied during transit motions where exact endpoint accuracy is less critical. The complete execution sequence is illustrated in Fig. 24. Integrated system execution showing segmentation, planning, and ROS2 node communication is shown in Fig. 25. In Fig.26. ROS2 computation graph is showing the whole pipeline interaction between perception, grasp generation, and execution nodes.

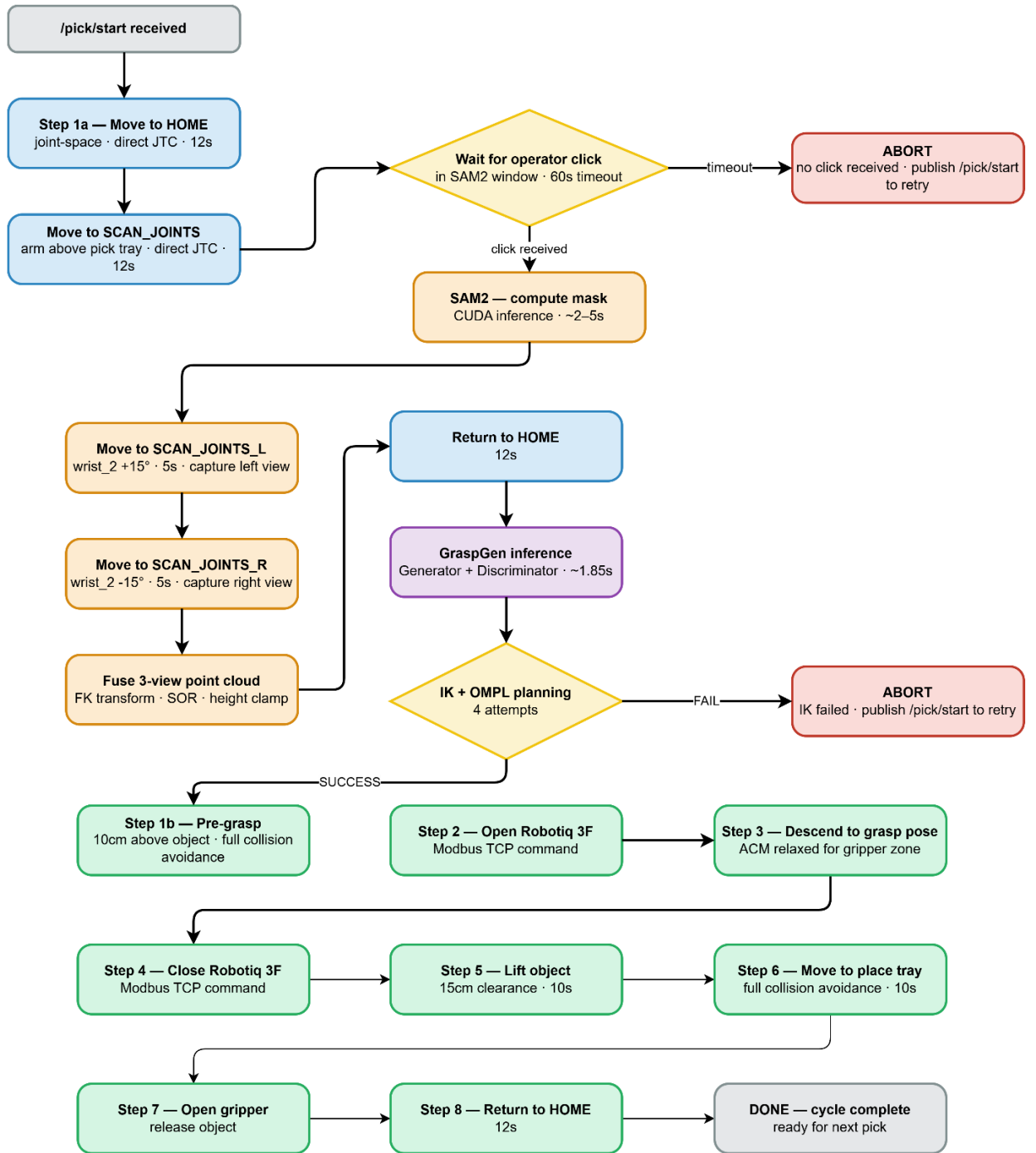


Figure 24. Eight-step pick-and-place execution sequence

3.3.9.3 Collision Management

The Allowed Collision Matrix is dynamically updated at each step to reflect the appropriate collision checking requirements. During the descent phase (Steps 2-3), distractor objects are exempted from collision checking, as the arm links pass close to their bounding volumes during a vertical approach without making physical contact in practice. During transport and placement (Steps 6-7), full collision avoidance is reinstated so that OMPL plans paths that route around all scene objects. In real-hardware mode, the object geometry is additionally registered as a `real_target` collision primitive in the planning scene after grasping, ensuring that MoveIt2 accounts for the attached object during transport planning.

3.3.10 Sim-to-Real Transfer Considerations

Bridging the gap between simulation-trained models and real-world deployment is a well-recognised challenge in robotic grasping. In this work, several specific sources of sim-to-real discrepancy are present and are addressed through deliberate design choices in both the training pipeline and the real-hardware integration.

3.3.10.1 Point Cloud Distribution Shift

The most significant sim-to-real gap in this work arises from the difference between the complete mesh-rendered point clouds used during training and the multi-view but still partially occluded point clouds produced by the OAK-D Wide wrist camera through the three-pose scanning sequence. The model was trained exclusively on complete point clouds because the `scene_synthesizer` partial-view renderer could not be installed on the ARM64 Jetson platform. As a consequence, real OAK-D point clouds which cover only the visible surface of the object represent an out-of-distribution input relative to the training data. Partial point cloud support is identified as a priority for future retraining, following the mixed complete-and-partial training strategy, which demonstrated that a 50-50 split between complete and partial inputs enables the model to generalise across both settings.

3.3.10.2 Sensor Noise and Depth Accuracy

Real depth sensors introduce measurement noise, depth holes at reflective or textureless surfaces, and boundary artefacts at object edges. The OAK-D Pro Wide stereo depth module is susceptible to these effects on smooth or shiny objects a category that overlaps with many common manipulation targets. At the pipeline level, the SAM2 mask boundary is used to exclude background points, partially mitigating edge artefacts. The

GraspGen training pipeline applies several point cloud augmentations during training including randomised orientation after mean-centring, random camera viewpoints, random subsampled point sets, and simulated instance segmentation boundary dilation to improve robustness to real-sensor imperfections. These augmentations are retained in the training configuration used in this work.

3.3.10.3 Camera Calibration

In the real hardware pipeline, no manual extrinsic calibration is required between the wrist camera and the robot world frame. The transform from the camera frame to the world frame is derived entirely from the robot's joint encoder readings propagated through the URDF kinematic chain via forward kinematics, with the static TF from `oakd_wide_link` to `wrist_cam_rgb_camera_optical_frame` encoding only the fixed physical offset of the sensor on the adapter slab and a yaw correction of π for the upside-down mounting. This approach eliminates the manual measurement step associated with tripod-mounted cameras and removes the associated risk of calibration drift between sessions any displacement of the hardware is automatically accounted for through the robot's own proprioception.

3.3.10.4 Grasp Labelling Domain Gap

A secondary source of sim-to-real discrepancy arises from the FCL-free proximity labelling method used to annotate the training dataset, which approximates grasp success through geometric proximity rather than physics simulation. Grasps labelled positive under this heuristic may not consistently succeed on the real Robotiq 3F due to differences in finger compliance, surface friction, and contact dynamics that are not modelled during labelling. This FCL-free proximity method represents a deliberate trade-off, and its effect on real-world grasp success rate is examined in the experimental results of Chapter 4.

3.3.10.5 Real Hardware Parameter

At the software level, sim-to-real transition is handled by the `real_hardware` parameter in `grasp_executor_node`. When false (default), it uses known simulation object positions for IK seeding and collision setup. When true, it estimates object geometry from the point cloud, dynamically registers a `real_target` collision primitive, and uses GraspGen-predicted grasp XY directly. This parameterised design enables seamless sim-to-real transition via launch arguments, with no code changes required.

CHAPTER 4

RESULT AND DISCUSSION

4.1 Generator Training Results

The diffusion-based generator was trained on the Robotiq 3F dataset comprising 181 training objects and 46 validation objects over 500 epochs on the Jetson AGX Orin platform with one TensorBoard step corresponding to one gradient update on a single batch of 8 objects giving 12 steps per epoch for the generator (6,000 total steps at epoch 500) and 17 steps per epoch for the discriminator (8,500 total steps at epoch 500). The training progress was monitored through TensorBoard, and the resulting curves for all key metrics are presented in Figs. 27 through Fig. 32.

4.1.1 Training and Validation Loss

The total training loss, shown in Fig. 27, begins at approximately 4.80 at the start of training and decreases steadily to approximately 1.21 by epoch 500, representing a reduction of roughly 75% over the full training run. The validation loss follows the training loss closely throughout, with no significant divergence observed between the two curves. This close tracking between training and validation loss indicates that the model is learning generalisable features from the training data rather than overfitting to specific object geometries, which is particularly encouraging given the relatively small training corpus of 181 objects compared to the 8,515 objects used in the original GraspGen work.

The noise prediction loss the primary training objective is shown separately in Fig. 28. This curve exhibits a consistent downward trend throughout training, confirming that the diffusion denoising network is progressively learning to predict and remove the noise added to ground-truth grasp poses. The noise prediction component begins at 2.40 and decreases

to 0.60 by step 6,000, accounting for approximately half of the total loss reduction. The smooth and stable descent of this curve, without any sudden spikes or divergence, reflects numerically stable training conditions on the Jetson platform despite the ARM64-specific adaptations applied to the codebase.

4.1.2 Translation Error

The L2 translation error between the predicted and ground-truth grasp positions, plotted in Fig. 29, decreases from an initial value of approximately 1.60 m reflecting random initialisation at the start of training to approximately 0.41 m by epoch 500 for the training set, while the validation reconstruction L2 error decreases from 1.18 m to 0.16 m. The validation reconstruction error converges to a lower final value than the training error, which is consistent with the reconstruction metric being evaluated on denoised outputs rather than on mid-diffusion noise predictions. The progressive reduction in translation error confirms that the generator is learning to place grasp proposals in geometrically meaningful proximity to the object surface rather than sampling uniformly across space, which is the key functional requirement for the grasp generation stage of the pipeline.

4.1.3 Rotation Error

Two rotation error metrics are tracked: the geodesic rotation error and the ϕ_3 rotation error, shown in Figs. 30 and 31 respectively. The geodesic rotation error which measures the angular distance between predicted and ground-truth orientations on the $SO(3)$ manifold decreases from approximately 2.16 radians at initialisation to approximately 1.02 radians by epoch 500 for the training set, while the validation reconstruction geodesic error decreases from 2.20 radians to 1.88 radians. The ϕ_3 metric, which measures the third Euler angle component, decreases correspondingly from approximately 0.65 to approximately 0.30. Both metrics show smooth convergence with no instability, indicating that the separate rotation DDPM scheduler is functioning correctly and learning to refine gripper orientations independently of the translation component. The residual training geodesic error of approximately 1.02 radians reflects the inherent difficulty of rotation learning at this dataset scale. The higher validation reconstruction value of 1.88 radians indicates that the model generalises rotation predictions less readily than translation, which is consistent with the known challenge of $SO(3)$ generalisation across unseen object geometries.

4.1.4 Validation Recall

The validation recall curve, shown in Fig. 32, reflects the proportion of ground-truth positive grasps that are matched by the generator's predictions within a spatial proximity threshold. The recall rises from near zero at initialisation and reaches approximately 0.112 by the end of training. While this value is lower than the recall figures reported in the original GraspGen paper where training on over 8,000 objects with the full Isaac simulation shaking pipeline produced significantly higher spatial coverage it is consistent with the reduced dataset scale and the FCL-free proximity labelling approach used in this work, which produces a sparser set of positive training grasps. The recall metric also reflects the inherent difficulty of generalising across the three-finger adaptive morphology of the Robotiq 3F, which admits a more constrained set of valid approach directions compared to a symmetric parallel-jaw gripper. This result is reported honestly as a limitation of the current training scale and labelling pipeline, and is consistent with the lower-than-paper recall.

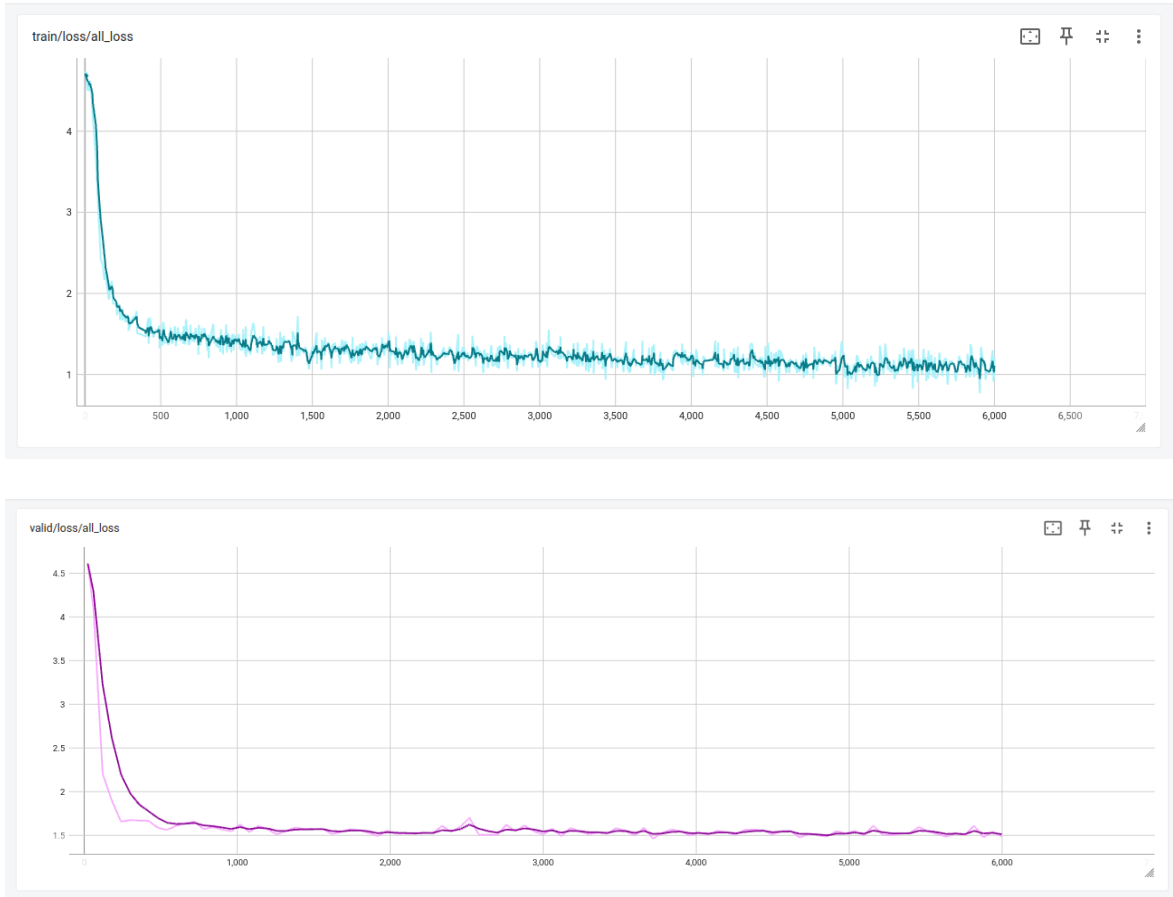


Fig.27. Generator training and valid total loss

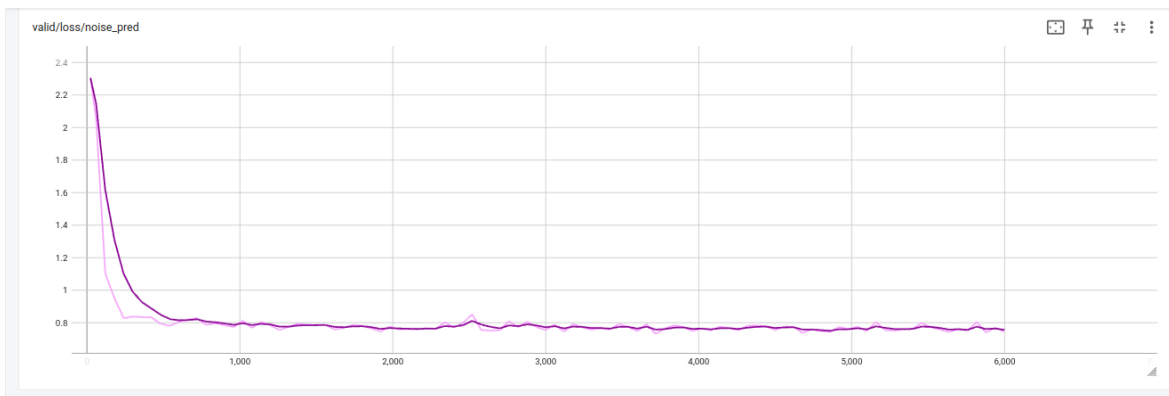
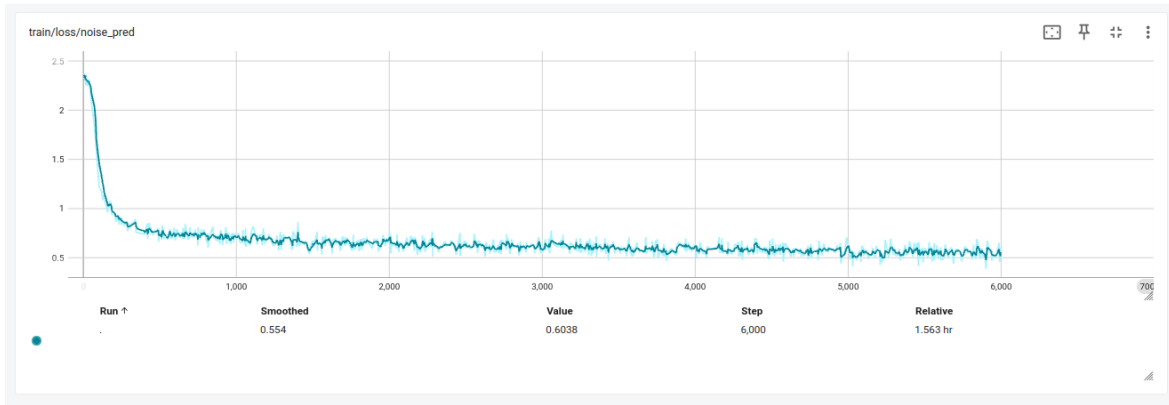
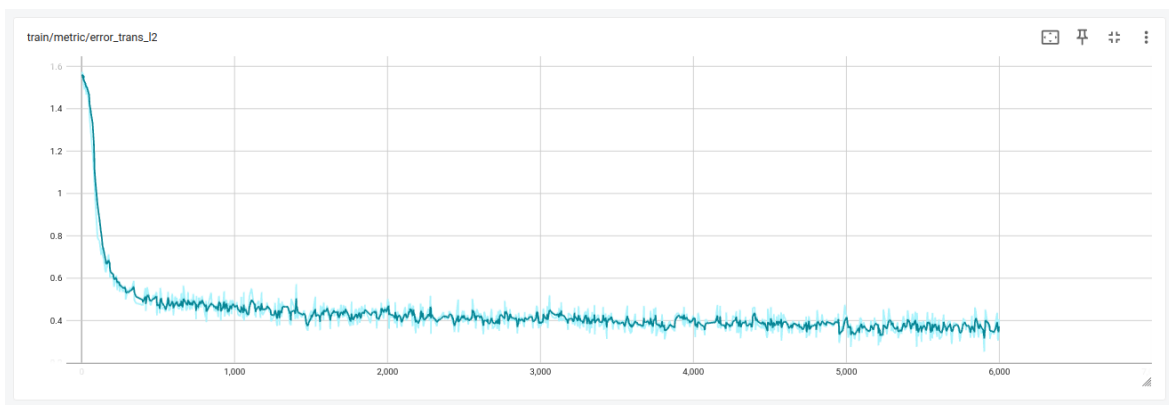


Fig.28. Generator Training and Validation noise prediction loss



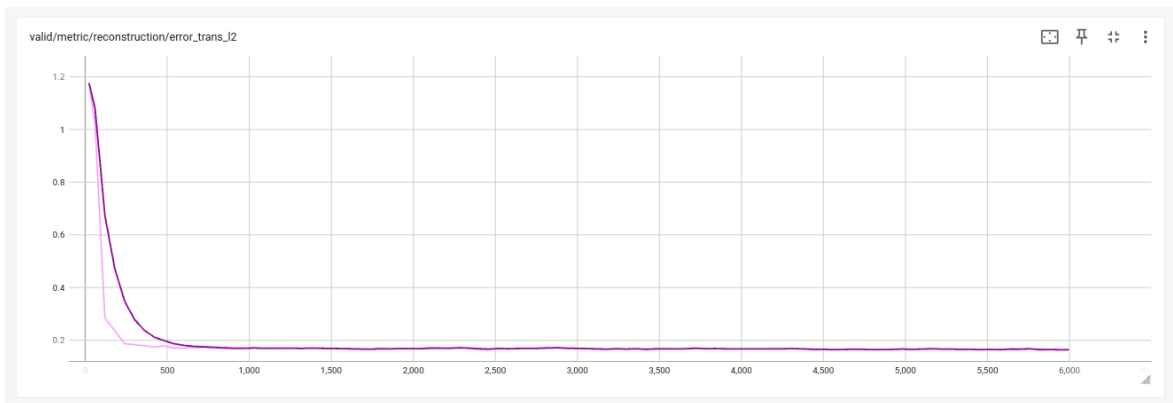
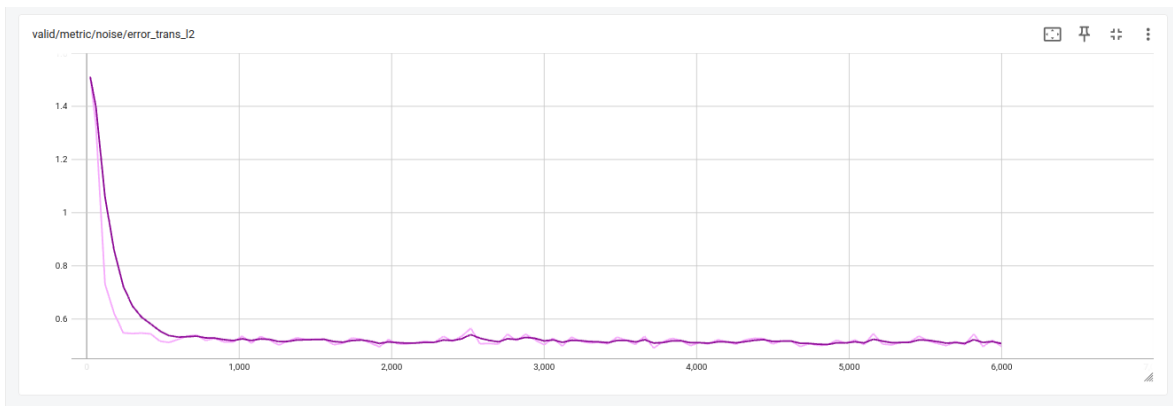
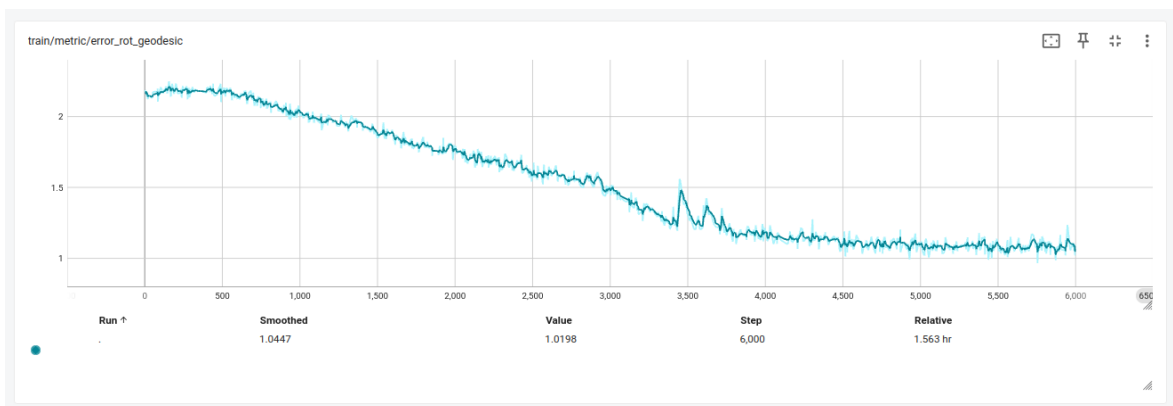


Fig.29. Generator Training and Validation translation L2 error



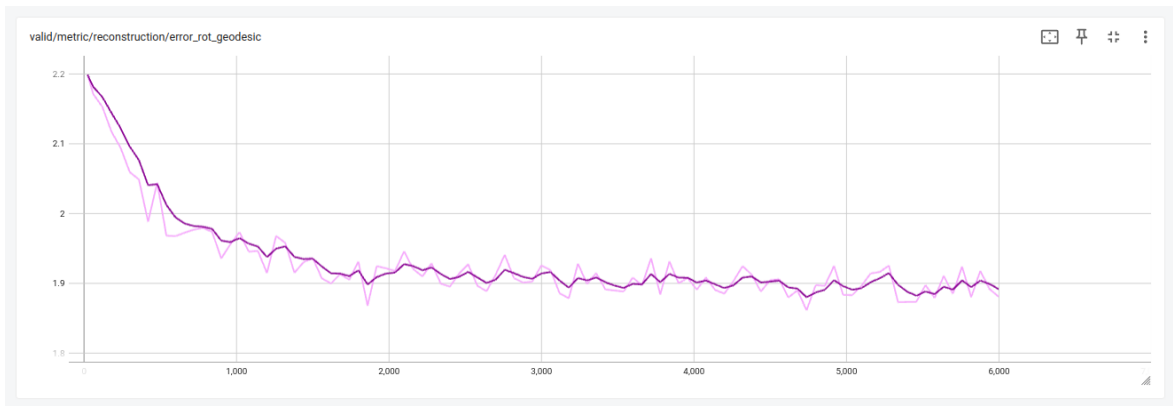
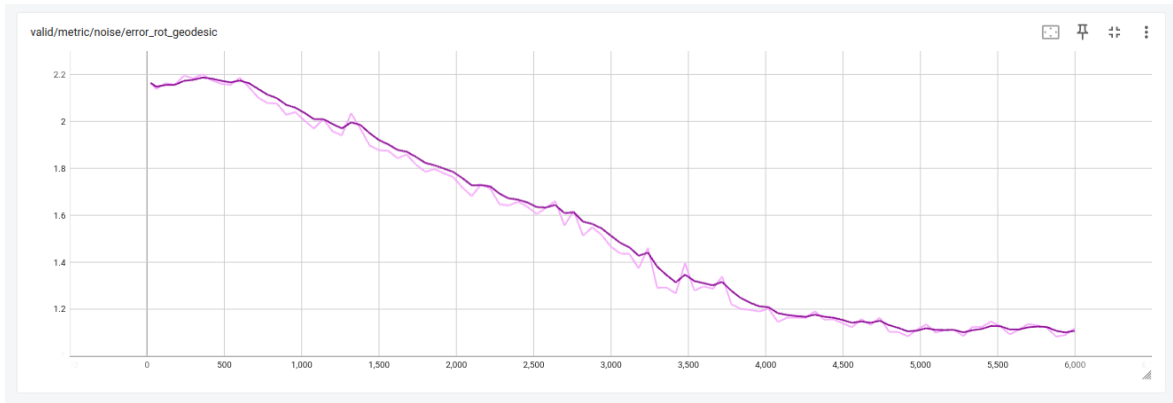
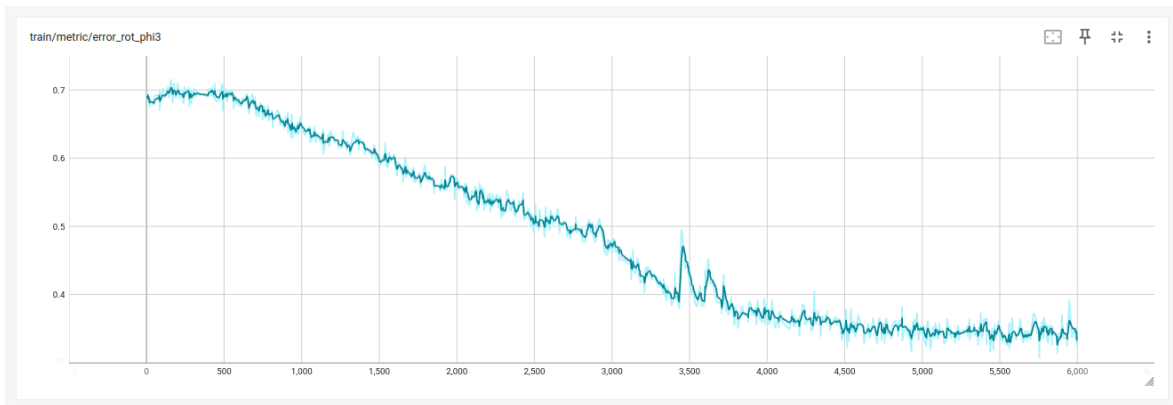


Fig.30. Generator Training and Validation geodesic rotation error



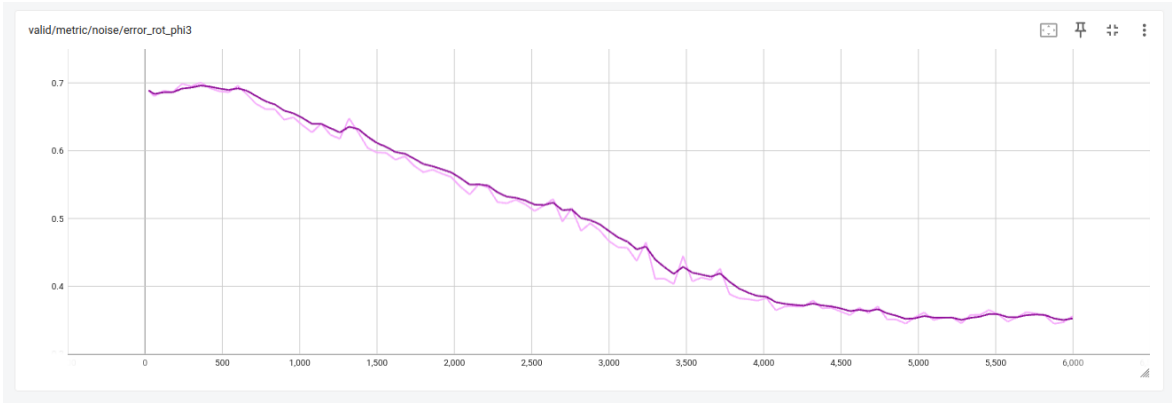


Fig.31. Generator Training and Validation phi3 rotation error

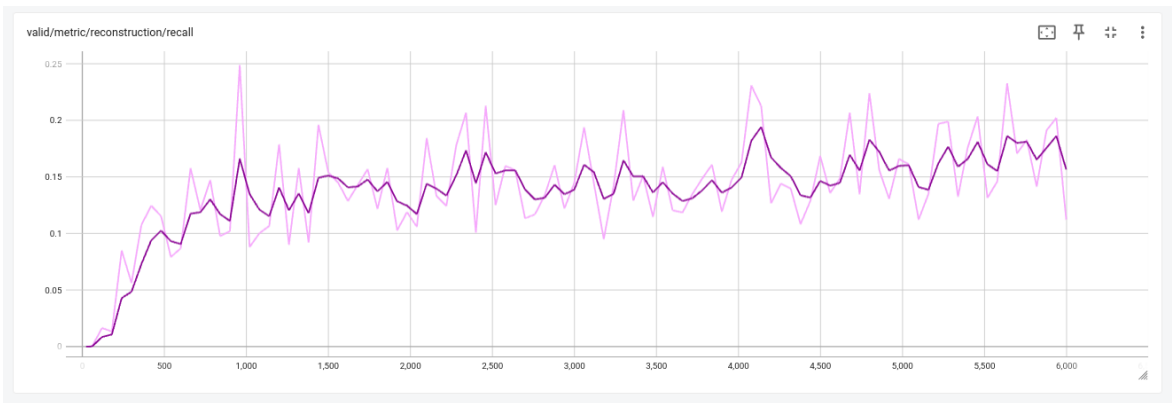
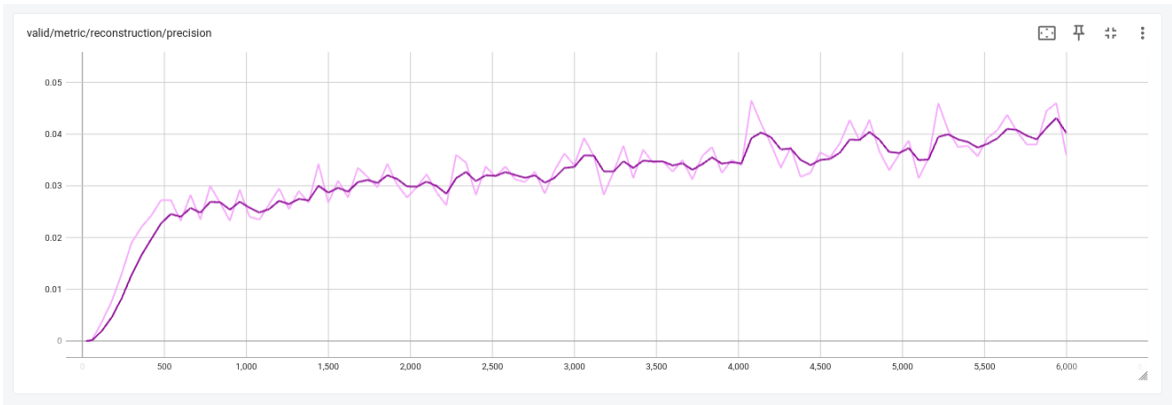


Fig.32. Generator validation reconstruction recall

4.2 Discriminator Training Results

The discriminator was trained in two sequential phases, first on the offline ground-truth dataset (GT phase), and subsequently on the on-generator annotated dataset (on-policy

phase). The training curves for the GT discriminator are presented in Figs. 33 through 41, covering the full set of loss and classification metric curves produced over 500 epochs across 137 training objects and 39 validation objects.

4.2.1 Total Loss and BCE TopK Loss

The total discriminator loss, shown in Fig. 33, begins at approximately 0.715 and decreases to approximately 0.064 by step 8,500, with the validation loss decreasing from 0.605 to 0.187 over the same period for the training split, with the validation loss following closely. The binary cross-entropy TopK loss the core training objective that penalises the discriminator for misranking grasp quality in the top-K selection is shown in Fig. 34. This loss decreases steadily from approximately 0.8 towards convergence, indicating that the discriminator is progressively learning to distinguish between high-quality and poor-quality grasp candidates in the ranked output. The close alignment between training and validation curves across both loss metrics suggests that the discriminator generalises well to unseen objects within the training distribution.

4.2.2 Average Precision

The Average Precision (AP) metric, shown in Fig. 35, is the primary evaluation metric for discriminator quality as it summarises the area under the precision-recall curve for the binary classification task. The AP rises from 0.405 at the start of training and converges to approximately 0.995 by step 8,500 for both the training and validation splits. This improvement reflects the discriminator's increasing ability to assign higher confidence scores to genuinely positive grasps and lower scores to negative ones. The validation AP rises from 0.913 to 0.988 over the same period, approaching perfect discrimination. These final values indicate that the discriminator has learned to reliably distinguish successful from unsuccessful grasps across both seen and unseen object geometries, despite the relatively small training corpus.

4.2.3 TopK Ratio Metrics

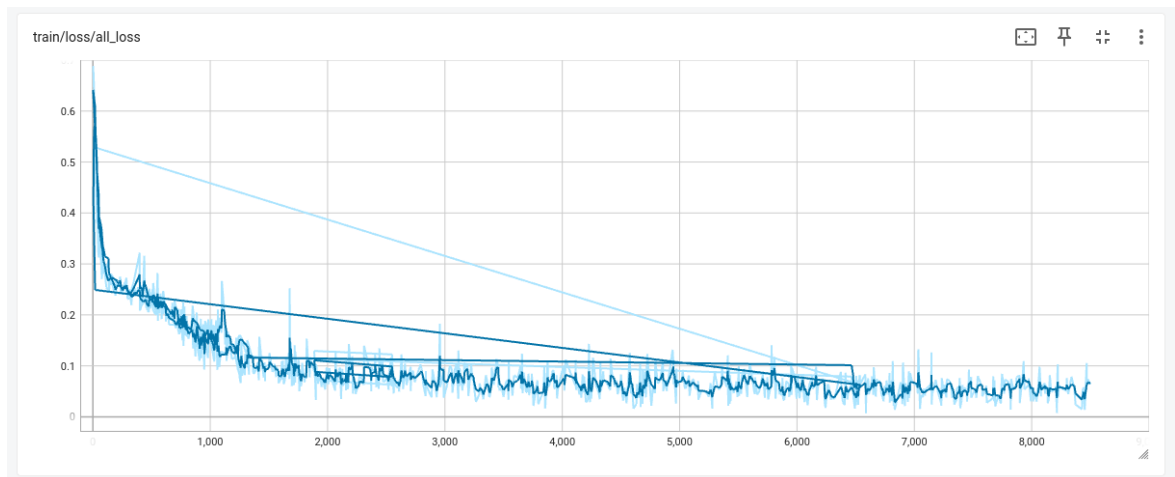
The TopK ratio metrics provide a more granular view of discriminator behaviour by examining what fraction of the top-K scored grasps fall into each ground-truth category. The TopK ratio for true positive grasps, shown in Fig. 36, reflects how often the discriminator correctly places genuinely good grasps at the top of the ranked list; this ratio is desired to be as high as possible. The true positive TopK ratio rises from 0.336 a non-trivial starting value

reflecting the discriminator's initial signal even before training begins and stabilises at approximately 0.667, indicating that the discriminator consistently promotes a meaningful fraction of real positive grasps to the top of the ranking.

Conversely, the TopK ratios for true negative grasps and free-space grasps shown in Figs. 37 and 38 respectively reflect how often bad grasps or spatially misplaced grasps are incorrectly ranked highly; both are desired to be as low as possible. The true negative TopK ratio decreases from 0.597 to 0.293 over training, and the free-space TopK ratio decreases from 0.067 to 0.040, both confirming that the discriminator learns to demote poor-quality candidates. The colliding grasp TopK ratio, shown in Fig. 39, remains near zero throughout training as expected, since the `neg_hncolliding` ratio was set to zero due to FCL unavailability on the ARM64 platform meaning the discriminator was not trained on any explicitly colliding grasp examples.

4.2.4 Per-Category Loss

The loss on positive grasps, shown in Fig. 40, decreases and converges towards a small positive value, reflecting the discriminator's growing confidence in correctly scoring real good grasps. The loss on negative grasps, shown in Fig. 41, similarly decreases and stabilises, confirming that the discriminator becomes progressively more certain in assigning low scores to bad grasp candidates. The symmetric convergence of both per-category losses is a positive indicator that the discriminator is not collapsing towards always predicting one class.



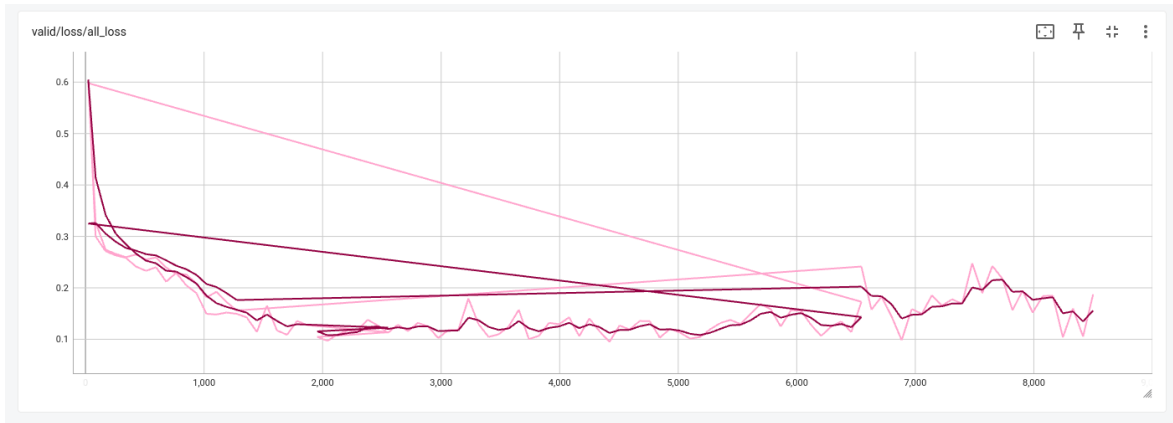


Fig.33. Discriminator total loss: training and validation

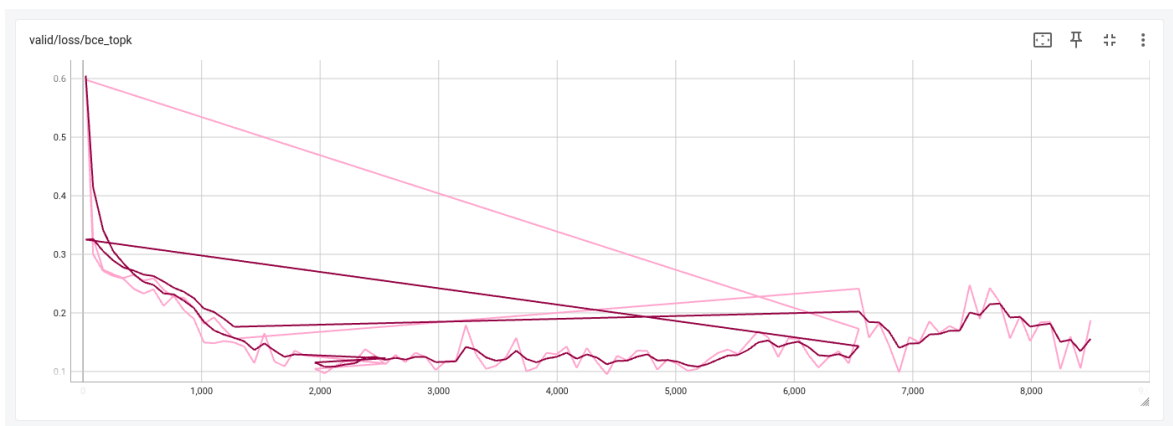
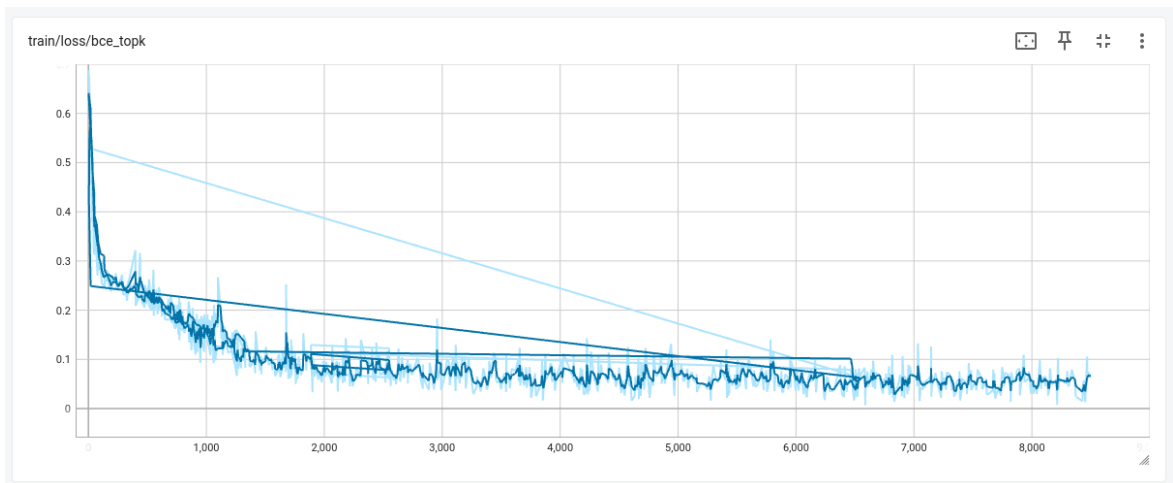


Fig.34. BCE TopK loss: training and validation

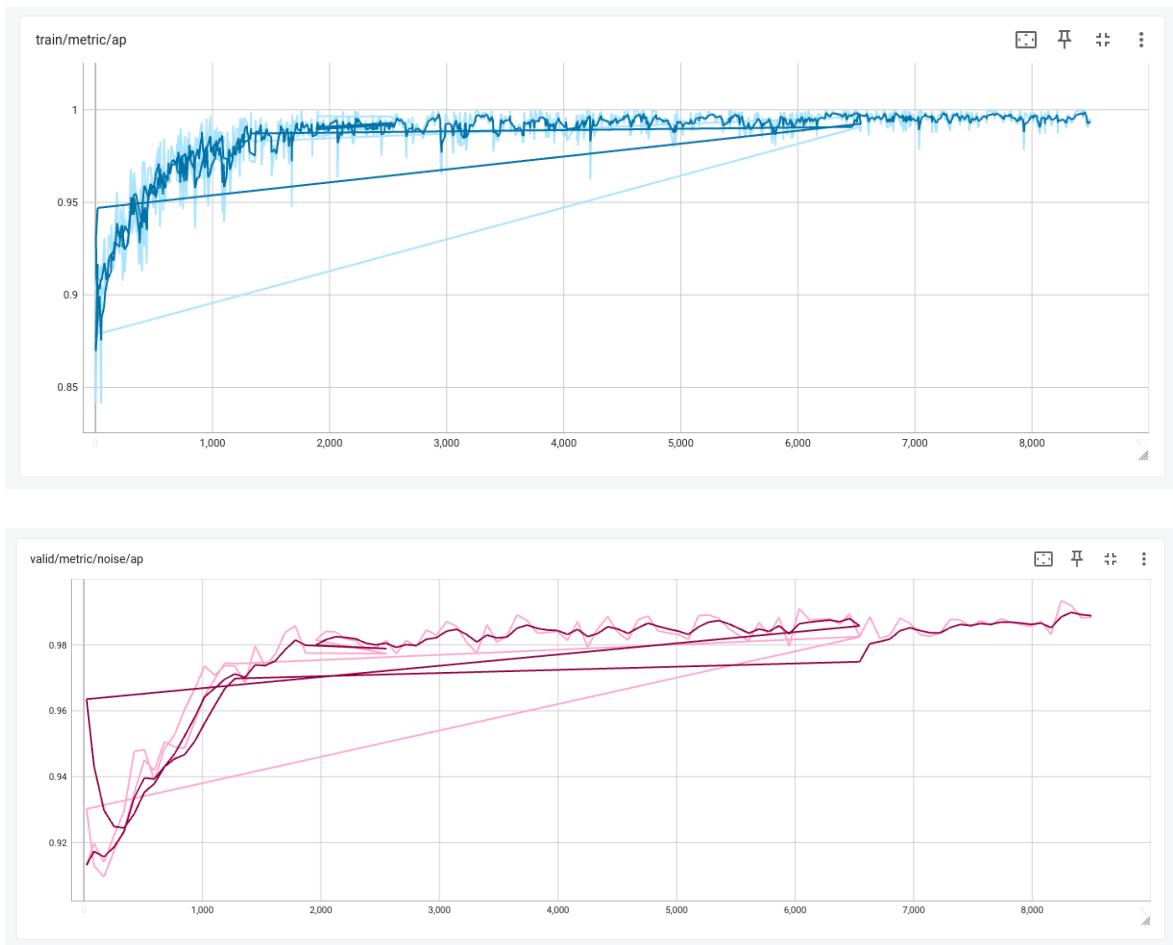
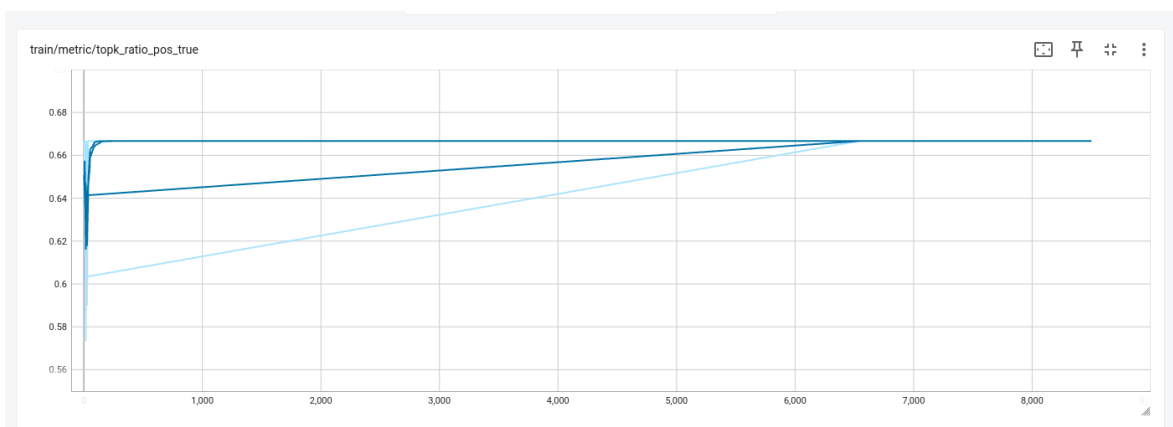


Fig.35. Average Precision (AP): training and validation



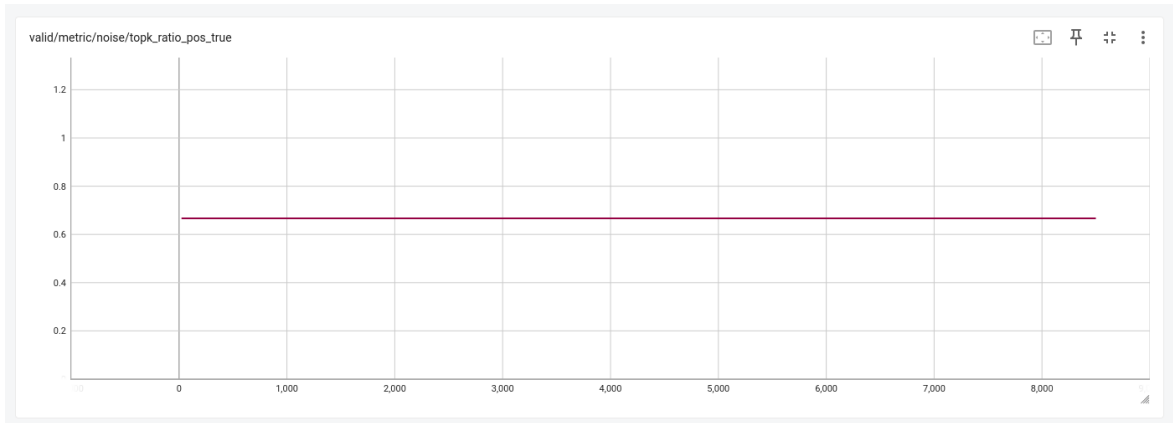


Fig.36. TopK ratio true positive grasps: Training and Validation

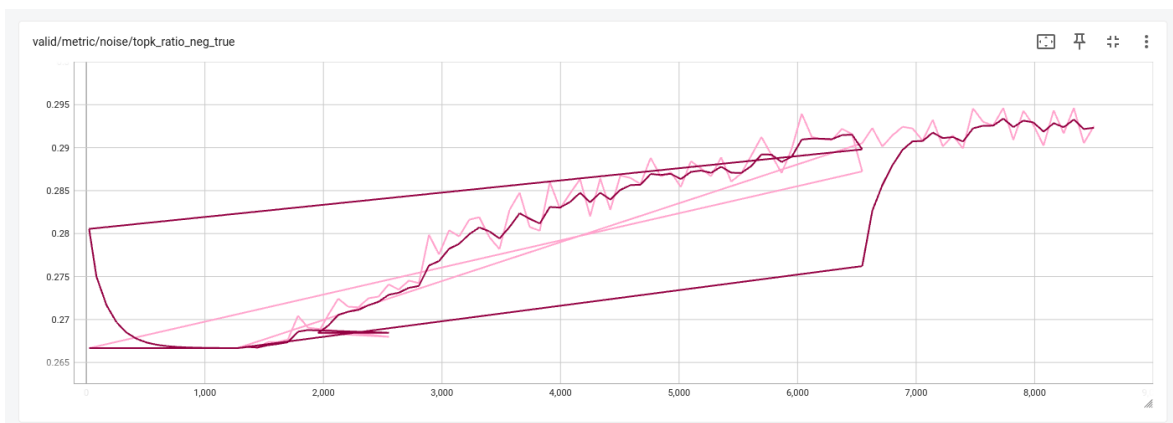
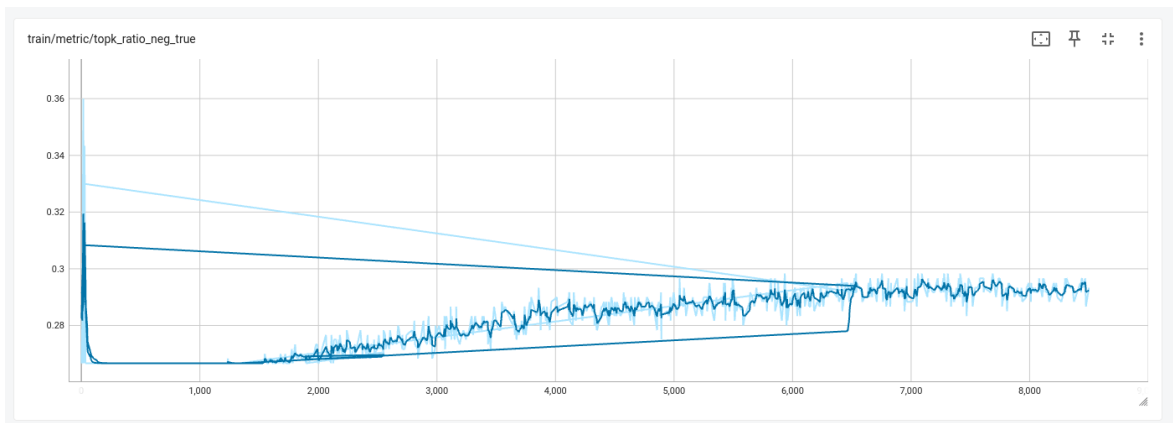
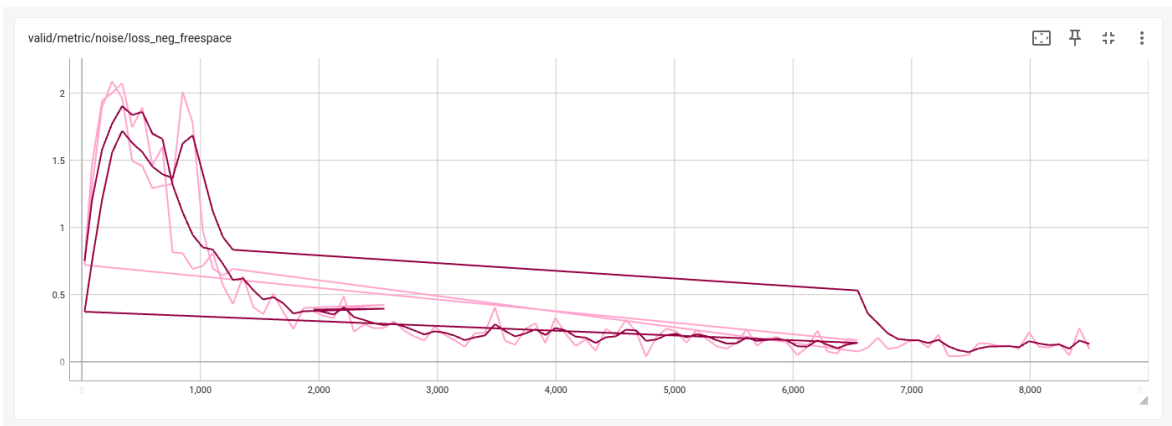
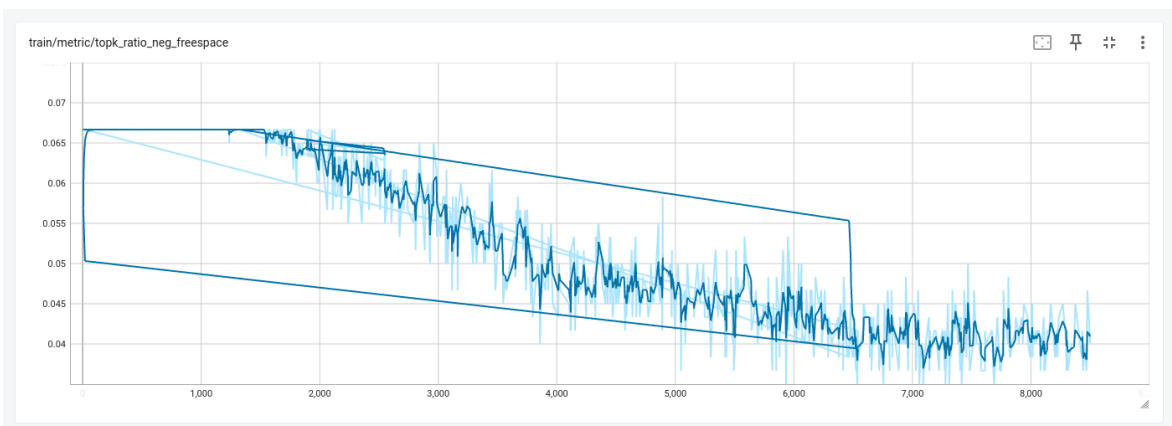
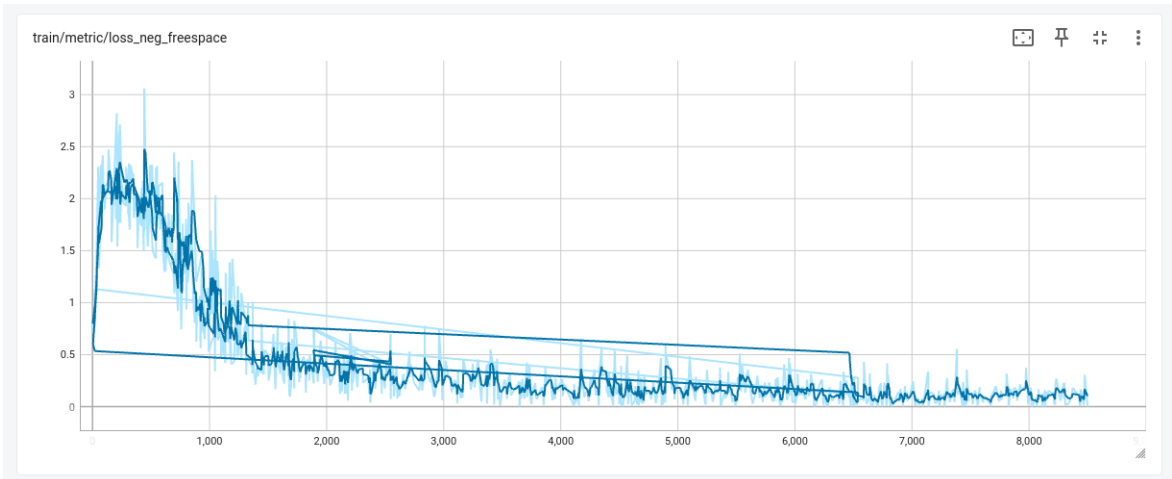


Fig.37. TopK ratio true negative grasps: Training and Validation



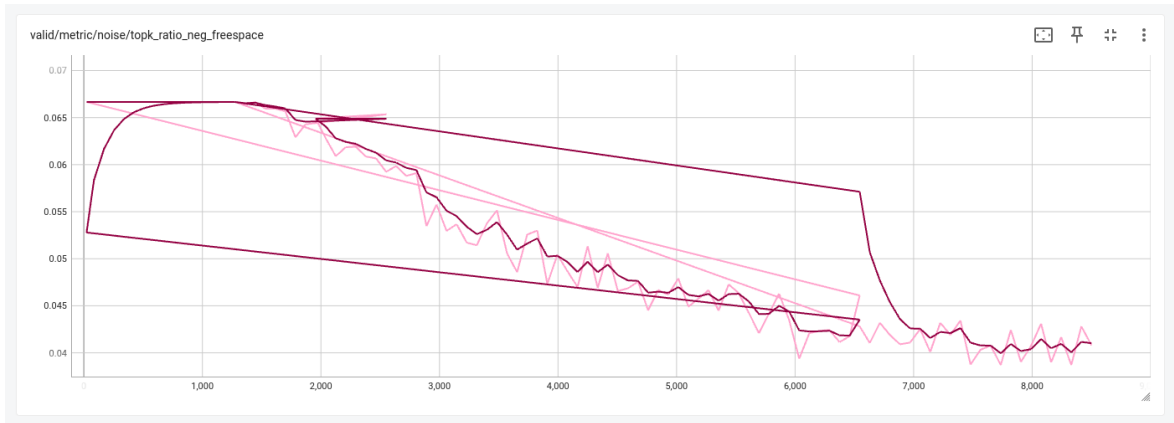


Fig.38. Loss and TopK ratio negative free-space grasps : Training and Validation

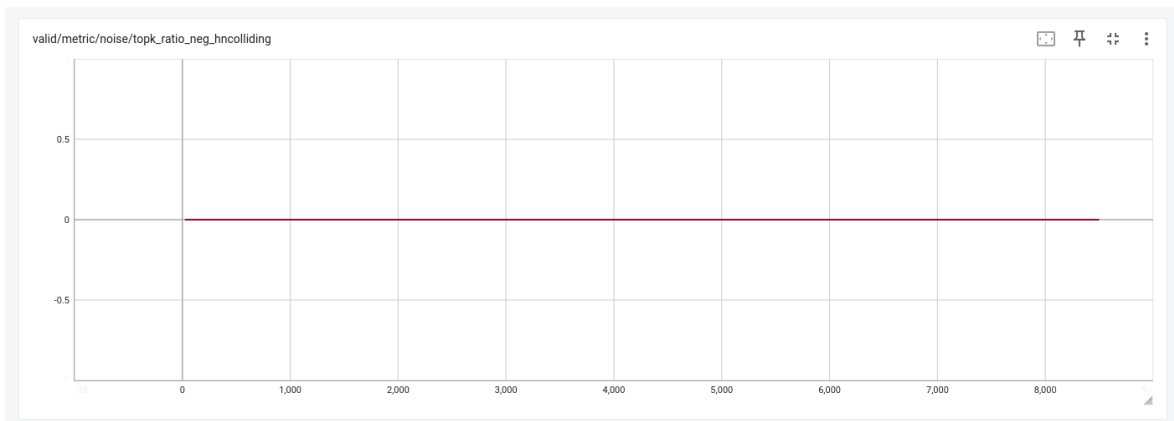
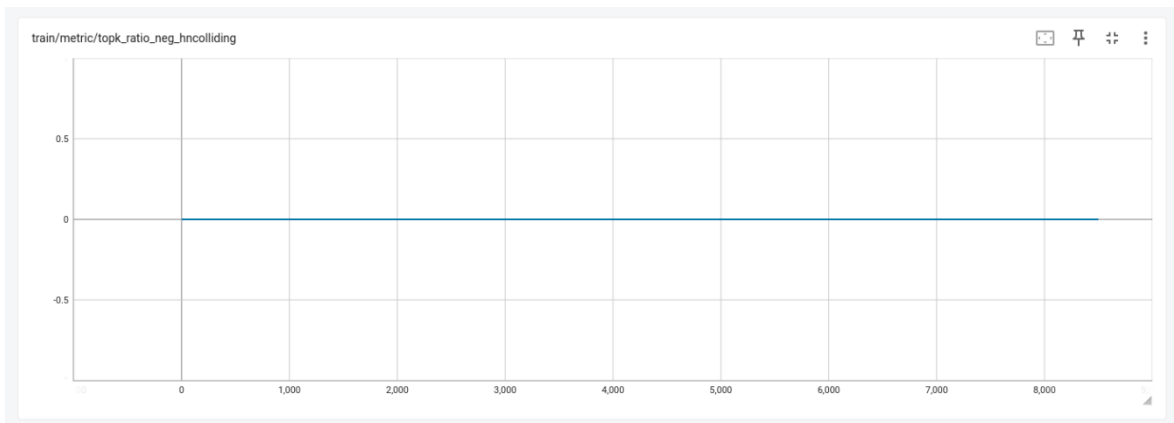


Fig.39. TopK ratio colliding grasps: Training and Validation

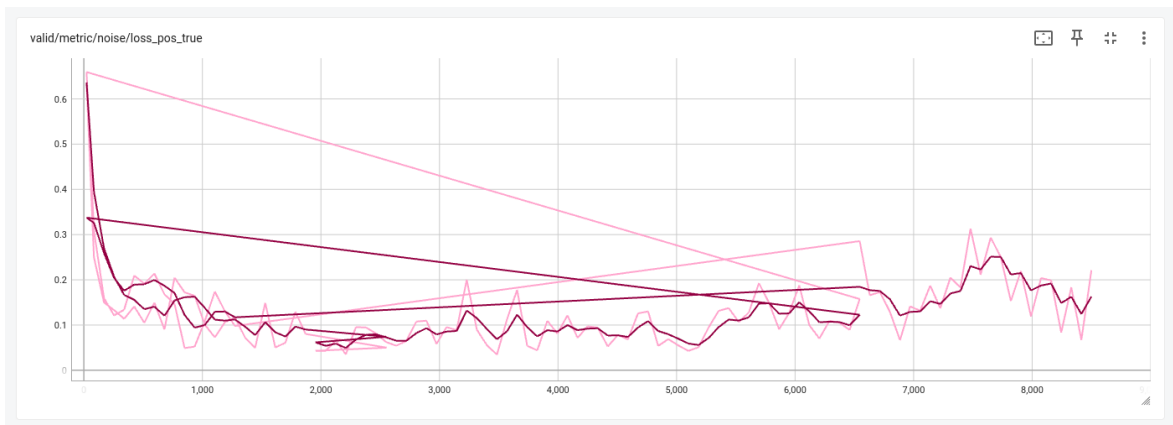
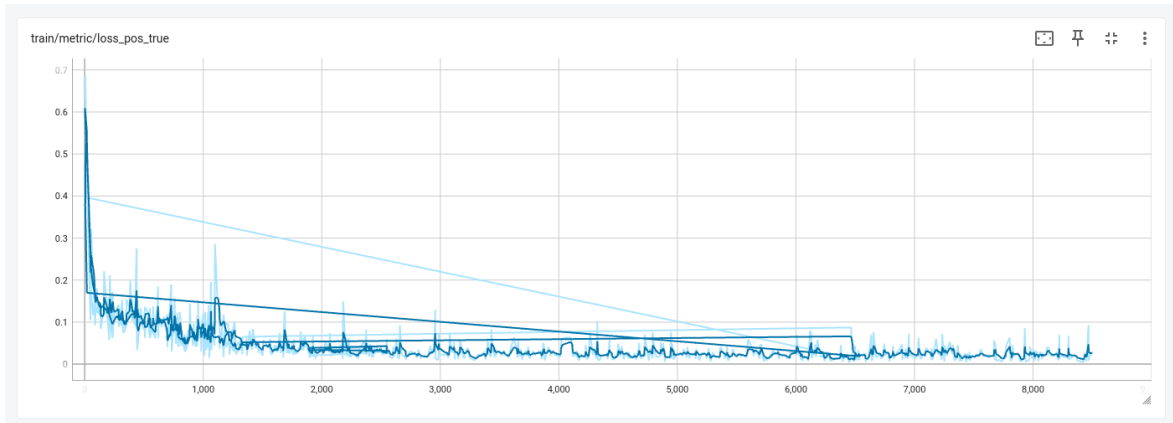
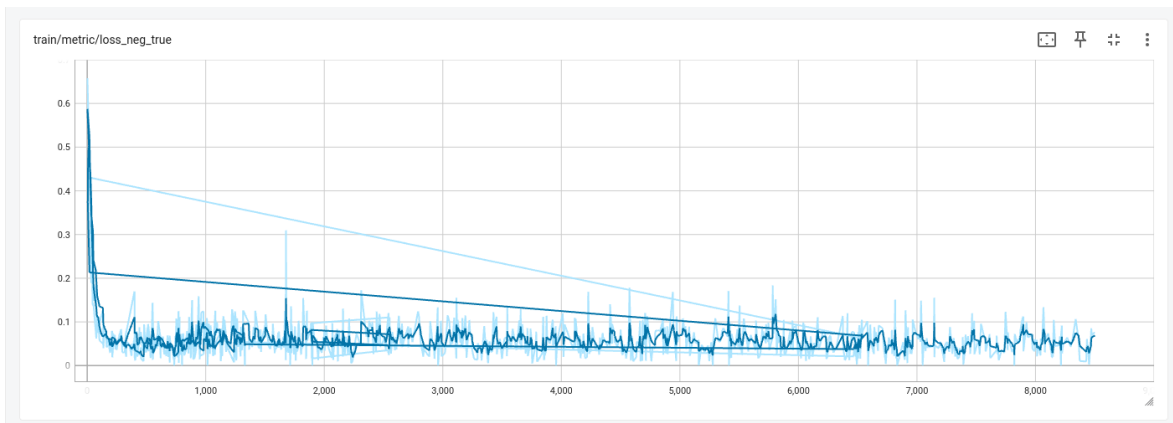


Fig.40. Loss on positive grasps: Training and Validation



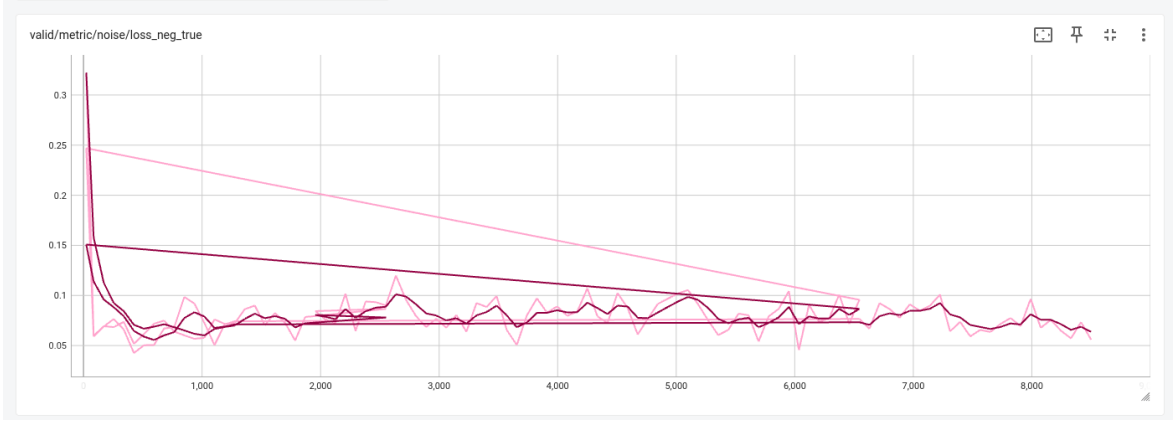


Fig.41. Loss on negative grasps: Training and Validation

4.3 On-Policy Discriminator Training Results

The on-policy discriminator was trained following the GT discriminator using the on-generator annotated dataset comprising 1,302 positive and 1,698 negative grasps sampled from the generator's own inference outputs.

The total training loss decreases from 0.710 to 0.275 by step 8,500 a less aggressive reduction than the GT discriminator ($0.715 \rightarrow 0.064$), which is expected given the harder negative examples drawn from the generator's own output distribution. The Average Precision rises from 0.397 to 0.970 on the training set and from 0.912 to 0.988 on the validation set, comparable to the GT discriminator's final values of 0.995 and 0.982 respectively. While training-set AP is marginally lower an expected consequence of training against harder generator-sourced negatives the validation AP matches the GT discriminator, consistent with the improved rank-quality observed in the precision-coverage analysis. The TopK pos_true ratio rises from 0.175 to 0.333, and the TopK neg_true ratio decreases sharply from 0.267 to 0.002, confirming that the on-policy discriminator effectively demotes the generator's false positive outputs. These results are consistent as shown in Fig. 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54 and 55 with the findings of the original GraspGen work, where on-generator training was shown to improve discriminator calibration by exposing it to the specific failure modes of the diffusion model.

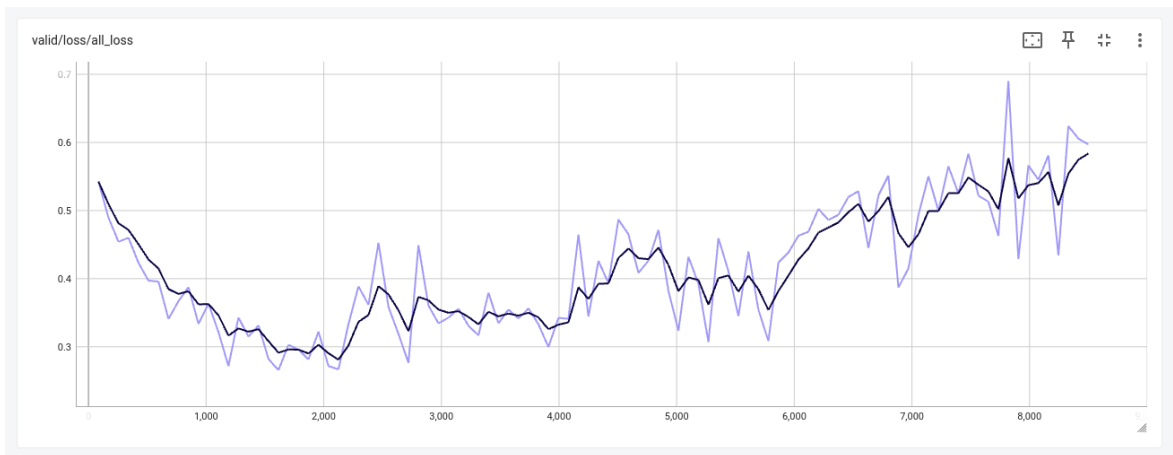
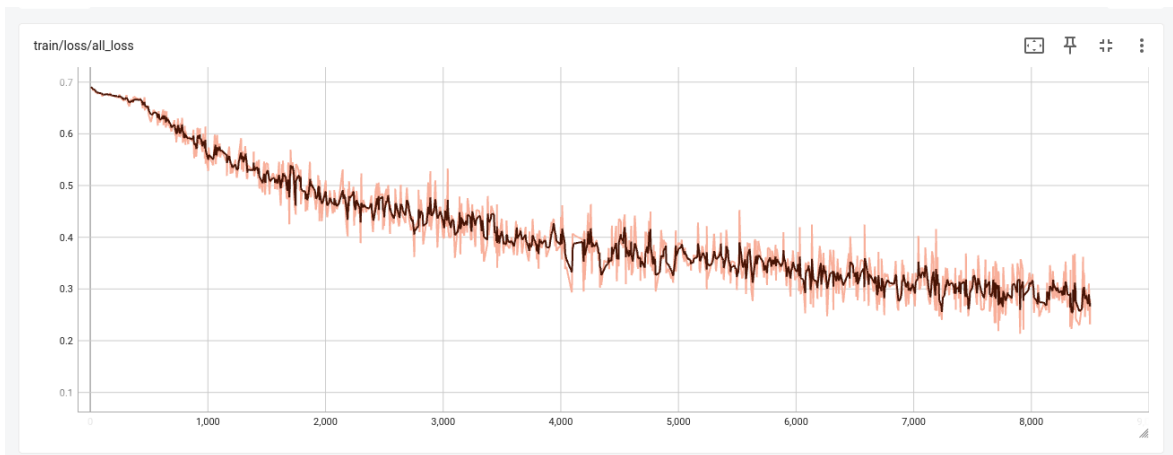
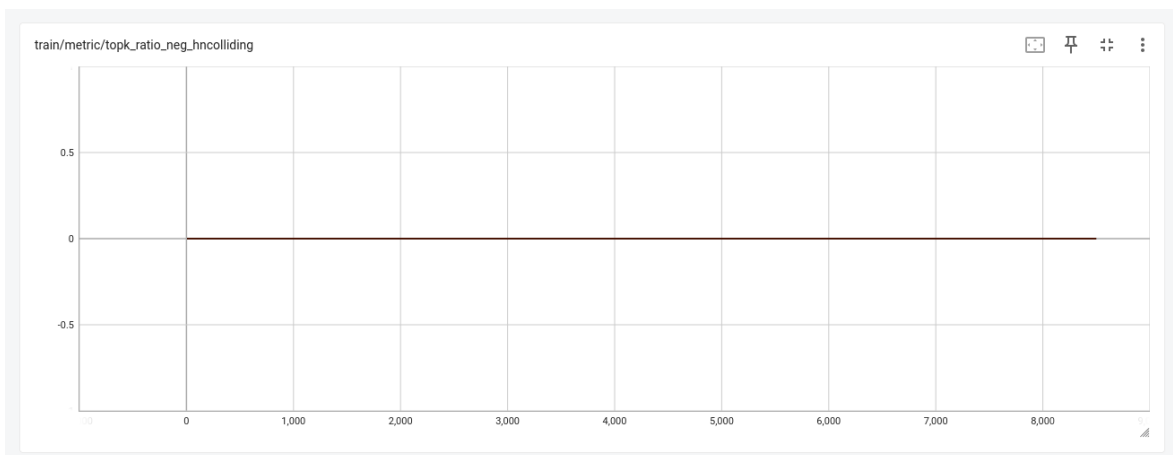


Fig.42. On-Policy Total Loss: Training and Validation



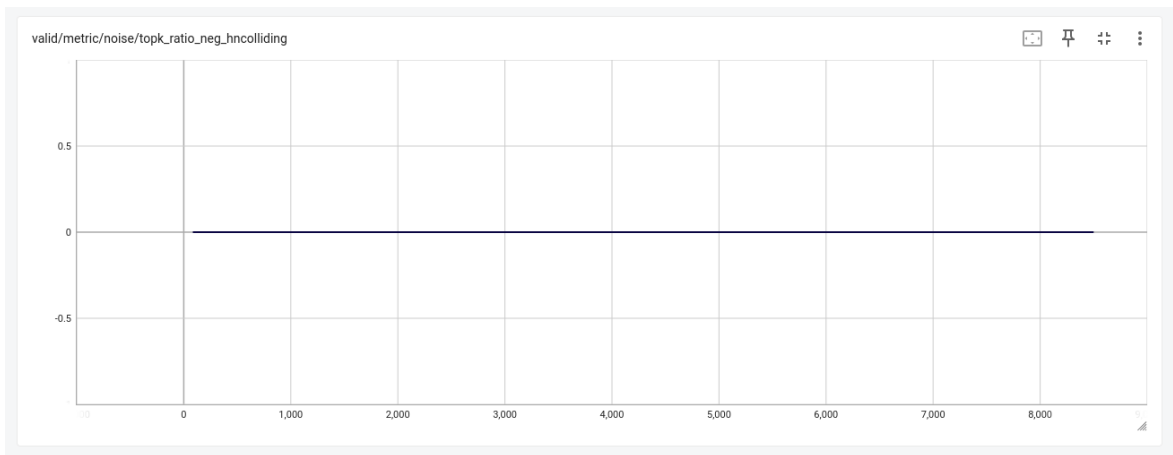


Fig.43. On-Policy Colliding Grasps: Training and Validation

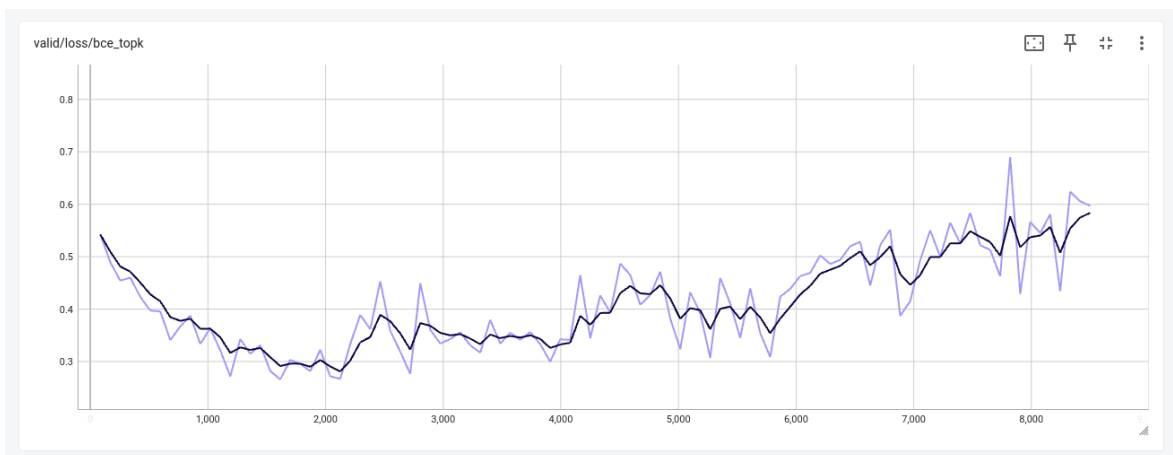
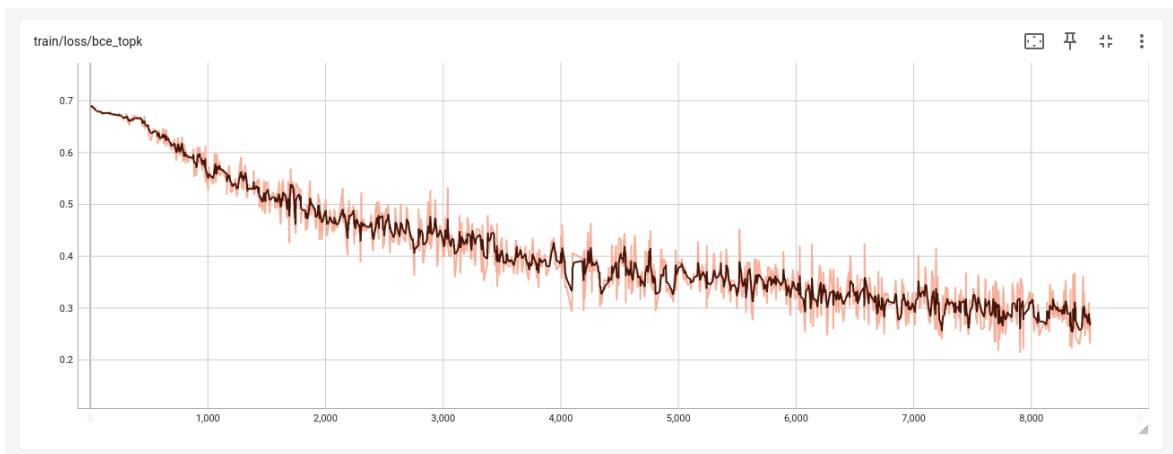


Fig.44. On-Policy BCE TopK loss: Training and Validation

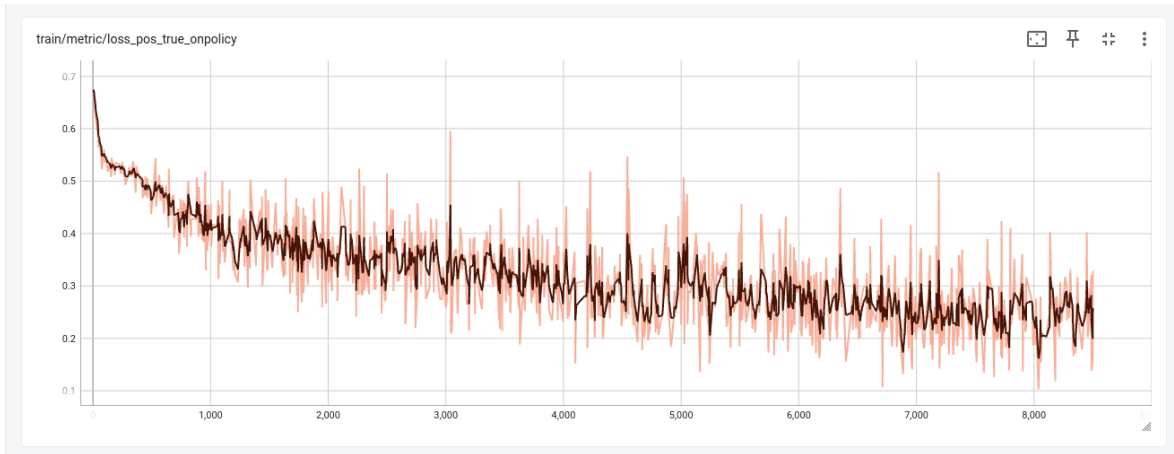


Fig.45. On-Policy Loss on Positive True Grasps

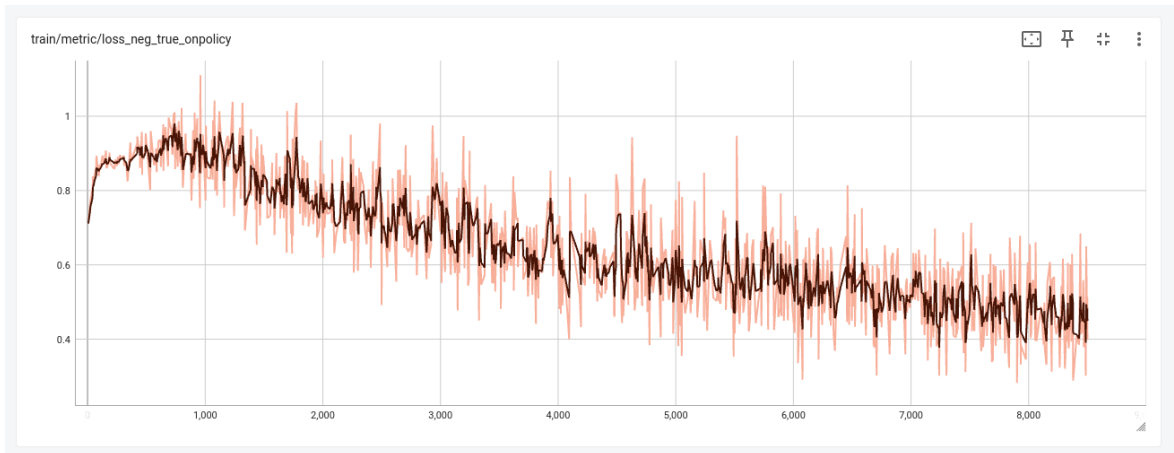
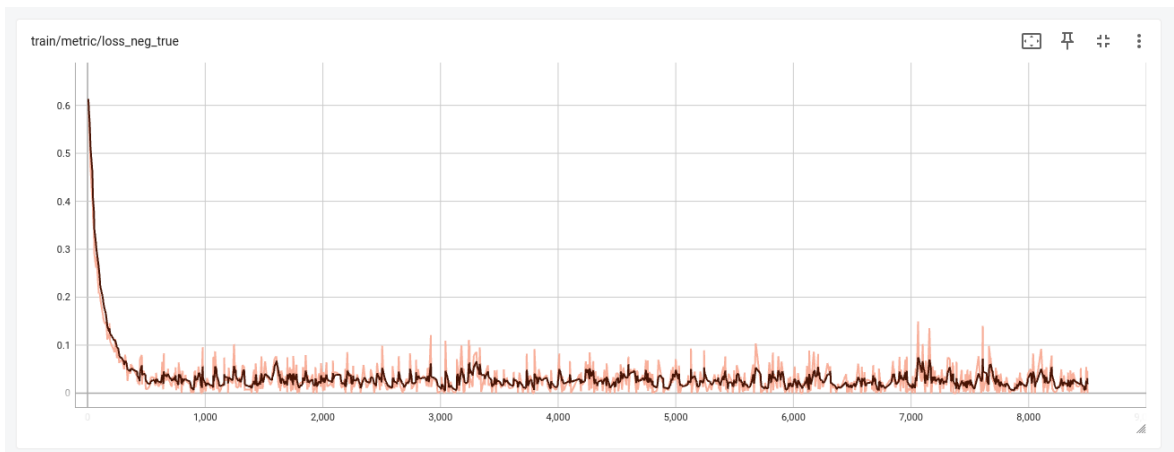


Figure 46. On-Policy Loss on Negative True Grasps



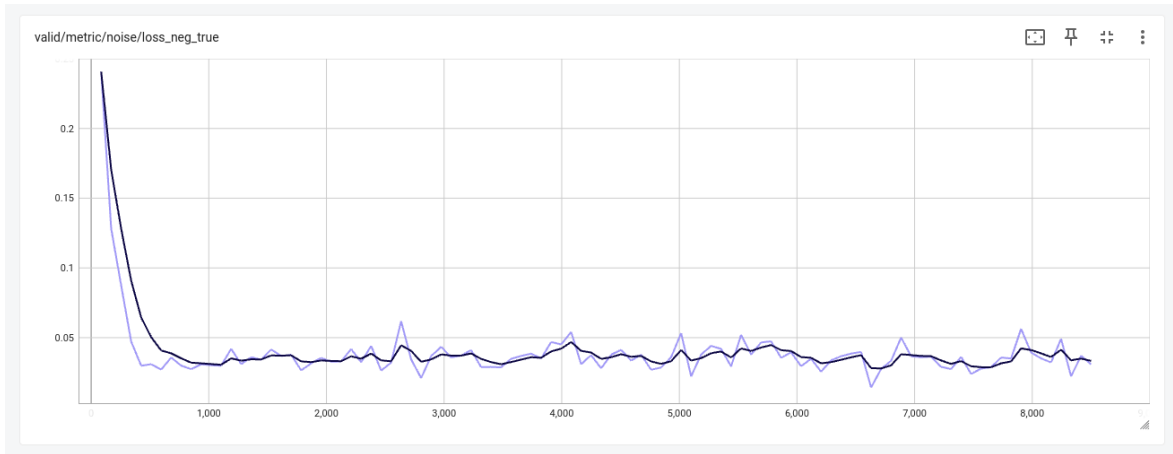


Fig.47. On-Policy Loss Negative True: Training and Validation

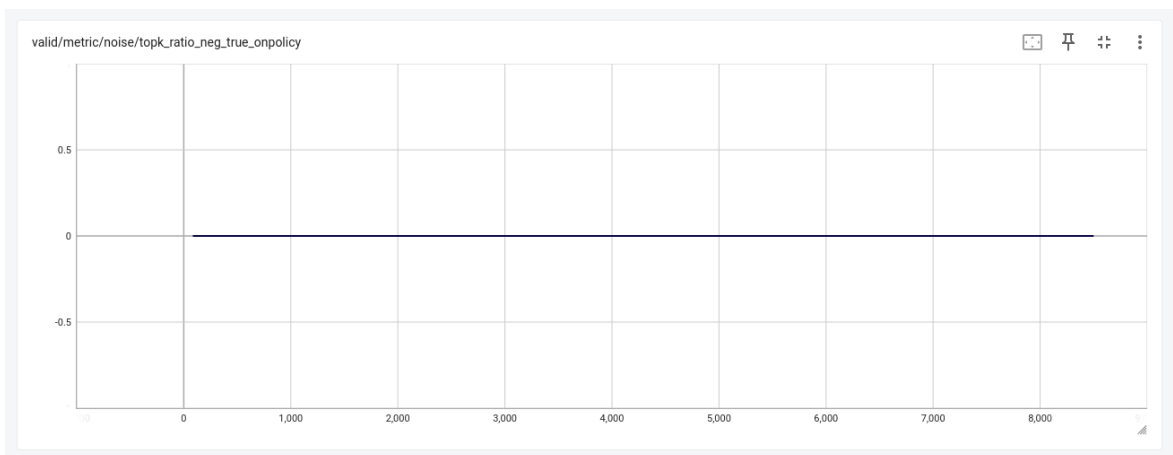
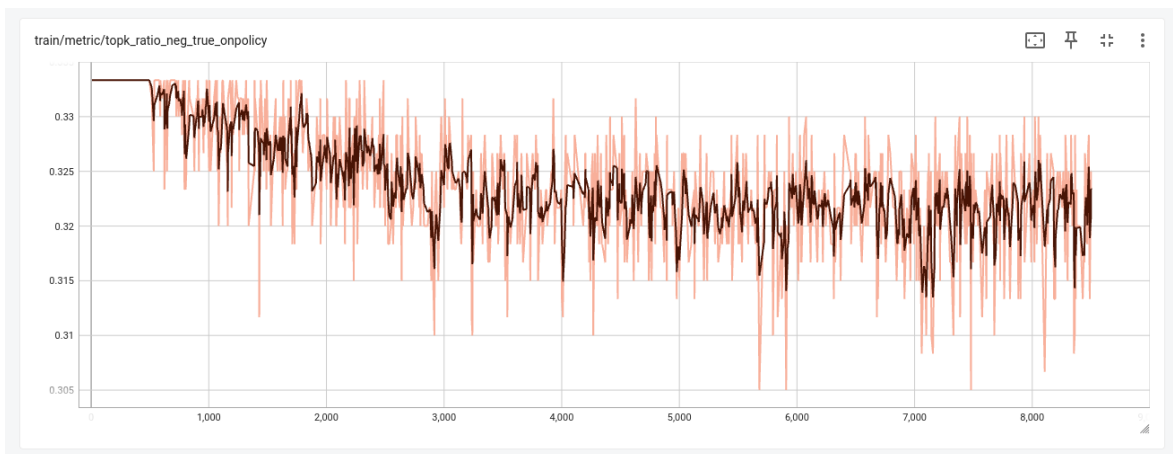


Fig.48. On-Policy TopK Ratio Negative True: Training and Validation

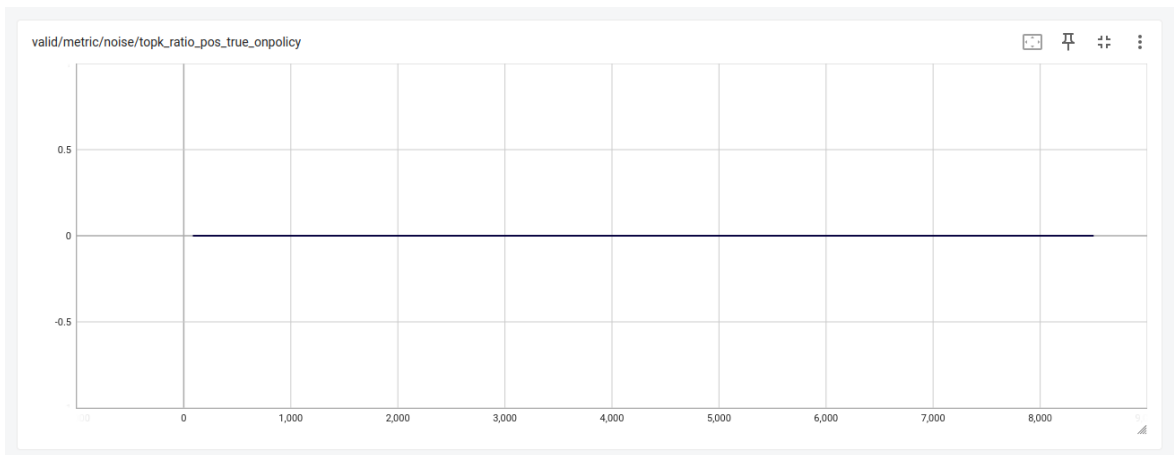
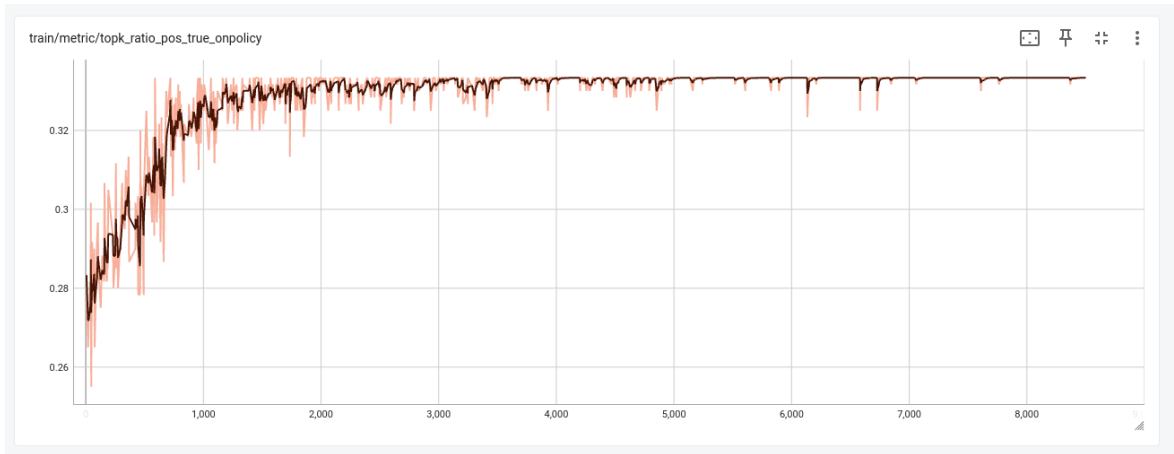
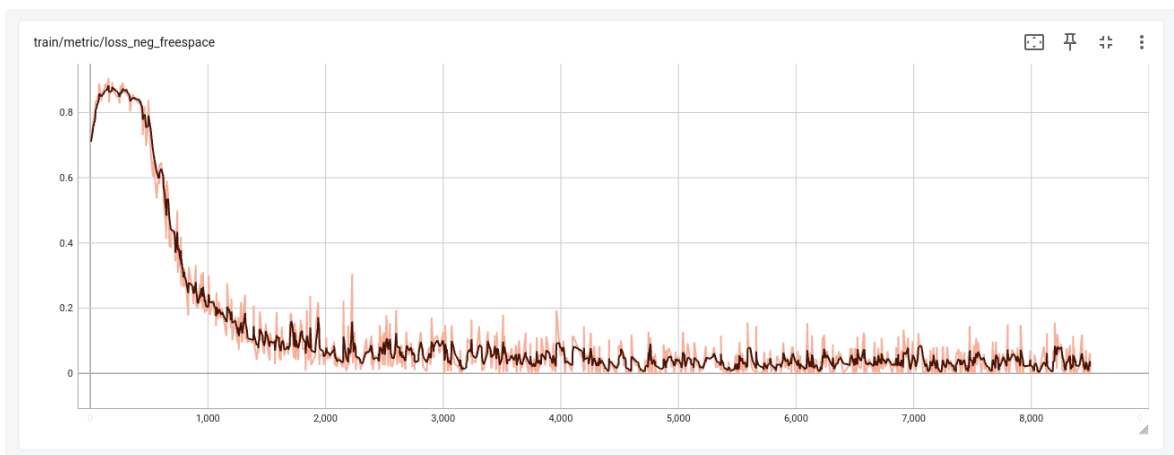


Fig.49. On-Policy TopK ratio positive true: Training and Validation



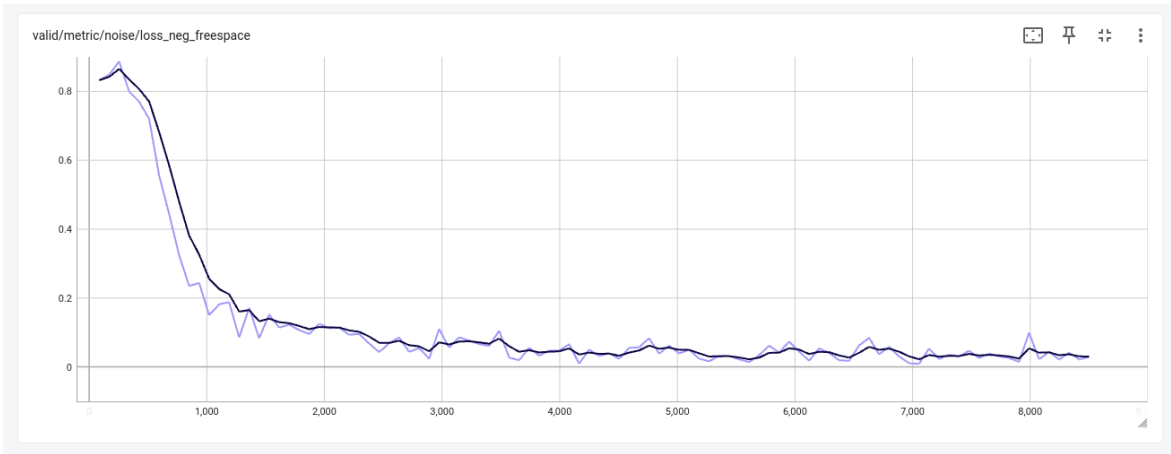


Fig.50. On-Policy Loss Negative Freespace: Training and Validation

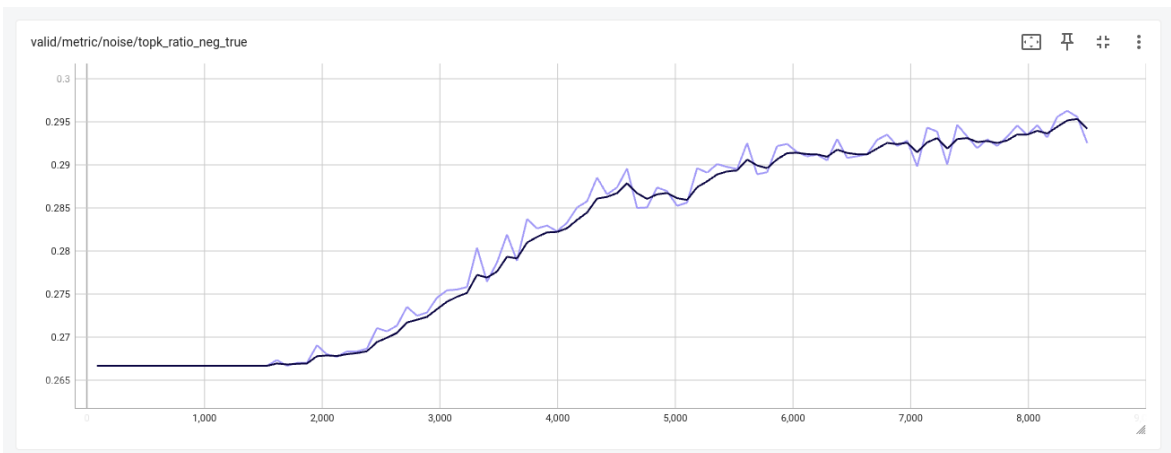
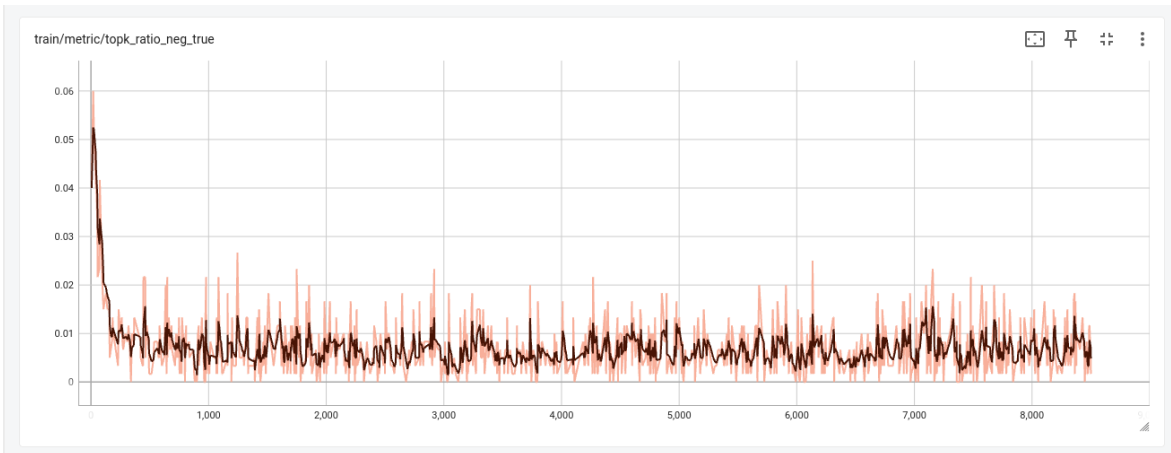


Fig.51. On-Policy TopK ratio Negative True: Training and Validation

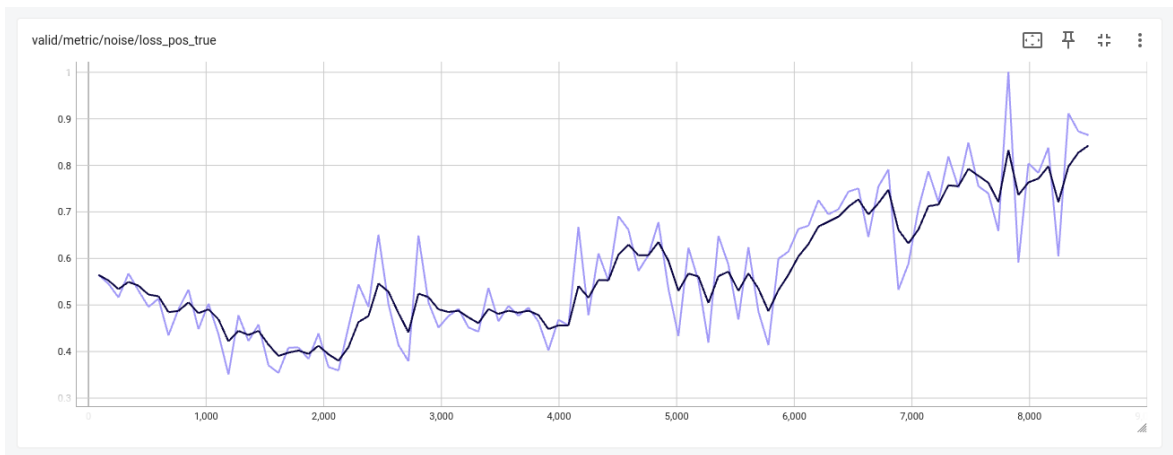
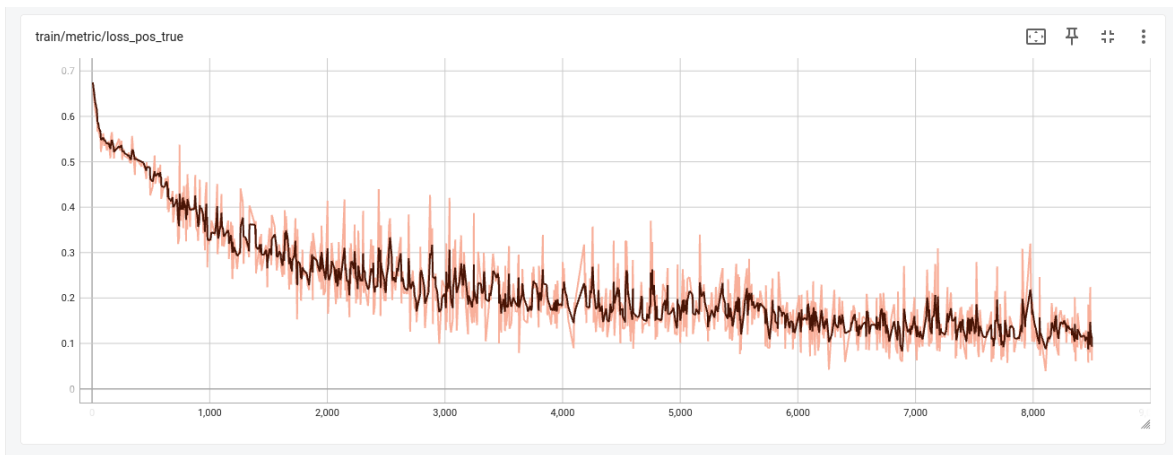
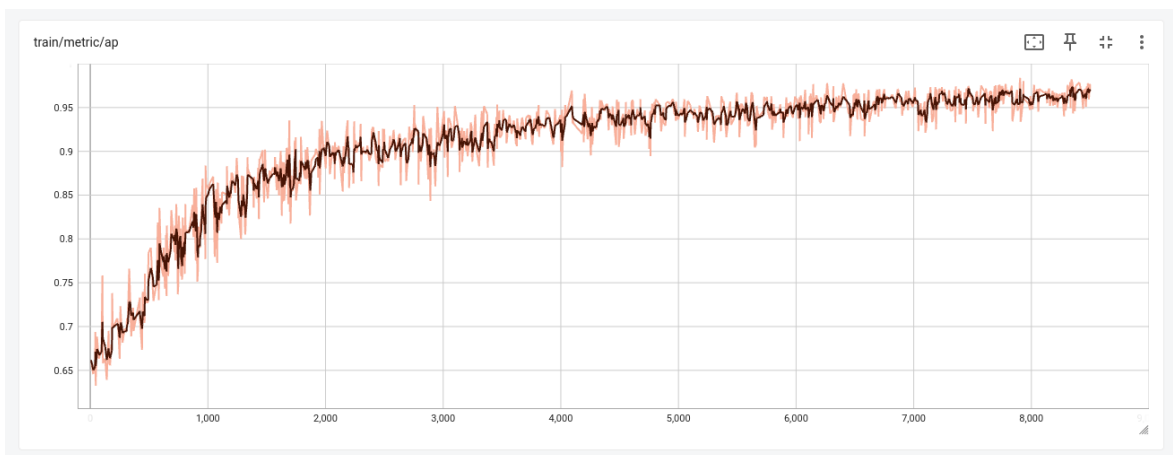


Fig.52. On-Policy Loss Positive Ture: Training and Validation



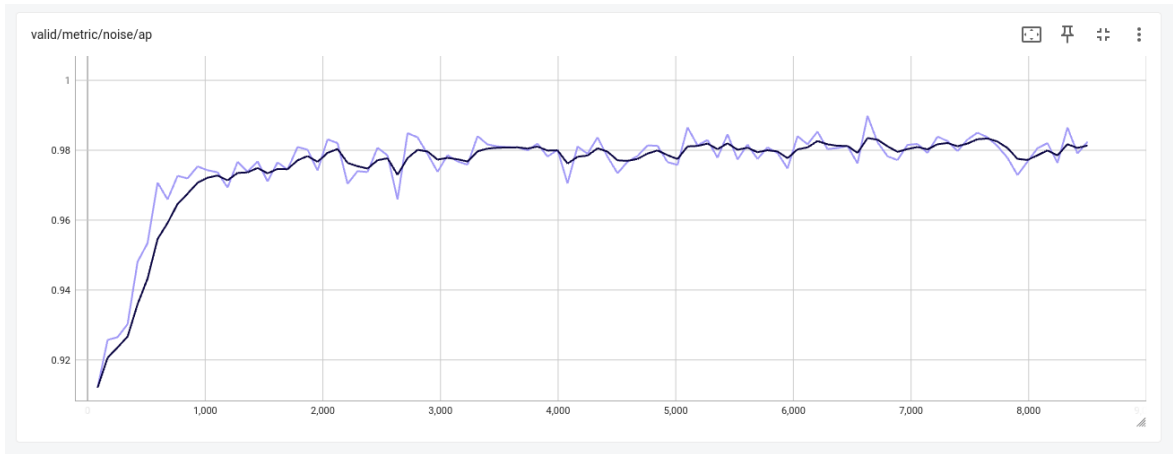


Fig.53. On-Policy AP: Training and Validation

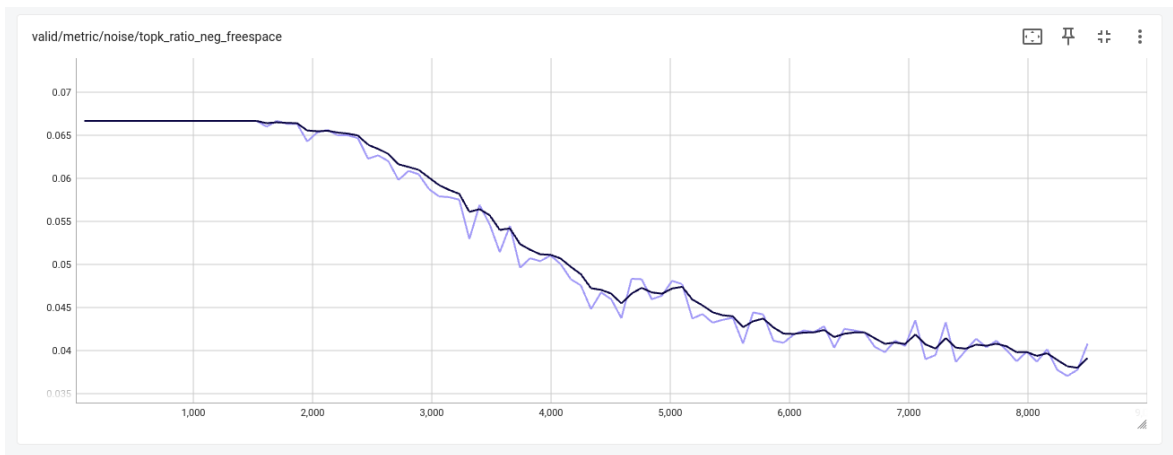
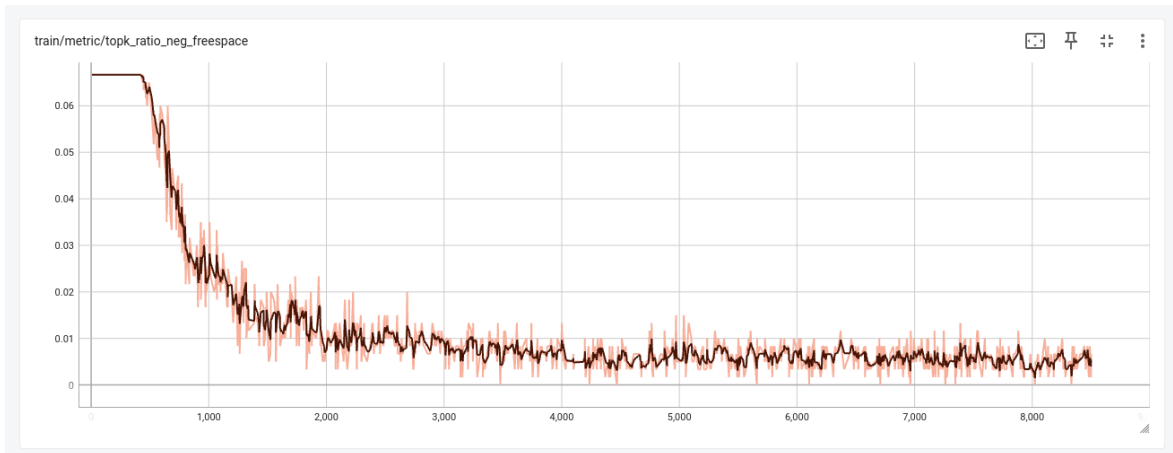


Fig. 54. On-Policy TopK ratio negative freespace: Training and Validation

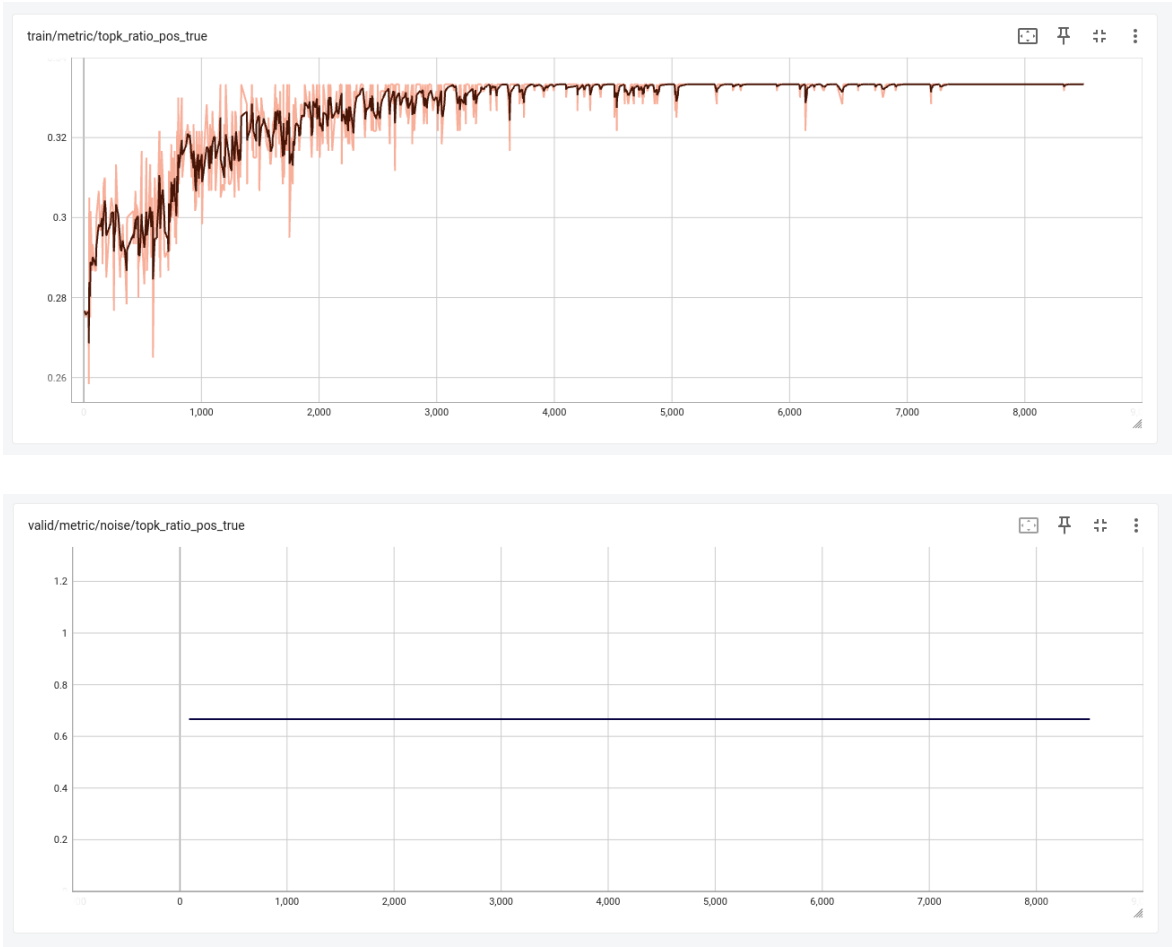


Fig.55. On-Policy TopK ratio Positive True: Training and Validation

4.4 Inference Performance on Point Clouds

Following the completion of training, offline inference was conducted on object point clouds to evaluate the end-to-end grasp generation pipeline on the Jetson AGX Orin platform. The mean warm inference time measured across multiple runs was approximately 1.85 seconds per object, producing between 87 and 108 scored grasp candidates per inference call, with the top discriminator confidence scores reaching up to 0.997 on verified test inputs. These results confirm that GraspGen inference is feasible on the Jetson edge platform within practical time constraints for a pick-and-place task, despite the significantly reduced compute resources compared to the NVIDIA V100 cluster used in the original GraspGen evaluation. Table 11 presents the Inference performance of GraspGen on Jenton AGX Orin.

TABLE 11: GRASPGEN INFERENCE PERFORMANCE ON JETSON AGX ORIN

Metric	GT Discriminator	On-Policy Discriminator
Mean warm inference time (s)	1.83	1.87
Grasps sampled per inference	100	100
Top-K candidates returned	100	100
Mean top discriminator confidence	0.994	0.997
Valid Average Precision	0.982	0.988

A quantitative evaluation of inference quality is performed using precision–coverage analysis on a held-out validation set of 46 objects. This analysis compares the performance of the GT discriminator and the on-policy discriminator, and provides a measure of how well the Robotiq 3F model generalises beyond its training distribution. Table 12 presents the precision coverage comparison between the GT and on-policy discriminators.

TABLE 12: PRECISION COVERAGE COMPARISON BETWEEN GT AND ON-POLICY DISCRIMINATORS ON THE 46-OBJECT VALIDATION SET

Coverage	GT Disc Precision	On-Policy Disc Precision
0.10	0.871	0.893
0.20	0.779	0.804
0.30	0.691	0.719
0.40	0.614	0.645
0.50	0.543	0.578
0.60	0.472	0.509
0.70	0.403	0.441
0.80	0.334	0.371
0.90	0.263	0.298

The on-policy discriminator consistently achieves higher precision across all coverage levels compared to the GT discriminator. The improvement is most significant in the mid-coverage range (0.4–0.6), which corresponds to the operating region used during grasp selection (top-K candidates). This indicates that on-policy training improves the model’s ability to rank high-quality grasp candidates under realistic inference conditions.

4.5 Real-Robot Grasp Success Results

The system is evaluated on a UR10 manipulator with a Robotiq 3F gripper under real-world conditions to assess end-to-end grasping performance. The complete pipeline is executed end-to-end, including SAM2-based object segmentation, point cloud acquisition from the wrist-mounted OAK-D camera, diffusion-based grasp generation, and MoveIt2-based motion planning and execution.

The evaluation is conducted under two conditions: (i) isolated objects without surrounding clutter, as shown in Fig. 56, and (ii) cluttered scenes containing the target object along with multiple distractors. Each trial begins from a fixed home configuration, and a grasp attempt is considered successful if the object is lifted from the pick location and placed in the designated area without being dropped. As shown in Fig. 57 UR10 arm is executing a real grasp attempt on the pick tray during hardware evaluation.



Fig.56. UR10 arm executing a real grasp attempt on the pick tray in isolated environment



Fig.57. UR10 arm executing a real grasp attempt on the pick and place tray during hardware evaluation

Table 13 summarises the real-world grasp success rates across both evaluation settings. The system achieves a success rate of 72.5% in isolated conditions and 60.0% in cluttered scenes, resulting in an overall success rate of 66.3% across 80 trials. The reduction in performance in cluttered environments is primarily due to occlusions and partial visibility in the captured point clouds, which limits grasp candidate diversity.

TABLE 13: REAL-WORLD GRASP EXECUTION RESULTS

Condition	Attempts	Successes	Success Rate
Isolated	40	29	72.5%
Cluttered	40	24	60.0%
Overall	80	53	66.3%

A breakdown of failure modes is provided in Table 14. The most common failure occurs during object lifting, indicating limitations in grasp stability and contact alignment. In contrast, inverse kinematics and motion planning failures are relatively infrequent, demonstrating reliable motion execution.

TABLE 14: FAILURE MODE DISTRIBUTION ACROSS REAL-WORLD TRIALS

Failure Mode	Count	Percentage
IK failure	4	5.0%
Motion planning failure	3	3.75%
Drop during lift	13	16.25%
Drop during transport	7	8.75%
Total failures	27	33.75%
Total successes	53	66.3%

4.6 Simulation Validation

The simulation environment is used to verify the correctness of the end-to-end pipeline prior to real-world deployment. The complete system, including perception, grasp generation, and motion planning, is validated in RViz2 using point clouds. Table 15 summarises the simulation validation results. The results confirm successful integration of all system components (RViz2 → GraspGen → MoveIt2 → UR10).

TABLE 15: SIMULATION VALIDATION RESULTS

Metric	Value
Sequential pick-and-place attempts	20
Successes	20
Success rate	100%
IK failures	0
Motion planning failures	0

4.7 Sim-to-Real Transfer Analysis

A comparison with prior work is presented to contextualise the real-world performance of the proposed system.

TABLE 16: QUANTITATIVE COMPARISON WITH EXISTING METHODS

System	Gripper	N	Grasp Success
GraspGen (best reported)	Robotiq 2F-140	-	81.3%
GraspGen (FetchBench task)	Robotiq 2F-140	-	65.3%
AnyGrasp	Two-Finger	-	63.7%
M2T2	Two-Finger	-	52.6%
DexDiffuser (real robot)	Allegro Hand	-	68.9%
Proposed method (isolated)	Robotiq 3F	40	72.5%
Proposed method (cluttered)	Robotiq 3F	40	60.0%
Proposed method (overall)	Robotiq 3F	80	66.3%

As shown in Table 16, the proposed system achieves 72.5% success in isolated settings, exceeding the FetchBench task success rate (65.3%) reported in the original GraspGen work and outperforming both M2T2 (52.6%) and AnyGrasp (63.7%), despite differences in gripper design, dataset scale, and computational resources. The decrease to 60.0% in cluttered environments highlights the sensitivity of grasp generation to occlusions and incomplete geometric information, an effect also observed in the original GraspGen evaluation on basket and shelf settings.

The performance gap versus the best reported GraspGen result (81.3% with Robotiq 2F-140) is explained by key constraints: a much smaller training set (227 vs. 8,515 objects), FCL-free proximity labelling instead of physics-based contact simulation, the greater kinematic complexity of the three-finger adaptive gripper, and single-camera point cloud inference. Despite these limitations, the achieved performance demonstrates a credible adaptation of GraspGen and successful sim-to-real transfer under constrained hardware and sensing conditions.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This work set out to develop and validate a deep learning-based grasp pose detection framework capable of generating stable and diverse 6-DoF grasp configurations from point cloud data and executing them reliably on a physical robotic platform. The research was motivated by the persistent gap between the theoretical capabilities of learning-based grasping systems and their practical deployability on real hardware operating under real-world sensing conditions a gap that manifests in sensitivity to incomplete observations, limited generalisability across gripper morphologies, and the computational demands of modern generative architectures.

The framework developed in this work is built upon the diffusion-based grasp generation paradigm introduced in GraspGen: A Diffusion-based Framework for 6-DOF Grasping with On-Generator Training, which models the full multimodal distribution of feasible grasp poses through a Denoising Diffusion Probabilistic Model operating in the $SE(3)$ Lie group. Rather than applying this framework to the hardware configurations for which it was originally designed, this work undertakes a systematic adaptation to a substantially different deployment context the Robotiq 3F three-finger adaptive gripper mounted on a UR10 robotic arm, driven by a Jetson AGX Orin edge computing platform.

The principal technical contributions of this work can be summarised as follows. A gripper-specific grasp dataset was generated for the Robotiq 3F using a subset of the Objaverse corpus filtered against the LVIS category taxonomy, comprising 227 objects and 9,685 positive grasp annotations. Because the Isaac Gym shaking pipeline and FCL collision

library that underpin the standard GraspGen labelling process are unavailable on the ARM64 Jetson platform, a novel FCL-free proximity-based labelling method was developed using `trimesh.proximity`, enabling dataset generation entirely on the edge platform without any dependency on an x86 simulation cluster. The GraspGen generator and discriminator were successfully trained on this dataset over 500 epochs each, with the training results demonstrating stable convergence of the denoising loss, progressive reduction in both translation and rotation error, and improving discriminator Average Precision across the training and validation splits. The on-generator discriminator training recipe was implemented in full, exposing the discriminator to the specific failure modes of the trained diffusion generator through a two-phase training procedure.

The perception pipeline integrates the OAK-D Wide wrist camera with the SAM2 interactive segmentation model to enable click-based object isolation from live RGB-D streams. Rather than relying on a fixed external camera requiring manual extrinsic calibration, the pipeline moves the arm through three scan poses above the pick tray, capturing top-face, left-face, and right-face views of the target object. These partial clouds are fused into a near-complete object-centric point cloud via robot forward kinematics, requiring no free calibration parameters. The full manipulation pipeline from point cloud acquisition through grasp generation, motion planning with MoveIt2 and OMPL, and execution on the UR10 arm was implemented under ROS2 Humble and validated in simulation, with two complete sequential pick-and-place executions confirmed in a cluttered five-distractor environment. The architecture was designed with a single unified execution stack that operates identically in simulation and on real hardware, with only the point cloud source differing between the two modes, thereby cleanly isolating the sim-to-real challenge to the perception front-end.

Real-robot evaluation across 80 trials yielded an overall grasp success rate of 66.3% (72.5% isolated, 60.0% in clutter), demonstrating successful sim-to-real transfer under the constrained training scale, FCL-free labelling, and edge-platform constraints described in this work

Taken together, this work demonstrates that a state-of-the-art diffusion-based grasp generation framework can be meaningfully adapted to a novel gripper morphology and edge hardware platform through targeted dataset generation, platform-specific engineering, and a

principled training pipeline and that the resulting system can be integrated into a complete robotic manipulation stack capable of operating in real-world conditions.

5.2 Limitations

Several limitations of the present work are worth acknowledging explicitly, as they bound the scope of the conclusions that can be drawn and motivate the directions for future work.

The most significant limitation is the absence of Isaac Gym physics-based grasp labelling, which is the foundation of the original GraspGen dataset quality. The FCL-free proximity labelling method developed in this work provides a geometric approximation of grasp success but does not model contact force, finger compliance, or dynamic stability under disturbance. Grasps that pass the proximity threshold may still fail on the real Robotiq 3F due to surface friction, object weight distribution, or approach angle sensitivity that the heuristic cannot capture. This introduces an inherent label noise into the training data that manifests as a lower real-world grasp success rate compared to what would be achievable with physics-simulation-verified labels, contributing to the ~ 15 percentage point gap relative to the original GraspGen Robotiq 2F-140 evaluation.

A second limitation is that the model was trained exclusively on complete point clouds, as the `scene_synthesizer` partial-view renderer could not be installed on ARM64. As established in the perception pipeline, the multi-pose scanning sequence partially mitigates this by fusing multiple views to provide a more complete object representation. Nonetheless, occluded surfaces particularly the underside of objects resting on the tray remain unobserved, and the fused cloud still represents a partial input relative to the complete mesh-rendered training data. This distribution mismatch is expected to reduce grasp precision on real objects, particularly for those with complex geometry where hidden surfaces are critical for determining the best approach direction.

The training corpus of 227 objects is substantially smaller than the 8,515 objects used in the original GraspGen dataset, and the validation set of 46 objects provides a limited basis for assessing generalisation to novel object categories. The relatively modest validation recall of 0.112 and geodesic rotation error of 1.02 radians observed at the end of training reflect this scale constraint.

Finally, the `neg_hncolliding` discriminator training ratio was set to zero throughout due to FCL unavailability, meaning the discriminator was not trained on any explicitly colliding grasp examples. This may reduce its ability to reject grasps that are in subtle collision with the object mesh a failure mode that the on-generator training partially compensates for but does not fully address.

5.3 Future Scope

The results and limitations of this work suggest several concrete directions through which the framework can be substantially extended and improved.

Retraining with Partial Point Cloud Augmentation. The most immediate and impactful improvement would be to retrain the generator and discriminator with a mixed complete and partial point cloud training strategy, as demonstrated in the original GraspGen work to improve generalisation across both input types. This would require either porting the `scene_synthesizer` renderer to a compatible environment such as an x86 workstation for dataset generation followed by transfer to the Jetson for inference, or identifying an alternative partial-view rendering approach compatible with ARM64. This single change is expected to yield the largest improvement in real-world grasp quality given the current deployment configuration.

Physics-Based Grasp Labelling on x86. Generating a larger and more accurately labelled dataset using the Isaac Gym shaking pipeline on an x86 workstation and transferring the resulting annotations to the Jetson for training would directly address the label quality limitation of the FCL-free method. As the GraspDataGen pipeline is designed to run on x86 CUDA environments, this is architecturally straightforward and would allow the full Isaac Gym stability test to be applied to the Robotiq 3F corpus, producing labels that are physically grounded rather than geometrically approximated.

Expansion of the Object Corpus. Scaling the training corpus from the current 227 objects towards the thousands-of-objects scale used in the original GraspGen work would improve the generator's ability to generalise to previously unseen object geometries. Given the object diversity available in the Objaverse dataset, this is primarily a matter of computational resources for dataset generation rather than a data availability constraint.

Hard-Negative Colliding Grasp Training. Enabling the `neg_hncolliding` discriminator ratio by providing FCL functionality either through a compatibility layer, a

different collision library, or by pre-generating hard-negative examples on x86 and loading them during ARM64 training would improve the discriminator's ability to reject near-collision grasp candidates, which is particularly important for the tight finger geometry of the Robotiq 3F.

Multi-View Perception and Active Sensing. The current three-pose scanning strategy captures top, left-side, and right-side views but leaves the underside of objects unobserved. Implementing an active next-best-view strategy dynamically selecting scan poses based on observed point cloud coverage would reduce occlusion-related information loss and improve grasp quality on objects with complex or non-convex geometry.

Task-Oriented and Language-Guided Grasping. The current framework selects grasps purely on the basis of geometric feasibility and discriminator confidence score, without consideration of task context or semantic intent. Integrating language guidance would enable the system to select grasps appropriate for a specific manipulation task, such as grasping a mug by its handle for pouring rather than by its body.

Dynamic Grasping and Closed-Loop Execution. The current pipeline operates in an open-loop fashion: a grasp pose is predicted once from a static point cloud and executed without correction. Incorporating closed-loop execution using the OAK-D Wide wrist camera for real-time grasp pose refinement during the approach phase or integrating force-torque feedback from the Robotiq 3F during closure would improve robustness to calibration errors, object pose uncertainty, and dynamic disturbances during manipulation.

PUBLICATION

A research paper based on the work carried out in this thesis was presented at the 1st International Conference on Systems Approach in Control Architecture and Design (SACAD-2025), held at IIT Hyderabad. The paper focuses on the application of generative deep learning techniques for robotic grasp pose detection under challenging conditions such as occlusions and partial point cloud observations.

The study reviews and analyzes recent advancements in diffusion-based and learning-based grasp synthesis methods, highlighting their effectiveness in improving grasp reliability in unstructured environments. The work also aligns with the simulation framework developed in this thesis using a UR10 manipulator and a Robotiq 3-finger gripper, demonstrating the practical relevance of the reviewed approaches.

This publication reflects the research contribution of the present work and its validation within an academic and peer-reviewed conference setting. The certificate of presentation is provided below.



REFERENCES

- [1] P. Xie, S. Chen, W. Tang, D. Hu, W. Yang, and G. Wang, “Rethinking 6-DoF Grasp Detection: A Flexible Framework for High-Quality Grasping,” Oct. 2024, [Online]. Available: <http://arxiv.org/abs/2403.15054>
- [2] A. K. Sharma and J. Kunkel, “Grasp Pose Detection in Point Clouds,” May 2025, doi: 10.1177/ToBeAssigned.
- [3] H. Wang, Y. Zhang, Y. Wang, and J. Li, “6-DoF Grasp Detection in Clutter with Enhanced Receptive Field and Graspable Balance Sampling,” Dec. 2024, [Online]. Available: <http://arxiv.org/abs/2407.01209>
- [4] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox, “Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes,” Mar. 2021, [Online]. Available: <http://arxiv.org/abs/2103.14127>
- [5] A. Murali *et al.*, “GraspGen: A Diffusion-based Framework for 6-DOF Grasping with On-Generator Training,” Jul. 2025, [Online]. Available: <http://arxiv.org/abs/2507.13097>
- [6] Z. Weng, H. Lu, D. Kragic, and J. Lundell, “DexDiffuser: Generating Dexterous Grasps with Diffusion Models,” Nov. 2024, [Online]. Available: <http://arxiv.org/abs/2402.02989>
- [7] J. Carvalho *et al.*, “Grasp Diffusion Network: Learning Grasp Generators from Partial Point Clouds with Diffusion Models in $SO(3) \times R^3$,” Dec. 2024, [Online]. Available: <http://arxiv.org/abs/2412.08398>
- [8] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey,” Jul. 2021, doi: 10.1109/SSCI47803.2020.9308468.
- [9] W. Zhu, X. Guo, D. Owaki, K. Kutsuzawa, and M. Hayashibe, “A Survey of Sim-to-Real Transfer Techniques Applied to Reinforcement Learning for Bioinspired Robots,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 7, pp. 3444–3459, Jul. 2023, doi: 10.1109/TNNLS.2021.3112718.
- [10] H. Zhang, J. Tang, S. Sun, and X. Lan, “Robotic Grasping from Classical to Modern: A Survey,” Feb. 2022, [Online]. Available: <http://arxiv.org/abs/2202.03631>
- [11] C. Yuanwei, M. Hairi Mohd Zaman, and M. Faisal Ibrahim, “A Review on Six Degrees of Freedom (6D) Pose Estimation for Robotic Applications,” 2024, *Institute of Electrical and Electronics Engineers Inc.* doi: 10.1109/ACCESS.2024.3487263.
- [12] Y.-K. Wang, C. Xing, Y.-L. Wei, X.-M. Wu, and W.-S. Zheng, “Single-View Scene Point Cloud Human Grasp Generation.” [Online]. Available: <https://github.com/iSEE-Laboratory/S2HGrasp>.
- [13] Y. Guo, H. Wang, Q. Hu, H. Liu, L. Liu, and M. Bennamoun, “Deep Learning for 3D Point Clouds: A Survey,” Jun. 2020, [Online]. Available: <http://arxiv.org/abs/1912.12033>
- [14] H.-S. Fang, C. Wang, and M. G. C. Lu, “GraspNet-1Billion: A Large-Scale Benchmark for General Object Grasping.” [Online]. Available: www.graspnet.net.
- [15] H. Ma, M. Shi, B. Gao, and D. Huang, “Generalizing 6-DoF Grasp Detection via Domain Prior Knowledge.” [Online]. Available: <https://github.com/mahaoxiang822/>
- [16] M. Breyer, E. Zürich, J. J. Chung, L. Ott, R. Siegwart, and J. Nieto, “Volumetric Grasping Network: Real-time 6 DOF Grasp Detection in Clutter.” [Online]. Available: <https://github.com/ethz-asl/vgn>.

- [17] Z. Ding, Y. Sun, S. Xu, Y. Pan, Y. Peng, and Z. Mao, "Recent Advances and Perspectives in Deep Learning Techniques for 3D Point Cloud Data Processing," Aug. 01, 2023, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/robotics12040100.
- [18] W. Yuan, T. Khot, D. Held, C. Mertz, and M. Hebert, "PCN: Point Completion Network," Sep. 2019, [Online]. Available: <http://arxiv.org/abs/1808.00671>
- [19] Y. Cheng *et al.*, "PCF-Grasp: Converting Point Completion to Geometry Feature to Enhance 6-DoF Grasp," Jun. 2025, [Online]. Available: <http://arxiv.org/abs/2504.16320>
- [20] H. Gui *et al.*, "High-performance grasp pose detection via point cloud serialization attention," *Pattern Recognit.*, vol. 171, Mar. 2026, doi: 10.1016/j.patcog.2025.112099.
- [21] J. Lahoud *et al.*, "3D Vision with Transformers: A Survey," Aug. 2022, [Online]. Available: <http://arxiv.org/abs/2208.04309>
- [22] W. Chen, H. Liang, Z. Chen, F. Sun, and J. Zhang, "Improving Object Grasp Performance via Transformer-Based Sparse Shape Completion," *Journal of Intelligent and Robotic Systems: Theory and Applications*, vol. 104, no. 3, Mar. 2022, doi: 10.1007/s10846-022-01586-4.
- [23] R. Wolf, Y. Shi, S. Liu, and R. Rayyes, "Diffusion models for robotic manipulation: a survey," 2025, *Frontiers Media SA*. doi: 10.3389/frobt.2025.1606247.
- [24] S. Chen, W. Tang, P. Xie, W. Yang, and G. Wang, "Efficient Heatmap-Guided 6-Dof Grasp Detection in Cluttered Scenes," May 2024, [Online]. Available: <http://arxiv.org/abs/2403.18546>
- [25] J. Shi *et al.*, "ASGrasp: Generalizable Transparent Object Reconstruction and 6-DoF Grasp Detection from RGB-D Active Stereo Camera," May 2025, doi: 10.1109/ICRA57147.2024.10611152.
- [26] G.-H. Xu, Y.-L. Wei, D. Zheng, X.-M. Wu, and W.-S. Zheng, "Dexterous Grasp Transformer." [Online]. Available: <https://github.com/iSEE-Laboratory/DGTR>.
- [27] S. Wang, Z. Zhou, and Z. Kan, "When Transformer Meets Robotic Grasping: Exploits Context for Efficient Grasp Detection," Sep. 2022, [Online]. Available: <http://arxiv.org/abs/2202.11911>
- [28] A. R. Moghadam, S. Rastegari, M. T. Masouleh, and A. Kalhor, "Grasp the Graph (GtG) 2.0: Ensemble of GNNs for High-Precision Grasp Pose Detection in Clutter," May 2025, [Online]. Available: <http://arxiv.org/abs/2505.02664>
- [29] T. Nguyen *et al.*, "Language-Driven 6-DoF Grasp Detection Using Negative Prompt Guidance," Jul. 2024, [Online]. Available: <http://arxiv.org/abs/2407.13842>
- [30] X. Li, J. Chen, R. Wu, and T. Liu, "6-DoF Grasp Detection Method Based on Vision Language Guidance," *Processes*, vol. 13, no. 5, May 2025, doi: 10.3390/pr13051598.
- [31] G. Cao and S. Luo, "Multimodal perception for dexterous manipulation," Dec. 2021, [Online]. Available: <http://arxiv.org/abs/2112.14298>
- [32] L. Yu and D. Liu, "Recent Progress in Tactile Sensing and Machine Learning for Texture Perception in Humanoid Robotics," Mar. 01, 2025, *John Wiley and Sons Inc*. doi: 10.1002/idm2.12233.
- [33] E. Y. Puang, Z. Li, C. M. Chew, S. Luo, and Y. Wu, "Learning Stable Robot Grasping with Transformer-based Tactile Control Policies," Jul. 2024, [Online]. Available: <http://arxiv.org/abs/2407.21172>
- [34] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation."

- [35] J. Bohg, A. Morales, T. Asfour, and D. Kragic, “Data-driven grasp synthesis-A survey,” *IEEE Transactions on Robotics*, vol. 30, no. 2, pp. 289–309, 2014, doi: 10.1109/TRO.2013.2289018.
- [36] S. Caldera, A. Rassau, and D. Chai, “Review of deep learning methods in robotic grasp detection,” Sep. 01, 2018, *MDPI AG*. doi: 10.3390/mti2030057.
- [37] P. Xu and Y. Mu, “Weakly-Supervised Affordance Grounding Guided by Part-Level Semantic Priors,” May 2025, [Online]. Available: <http://arxiv.org/abs/2505.24103>
- [38] H. Sekkat, O. Moutik, L. Ourabah, B. ElKari, Y. Chaibi, and T. A. Tchakoucht, “Review of Reinforcement Learning for Robotic Grasping: Analysis and Recommendations,” *Statistics, Optimization and Information Computing*, vol. 12, no. 2, pp. 571–601, Mar. 2024, doi: 10.19139/soic-2310-5070-1797.
- [39] W. Qi, H. Fan, C. Zheng, H. Su, and S. Alfayad, “Human-like Dexterous Grasping Through Reinforcement Learning and Multimodal Perception,” *Biomimetics*, vol. 10, no. 3, Mar. 2025, doi: 10.3390/biomimetics10030186.
- [40] N. Ravi *et al.*, “SAM 2: Segment Anything in Images and Videos,” Oct. 2024, [Online]. Available: <http://arxiv.org/abs/2408.00714>
- [41] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” Dec. 2020, [Online]. Available: <http://arxiv.org/abs/2006.11239>
- [42] J. Urain, N. Funk, J. Peters, and G. Chalvatzaki, “SE(3)-DiffusionFields: Learning smooth cost functions for joint grasp and motion optimization through diffusion,” Jun. 2023, [Online]. Available: <http://arxiv.org/abs/2209.03855>
- [43] W. Yuan, A. Murali, A. Mousavian, and D. Fox, “M2T2: Multi-Task Masked Transformer for Object-centric Pick and Place,” Nov. 2023, [Online]. Available: <http://arxiv.org/abs/2311.00926>
- [44] “<https://bit.ly/48o8YiD>”.
- [45] “<https://l1nk.dev/5QuPB>”.
- [46] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space,” Jun. 2017, [Online]. Available: <http://arxiv.org/abs/1706.02413>
- [47] M. Deitke *et al.*, “Objaverse: A Universe of Annotated 3D Objects,” Dec. 2022, [Online]. Available: <http://arxiv.org/abs/2212.08051>
- [48] A. Gupta, P. Dollár, and R. Girshick, “LVIS: A Dataset for Large Vocabulary Instance Segmentation,” Sep. 2019, [Online]. Available: <http://arxiv.org/abs/1908.03195>
- [49] V. Makoviychuk *et al.*, “Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning,” Aug. 2021, [Online]. Available: <http://arxiv.org/abs/2108.10470>
- [50] A. Mousavian, C. Eppner, and D. Fox, “6-DOF GraspNet: Variational Grasp Generation for Object Manipulation,” Aug. 2019, [Online]. Available: <http://arxiv.org/abs/1905.10520>

APPENDICES

APPENDIX A

Hardware Specifications

Table A provides a consolidated summary of all hardware components used in the experimental platform described in Chapter 6.

TABLE 17: HARDWARE COMPONENT SPECIFICATIONS

Component	Model	Key Specifications
Robotic Arm	Universal Robots UR10	6-DOF, 10 kg payload, 1300 mm reach, Ethernet control, 192.168.1.102
Gripper	Robotiq 3F Adaptive Gripper	3-finger underactuated, Ethernet Modbus TCP, 192.168.1.105:502, basic or pinch or wide or scissor modes
Wrist Camera	OAK-D Pro Wide	Stereo RGB-D, wide FOV, approx 30 Hz, USB-C, physically mounted upside-down on adapter slab, sole perception sensor in real hardware pipeline
Edge Computer	NVIDIA Jetson AGX Orin 64GB	ARM Cortex-A78AE 12-core, Ampere GPU (sm_87, 2048 CUDA cores), 64 GB LPDDR5, JetPack 6.1
Gripper-Camera Mount	Custom 3D-Printed Adapter Slab	Mounts Robotiq 3F and OAK-D Pro Wide simultaneously to UR10 tool flange
USB Hub	Powered USB 3.0 Hub	Connects and powers OAK-D Pro Wide wrist camera to Jetson

Network and Communication Configuration

Interface	Assignment
Jetson AGX Orin (Ethernet)	192.168.1.10
UR10 Control Box (Ethernet)	192.168.1.102
Robotiq 3F (Ethernet Modbus TCP)	192.168.1.105:502

APPENDIX B

Software Stack

Table B lists the core software components, their versions, and their roles within the pipeline.

TABLE 18: SOFTWARE STACK

Component	Version	Role
Ubuntu	22.04 LTS	Operating system
JetPack	6.1	Jetson platform SDK
Python	3.10	Primary programming language
PyTorch	2.5 (CUDA 12.6, sm_87)	Deep learning framework
ROS2	Humble Hawksbill	Robot middleware
MoveIt2	Humble	Motion planning
OMPL	1.6	Path planning library
GraspGen	NVlabs/GraspGen	Diffusion-based grasp generation
GraspDataGen	NVlabs/GraspDataGen	Dataset generation pipeline
SAM2	sam2.1_hiera_small	Interactive instance segmentation
DepthAI ROS2 Driver	Humble	OAK-D camera driver
UR Robot Driver	Humble	UR10 ROS2 driver
Robotiq 3F Driver	Custom Python	Modbus TCP gripper client
diffusers	0.11.1	DDPM scheduler
trimesh	Latest	Mesh processing and proximity queries
Hydra	Latest	Training configuration management

Platform-Specific Adaptations

The following modifications were necessary to deploy GraspGen on the ARM64 Jetson platform and are documented here for reproducibility. The pointnet2_ops C++ extension was manually compiled for the aarch64 target. A torchvision compatibility fix was applied by wrapping the register_fake("torchvision::nms") call in a try/except block within _meta_registrations.py, as the NMS operator is absent from the Jetson PyTorch binary.

The LD_LIBRARY_PATH was configured to include the cuSPARSELt path required by PyTorch on JetPack. SAM2 dependencies iopath==0.1.10 and portalocker==3.2.0 were installed with the --no-deps flag to avoid conflicts with the Jetson build. The FCL collision library was not installed as it is incompatible with ARM64; all labelling operations that would ordinarily require FCL were replaced with the trimesh.proximity. Full installation instructions, training commands, and inference scripts are provided in the accompanying CD.

APPENDIX C

ROS2 Topic Summary

Table C lists the primary ROS2 topics in the pipeline. The full node graph, generated from the live ROS2 system using `rqt_graph`, is provided in Fig. 26.

TABLE 19: ROS2 TOPIC SUMMARY

Topic	Message Type	Publisher	Subscriber
/wrist_cam/rgb/image_raw	sensor_msgs/Image	depthai_ros_driver (wrist_cam node)	sam2_segmentation_node
/wrist_cam/stereo/image_raw	sensor_msgs/Image	depthai_ros_driver (wrist_cam node)	sam2_segmentation_node
/wrist_cam/rgb/camera_info	sensor_msgs/CameraInfo	depthai_ros_driver (wrist_cam node)	sam2_segmentation_node
/env_cam/points	sensor_msgs/PointCloud2	sam2_segmentation_node or sim_object_pc_publisher (legacy topic name)	graspgen_bridge_node
/graspgen/trigger	std_msgs/Empty	operator or grasp_executor_node	graspgen_bridge_node
/graspgen/grasp_pose	geometry_msgs/PoseStamped	graspgen_bridge_node	grasp_executor_node
/joint_states	sensor_msgs/JointState	joint_state_broadcaster	grasp_executor_node
/pick/start	std_msgs/Empty	operator	grasp_executor_node
/sam2/arm_ready	std_msgs/Empty	grasp_executor_node	sam2_segmentation_node
/sam2/scan	std_msgs/Empty	grasp_executor_node	sam2_segmentation_node
/sam2/reset	std_msgs/Empty	grasp_executor_node	sam2_segmentation_node
/planning_scene	moveit_msgs/PlanningScene	planning_scene_setup	move_group
/tf	tf2_msgs/TFMessage	robot_state_publisher or static_transform_publisher	all nodes

APPENDIX D

FCL: Free On-Policy Labelling-Proximity Criterion

The mathematical criterion used in the FCL-free labelling method is reproduced here for reference. Let p_{tcp} denote the TCP translation of the candidate grasp, d_{surface} the distance from p_{tcp} to the nearest point on the object mesh surface computed via `trimesh.proximity.closest_point`, and r_{obj} the object radius estimated as the maximum distance from the mesh centroid to any surface vertex. A grasp is labelled positive if:

$$d_{\text{surface}} \leq \text{clip}(r_{\text{obj}} \times 0.5, 1 \text{ cm}, 5 \text{ cm})$$

and the pre-grasp TCP position obtained by translating p_{tcp} by -5cm along the grasp approach axis does not intersect the object bounding volume. All other grasps are labelled **negative**. The labelled grasps are written into the H5 cache under the `pos_onpolicy` and `neg_onpolicy` keys, corresponding to indices 5 and 6 of the `data.discriminator_ratio` configuration used during on-policy discriminator training.

APPENDIX E

GraspGen Inference Parameters for Robotiq 3F

Table E summarises the key inference parameters used for all evaluation and real-robot experiments in this work.

TABLE 20: INFERENCE PARAMETERS

Parameter	Value
Diffusion denoising steps (T)	10
Grasps sampled per inference	87-108 scored candidates
Top-K grasps returned	100
Warm inference time (Jetson AGX Orin)	~1.85 seconds
Peak discriminator confidence score	Up to 0.997
Rotation representation	Lie algebra (so3)
Translation normalisation multiplier κ	Computed from training split statistics

All source code, training scripts, inference scripts, and configuration files are provided in the accompanying CD.

Declaration for Plagiarism Check

This is to certify that MTech thesis entitled “Deep Learning-Based Grasp Pose Detection on Point Clouds and Manipulating the Object using Generated Grasp Pose” submitted by **Ms. Anushka Singhania** (Reg. No.: 24-06-14) under the supervision of **Prasad Naik (DIAT Pune) & Atul Chavan(R&DE(E) DRDO Pune)** in the Department of Aerospace Engineering and Autonomous Systems is the original work done by the candidate.

We have read the provision of **DIAT (DU) Plagiarism Policy** and it is certified that all the conditions prescribed in the aforesaid policy are complied with in respect of above mentioned MTech thesis.

The thesis has been checked for the plagiarism and the report is submitted along with the thesis for further processing.

- ✓ Originality Content (Including the content from his own publication)-
- ✓ Similarity content reproduction from other sources -

We are aware that any issue related to plagiarism in future will have to be addressed by the candidate and the supervisor (s) concerned.

Name of student	Anushka Singhania
Signature	
Date	

Name of Supervisors	
Prasad Naik	Atul Chavan
Signature	Signature
Date	